

IT3212: Data Pre-processing and Calibration

Kshitij (K) Sharma

Department of Computer and Information Science
Faculty of Information Technology and Electrical Engineering

General Data preprocessing and calibration

- Four key steps
 - Data cleaning
 - Data integration
 - Data transformation
 - Data/Dimensionality reduction

Data Cleaning

- Removing Duplicates: Identifying and removing duplicate records in a customer database.
- For instance, when merging customer data from different sources, you might find that the same customer is listed multiple times with slight variations in their

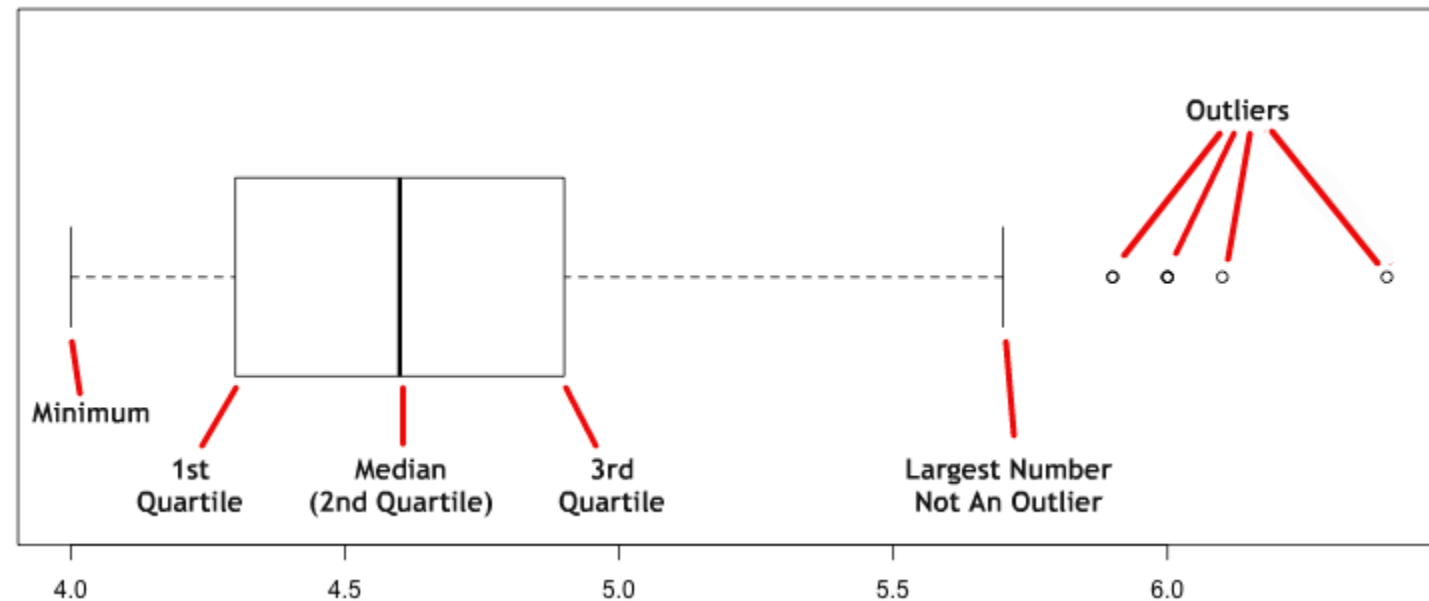
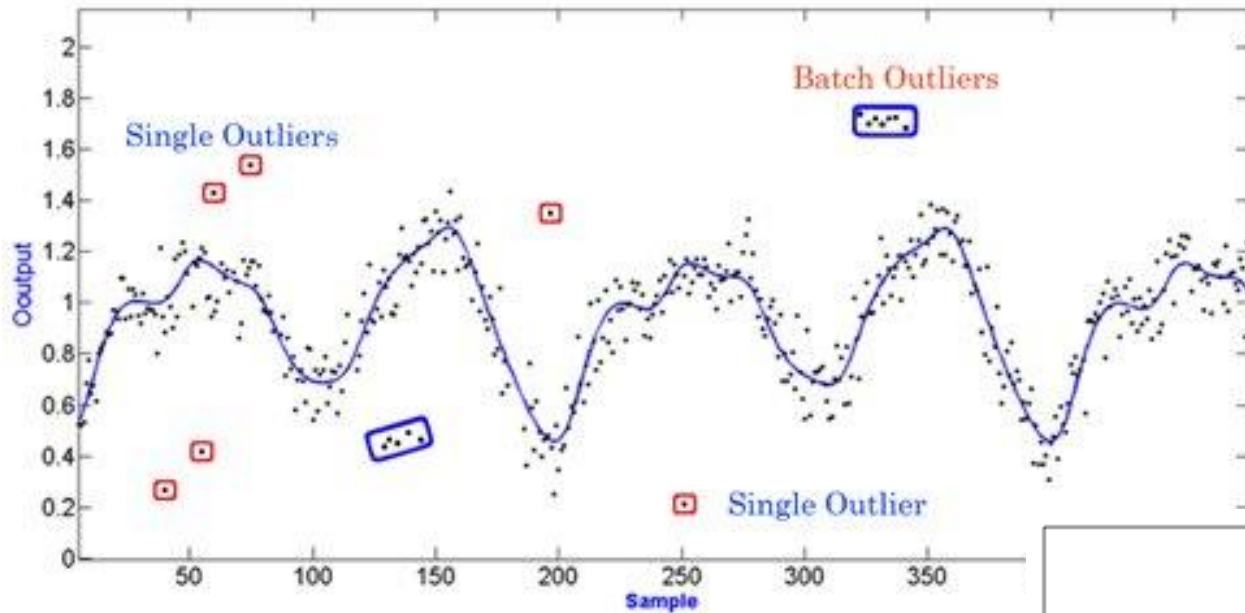
Data Cleaning – missing values

- Handling Missing Values: Dealing with missing data in a dataset. For instance, in a survey dataset, some respondents might have skipped certain questions, leading to missing values that need to be imputed or handled
- Two primary ways
 - Ignore → if the dataset is large
 - Fill in the missing values

Data Cleaning – outliers

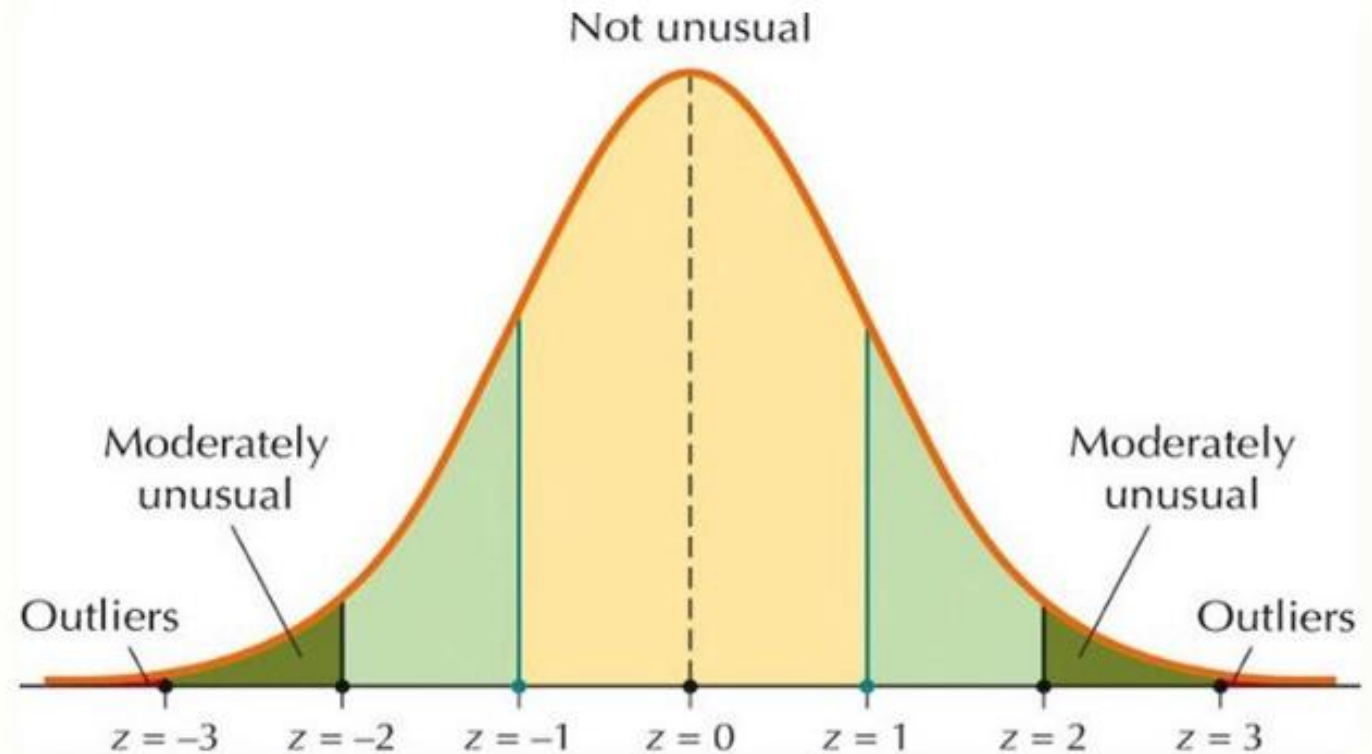
- Stock price → jump/drop
- Heavy credit card transactions
- Heart rate spikes
- Low test scores
- Manufacturing defects
- Extreme weather conditions
- Viral posts
- Traffic congestions
- Flight delays

Data Cleaning – outlier removal

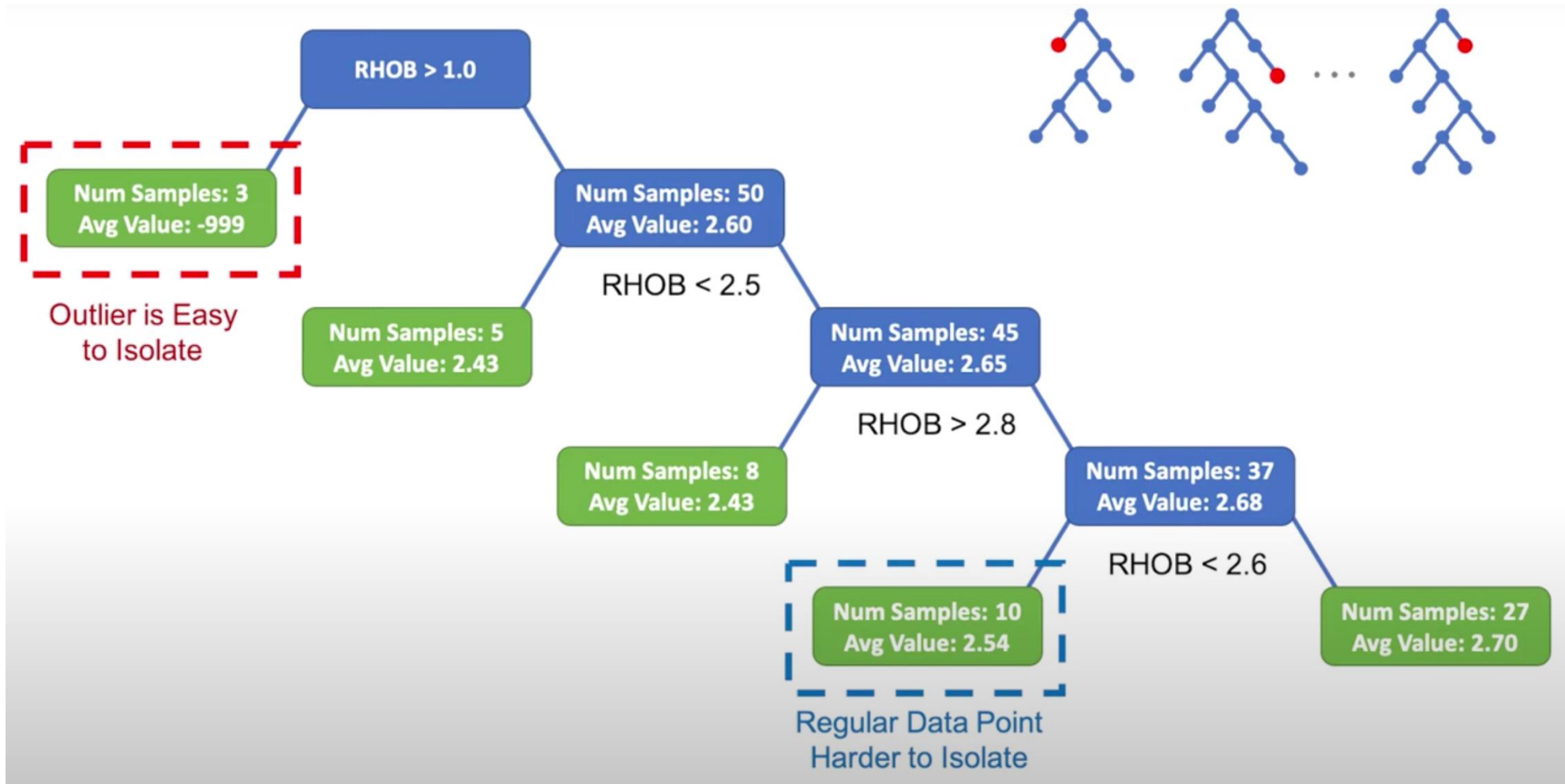


Data Cleaning – outlier removal

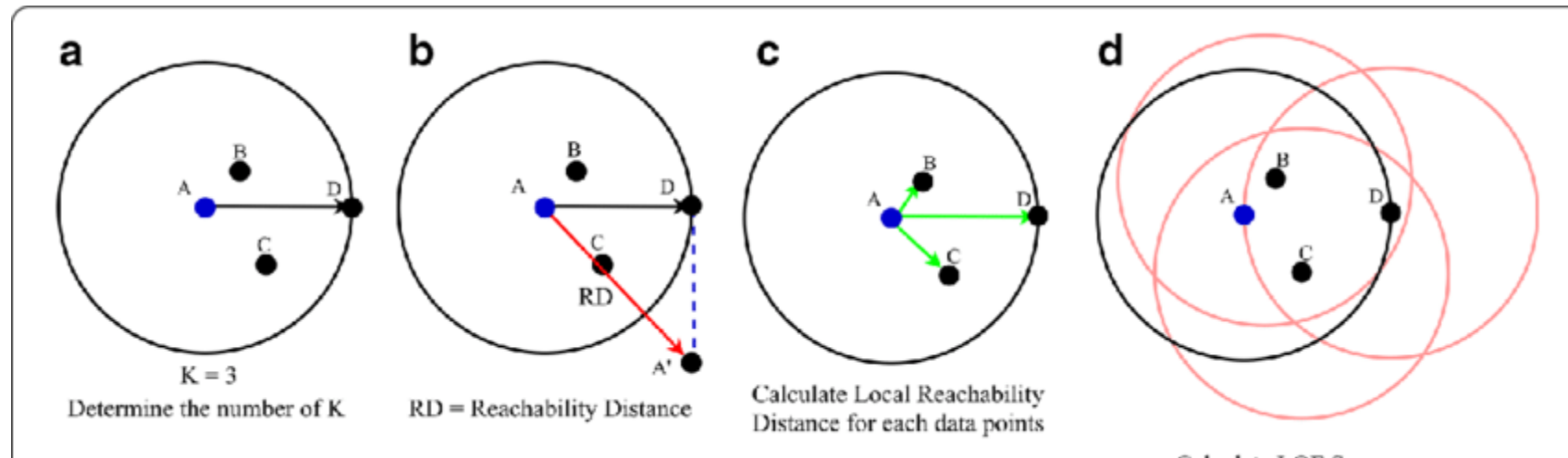
- AKA → “binning”
- This can be done with the mean and standard deviation
- Or with the median and the deviation from it
- Or with the percentiles (higher/lower)



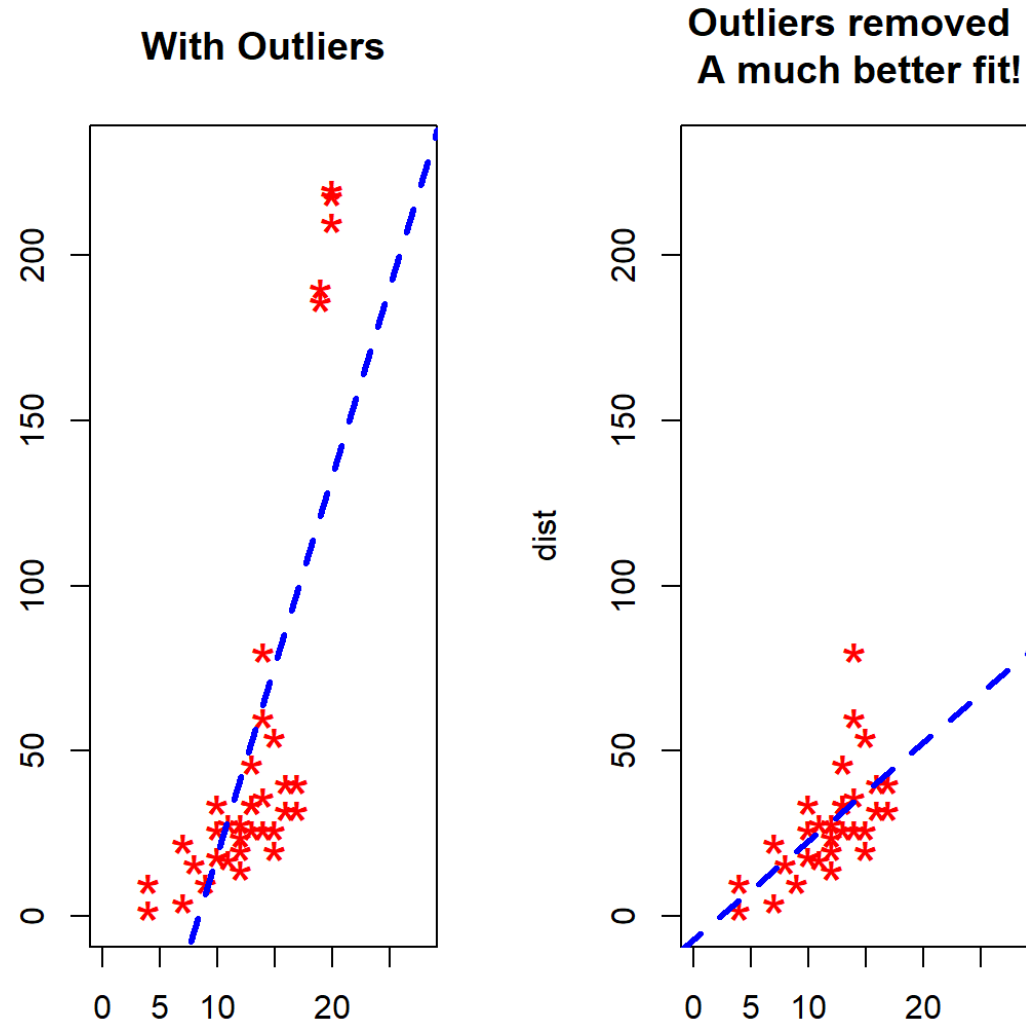
Data Cleaning – outlier removal



Data Cleaning – outlier removal



Data Cleaning – outlier removal



Data Cleaning – text data

- **Standardizing Text Data:** Standardizing text data, like addresses or product names. In a retail database, you might have inconsistent product names due to typos or variations in formatting. Cleaning involves normalizing these names for consistency.
- **Correcting Typos and Inconsistencies:** Correcting typos and inconsistencies in data. For example, in a list of customer email addresses, you might encounter misspelled domains or incorrect email formats that need to be fixed.

Data Cleaning – text data

- **Parsing and Extracting Information:** Parsing unstructured text data to extract structured information. This is common in fields like natural language processing (NLP), where you need to extract specific entities or facts from text documents.
- **Handling Special Characters and Encoding Issues:** Identifying and handling special characters or encoding issues that can cause problems during data analysis or processing, especially when dealing with text data from various sources.

Data Cleaning

- **Dealing with Inconsistent Categorical Values:** Standardizing categorical variables. In a dataset with a "gender" column, you might have variations like "male," "M," and "Man" that should be cleaned and consolidated into a consistent format.
- **Date and Time Data Cleaning:** Cleaning and formatting date and time data. This includes handling different date formats, time zones, and ensuring data consistency.

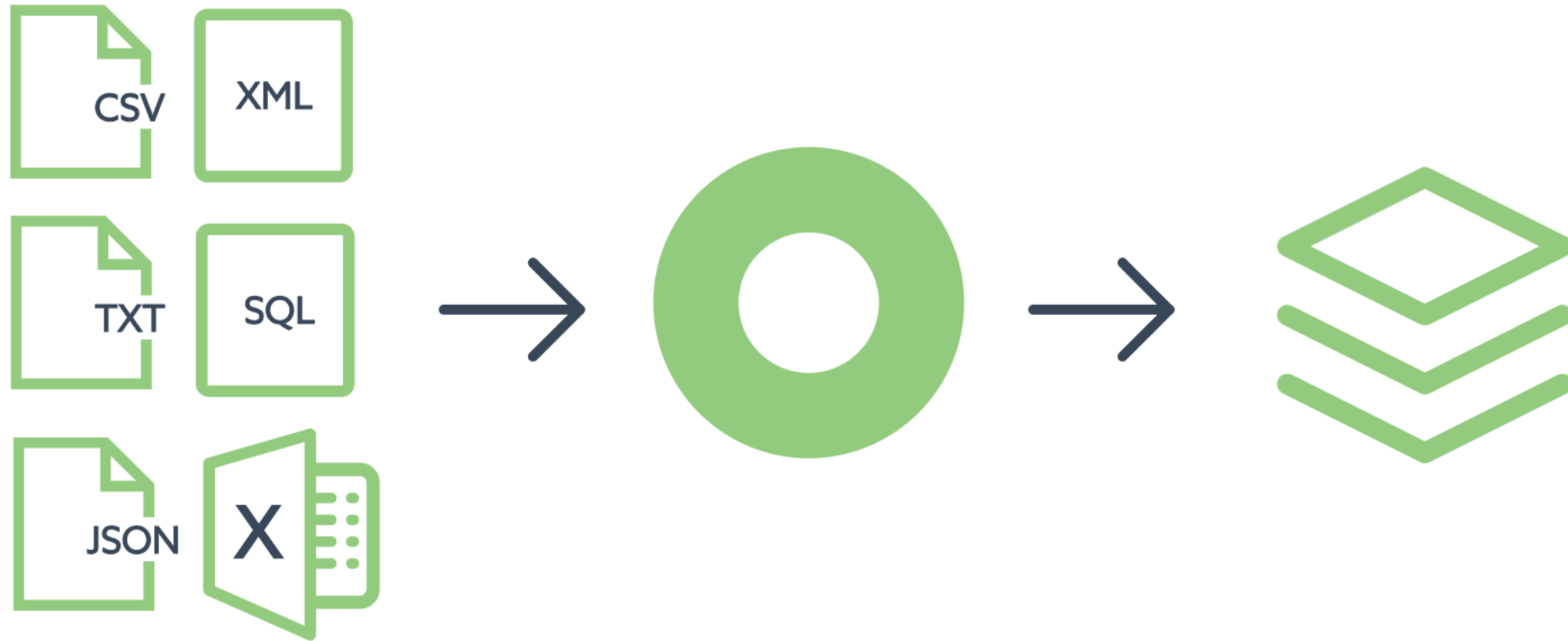
Data Cleaning

- **Dealing with Inconsistent Data Entry:** Addressing variations in data entry methods. For example, different people entering data might use different abbreviations or formats for states or countries, which need to be standardized.
- **Cleaning Web Scraped Data:** When scraping data from websites, you may encounter irregularities or unexpected changes in the website structure. Cleaning involves adapting your scraping code to handle such situations.

Data Cleaning

- **Converting Units:** Converting data from one unit to another. For instance, if you have a dataset with temperatures recorded in Fahrenheit but your analysis requires Celsius, you'll need to convert the values.

Data integration



Data integration

- **Manual data integration** is labour intensive, involving hours of copy-pasting, CSV uploads, spreadsheets, frustration, time-wasted, etc.
- **Integration libraries** full of APIs are used to automatically sync with 'one-click'.
- **Custom APIs and webhooks** are created and maintained by an engineering team to create data-flows and enable integration.

Manual



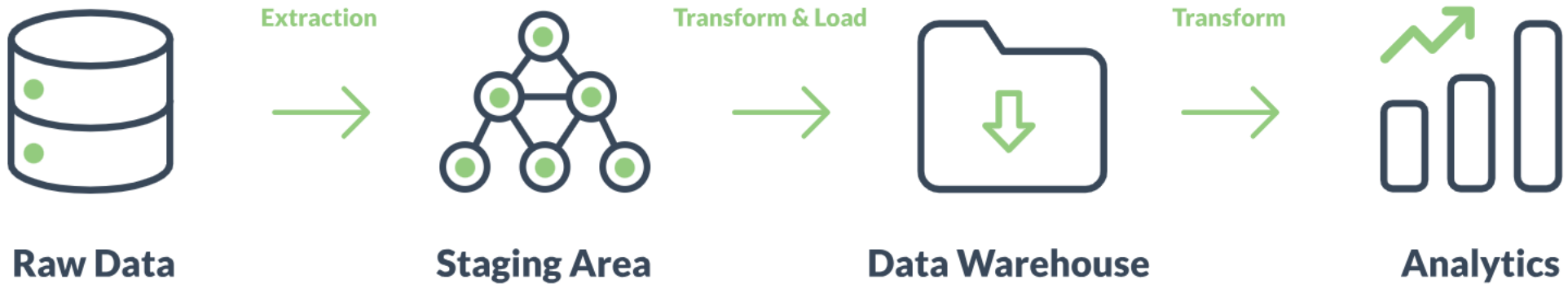
Automated



Engineered



Data integration ETL



Extract - the tool extracts the necessary data from the source by using connectors or the source API.

Transform - due to data coming from multiple points of origin, it is then standardized (formatting values such as currencies, time-zones, units of measurement), enriched and validated (missing values filled and duplicates discounted) to ensure consistency at the end-point.

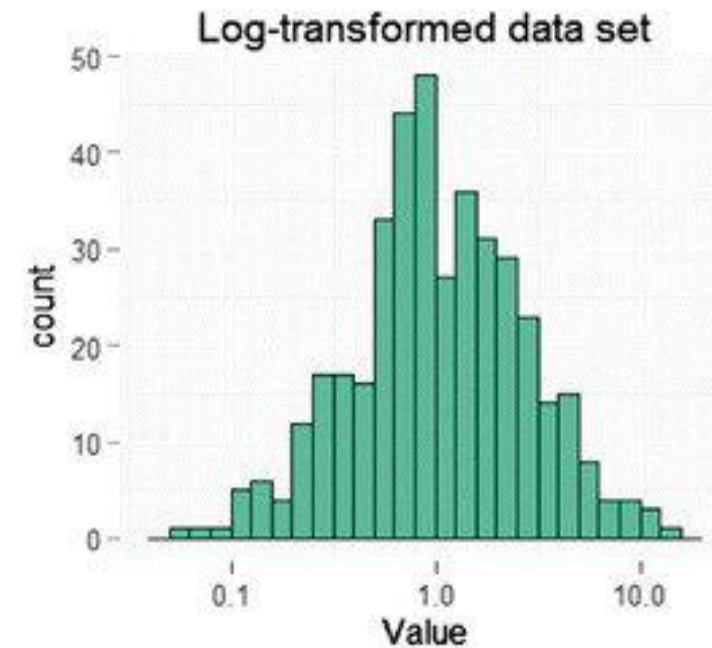
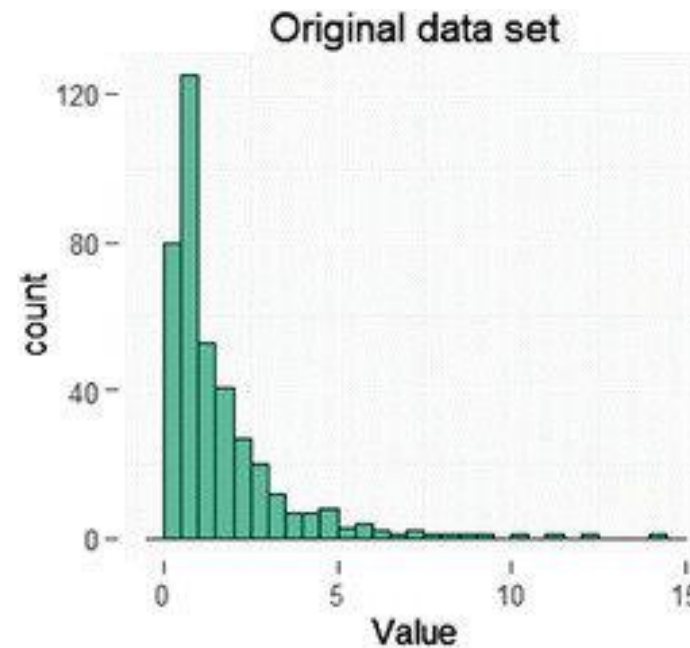
Load - the newly transformed data is loaded into the central location to be used for analysis and reporting purposes.

Data transformation – Normalization

- **Scaling numerical data** to a standard range (e.g., 0 to 1) to ensure that all features have equal influence in machine learning models.
- Common techniques include **Min-Max scaling** and **Z-score normalization**.

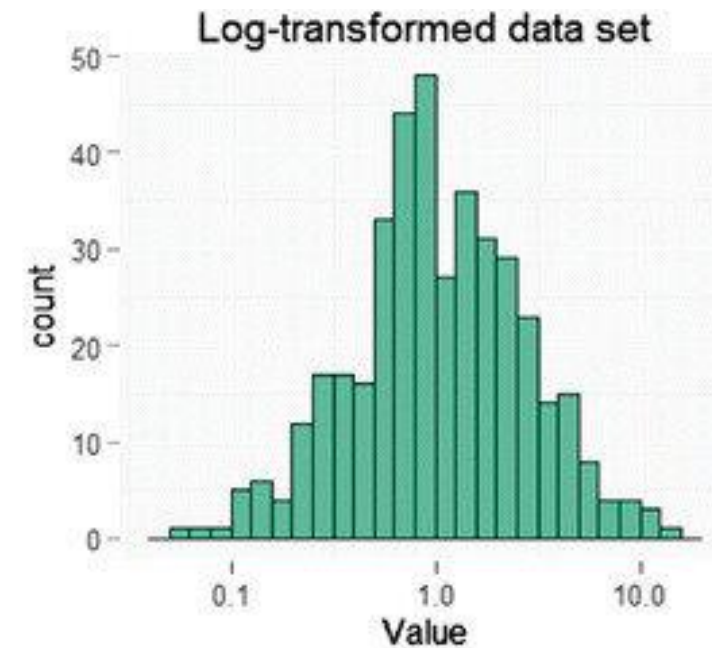
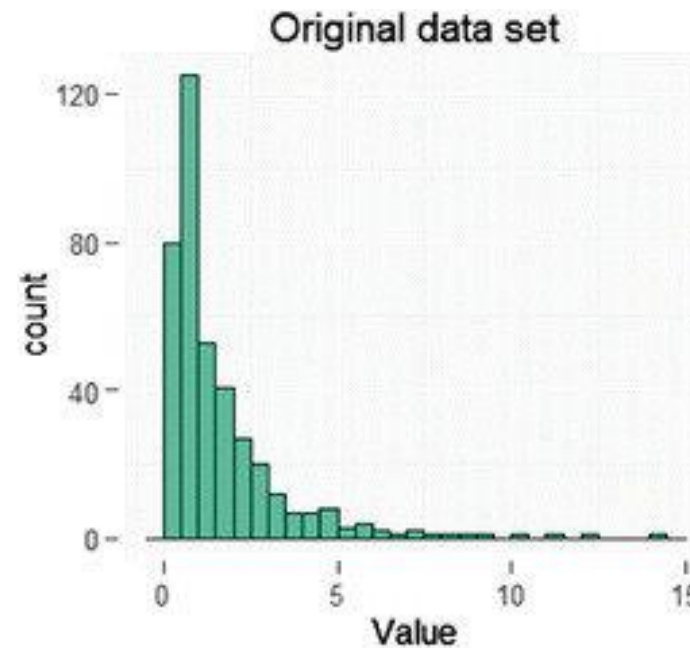
Data transformation – log transform

- Applying the logarithm function to data to reduce the impact of extreme values and make the data more normally distributed.
- Useful for data with a skewed distribution, such as income or population data.



Data transformation – exponential transform

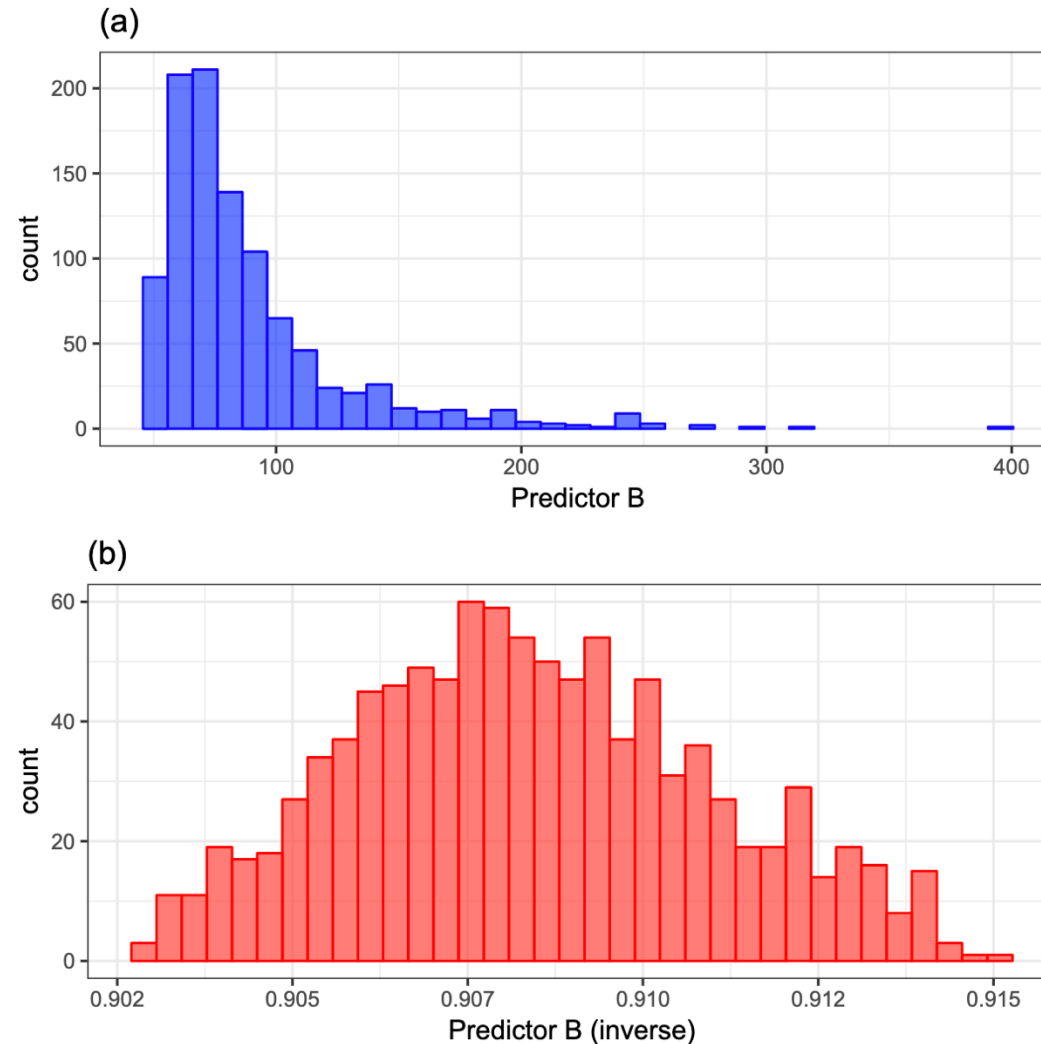
- Applying the logarithm function to data to reduce the impact of extreme values and make the data more normally distributed.
- Useful for data with a skewed distribution, such as income or population data.



The opposite of logarithmic transformation, used to amplify differences in data, especially when dealing with exponential growth or decay.

Data transformation – box-cox transform

- A family of power transformations used to stabilize variance and make data more closely resemble a normal distribution.
- It is especially useful when dealing with non-normally distributed data.



Data transformation – one hot encoding

Original Data

Team	Points
A	25
A	12
B	15
B	14
B	19
B	23
C	25
C	29



One-Hot Encoded Data

Team_A	Team_B	Team_C	Points
1	0	0	25
1	0	0	12
0	1	0	15
0	1	0	14
0	1	0	19
0	1	0	23
0	0	1	25
0	0	1	29

Data transformation – label encoding

SAFETY-LEVEL (TEXT)	SAFETY-LEVEL (NUMERICAL)
None	0
Low	1
Medium	2
High	3
Very-High	4

BRIDGE-TYPE (TEXT)	BRIDGE-TYPE (NUMERICAL)
Arch	0
Beam	1
Truss	2
Cantilever	3
Tied Arch	4
Suspension	5
Cable	6

Data transformation – label encoding

Label Encoding

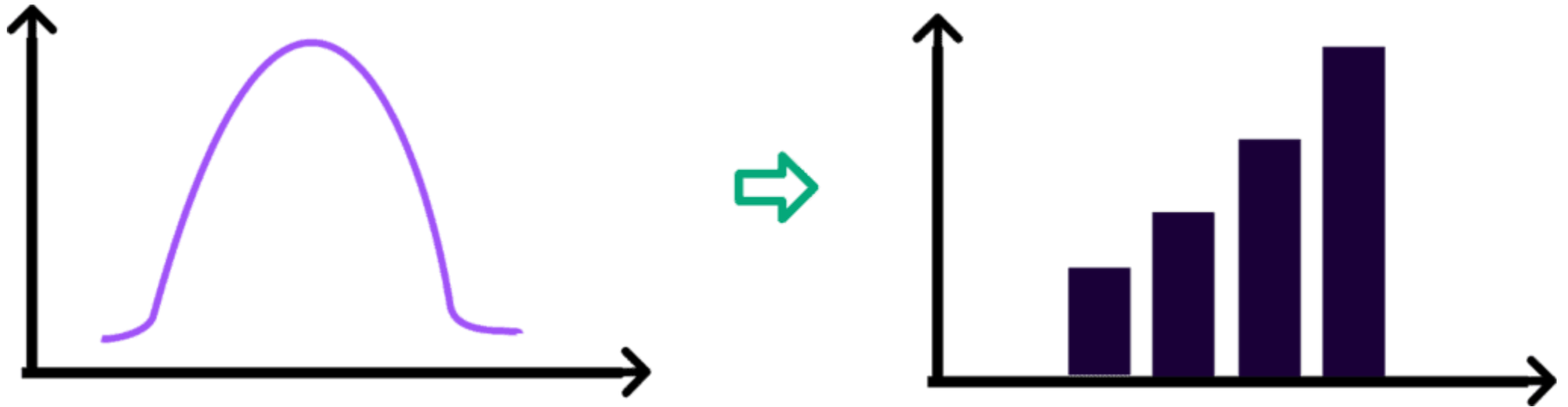
Food Name	Categorical #	Calories
Apple	1	95
Chicken	2	231
Broccoli	3	50



One Hot Encoding

Apple	Chicken	Broccoli	Calories
1	0	0	95
0	1	0	231
0	0	1	50

Data transformation – discretization



Data transformation – discretization

- **Supervised**

- Using class boundaries: This method involves sorting the values of the continuous variable and placing breakpoints between values belonging to different classes.
- Entropy-based discretization: This method approaches the iterative partitioning of a continuous attribute by bisecting the initial values so that the resulting interval gives the minimum entropy.

- **Unsupervised**

- Equal-interval (equal-width) binning: split the whole range of numbers into intervals with equal size.
- Equal-frequency (equal-depth) binning: use intervals containing an equal number of values.
- Cluster analysis: Clustering algorithms may be implemented by partitioning the continuous variable values into clusters or classes to isolate a computational feature.

Data transformation – text data

- **Tokenization:** Splitting text into words or tokens.
- **Stopword Removal:** Removing common words (e.g., "the," "and") that carry little information.
- **Stemming or Lemmatization:** Reducing words to their base or root form.

Data transformation – text data

- **Tokenization:**
Splitting text into words or tokens.

Input Text

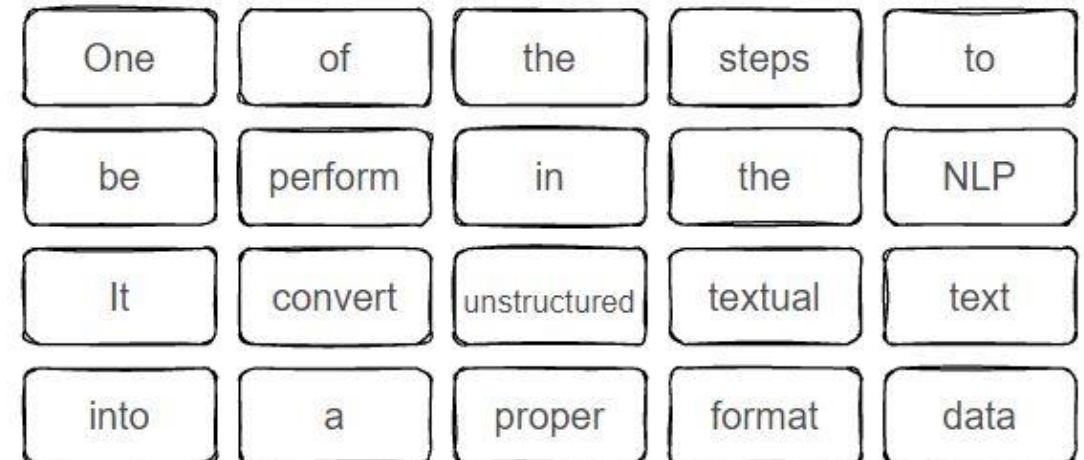
One of the steps to be perform in the NLP. It convert unstructured textual text into a proper format of data.

**Sentence
Tokenization**

One of the steps to be perform in the NLP.

It convert unstructured textual text into a proper format of data.

**Word
Tokenization**



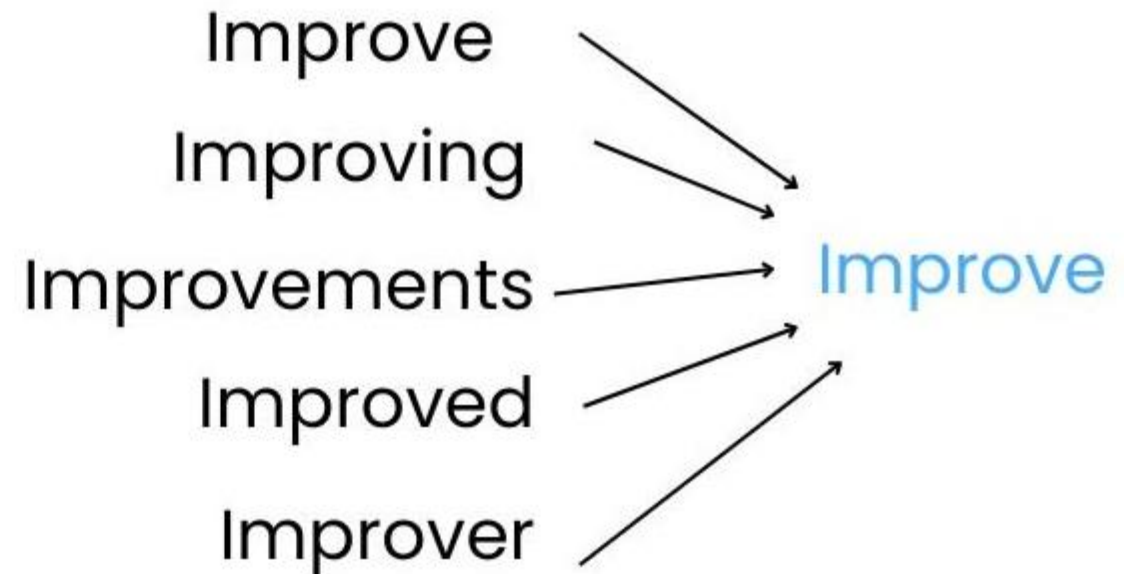
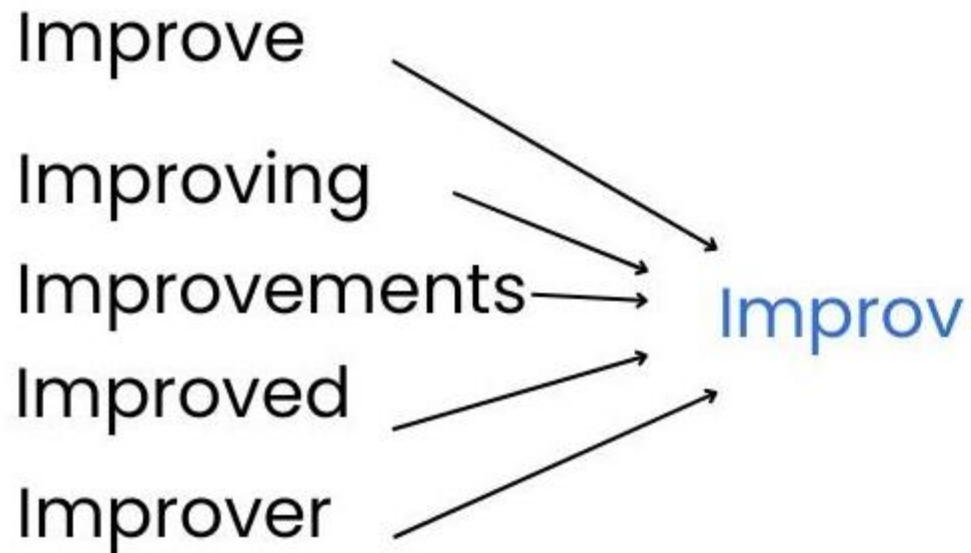
Data transformation – text data

- **Stopword Removal:**
Removing common words (e.g., "the," "and") that carry little information.

a	did	herself	not	the	we've
about	didn't	him	of	their	were
above	do	himself	off	theirs	weren't
after	does	his	on	them	what
again	doesn't	how	once	themselves	what's
against	doing	how's	only	then	when
all	don't	i	or	there	when's
am	down	i'd	other	there's	where
an	during	i'll	ought	these	where's
and	each	i'm	our	they	which
any	few	i've	ours	they'd	while
are	for	if	ourselves	they'll	who
aren't	from	in	out	they're	who's
as	further	into	over	they've	whom
at	had	is	own	this	why
be	hadn't	isn't	same	those	why's
because	has	it	shan't	through	with
been	hasn't	it's	she	to	won't
before	have	its	she'd	too	would
being	haven't	itself	she'll	under	wouldn't
below	having	let's	she's	until	you
between	he	me	should	up	you'd
both	he'd	more	shouldn't	very	you'll
but	he'll	most	so	was	you're
by	he's	mustn't	some	wasn't	you've
can't	her	my	such	we	your
cannot	here	myself	than	we'd	yours
could	here's	no	that	we'll	yourself

Data transformation – text data

- **Stemming or Lemmatization:** Reducing words to their base or root form.

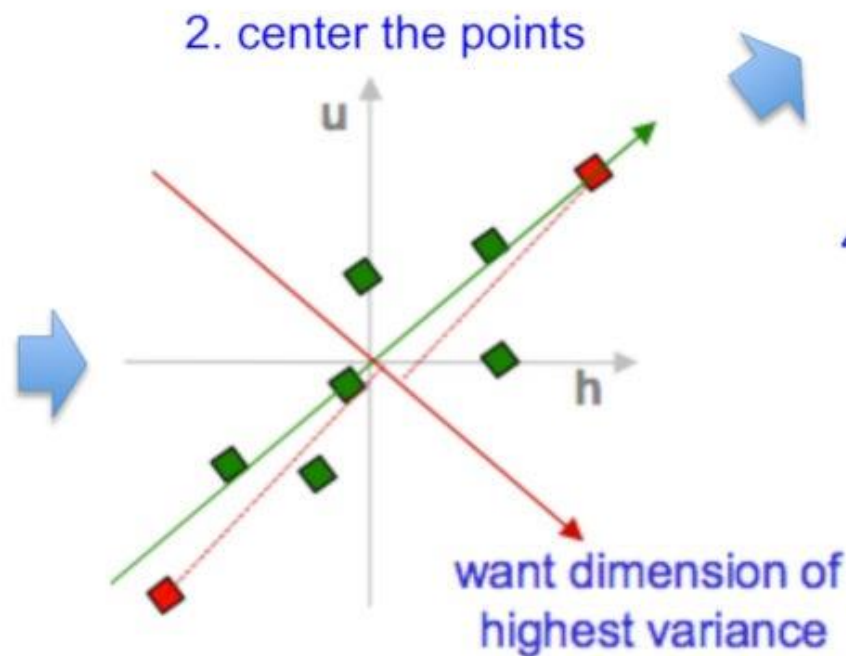
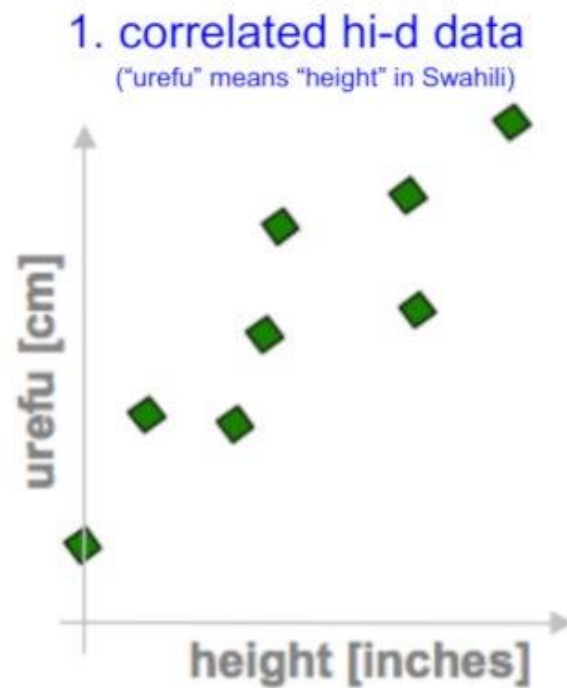


Data transformation

- **Time Series Decomposition**: Breaking down time series data into its constituent components such as trend, seasonality, and residual (error) for more meaningful analysis.
- **Feature Engineering**: Creating new features from existing ones based on domain knowledge or specific requirements.
- **Principal Component Analysis (PCA)**: Reducing the dimensionality of data by transforming it into a set of uncorrelated variables (principal components).

Data/Dimensionality reduction – PCA

PCA in a nutshell



3. compute covariance matrix

$$\begin{matrix} & h & u \\ \begin{matrix} h \\ u \end{matrix} & \begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \end{matrix} \rightarrow \text{cov}(h,u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

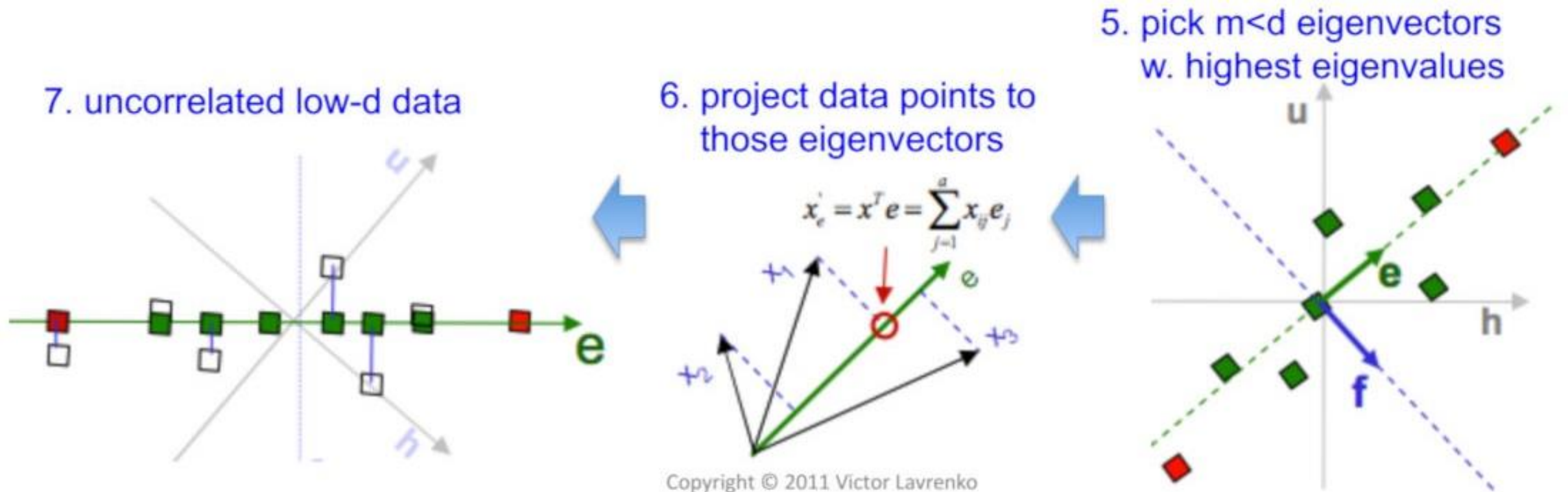
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{bmatrix} e_h \\ e_u \end{bmatrix} = \lambda_e \begin{bmatrix} e_h \\ e_u \end{bmatrix}$$

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{bmatrix} f_h \\ f_u \end{bmatrix} = \lambda_f \begin{bmatrix} f_h \\ f_u \end{bmatrix}$$

`eig(cov(data))`

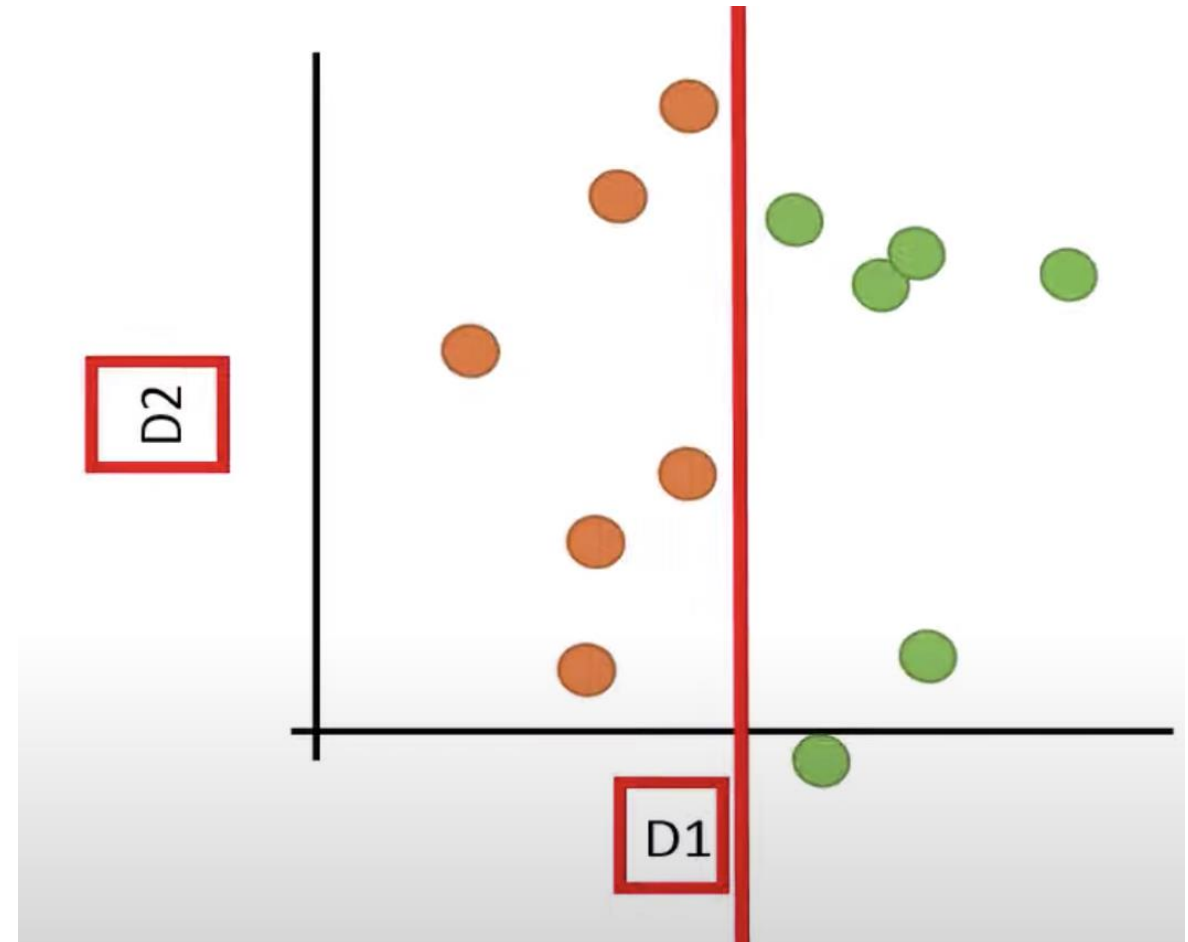
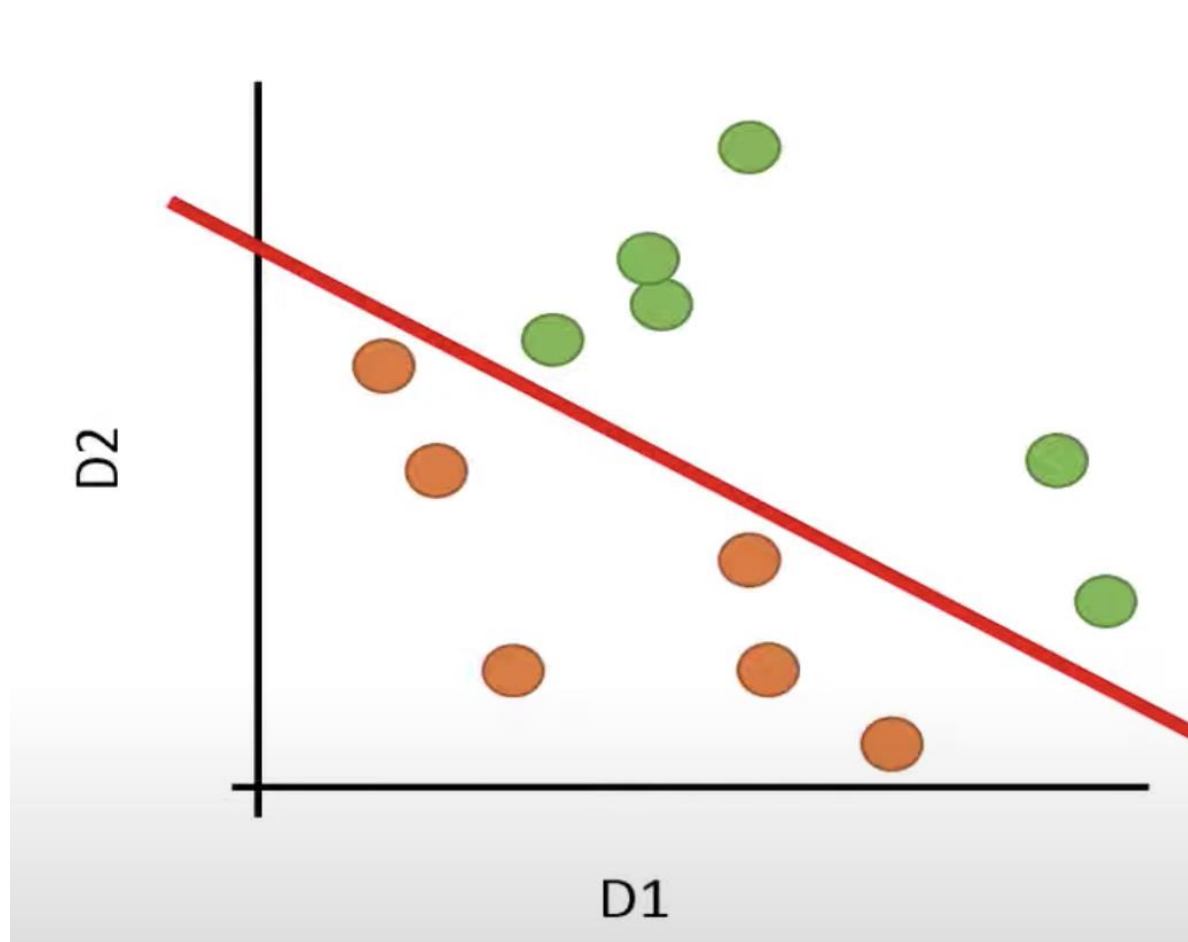
Data/Dimensionality reduction – PCA



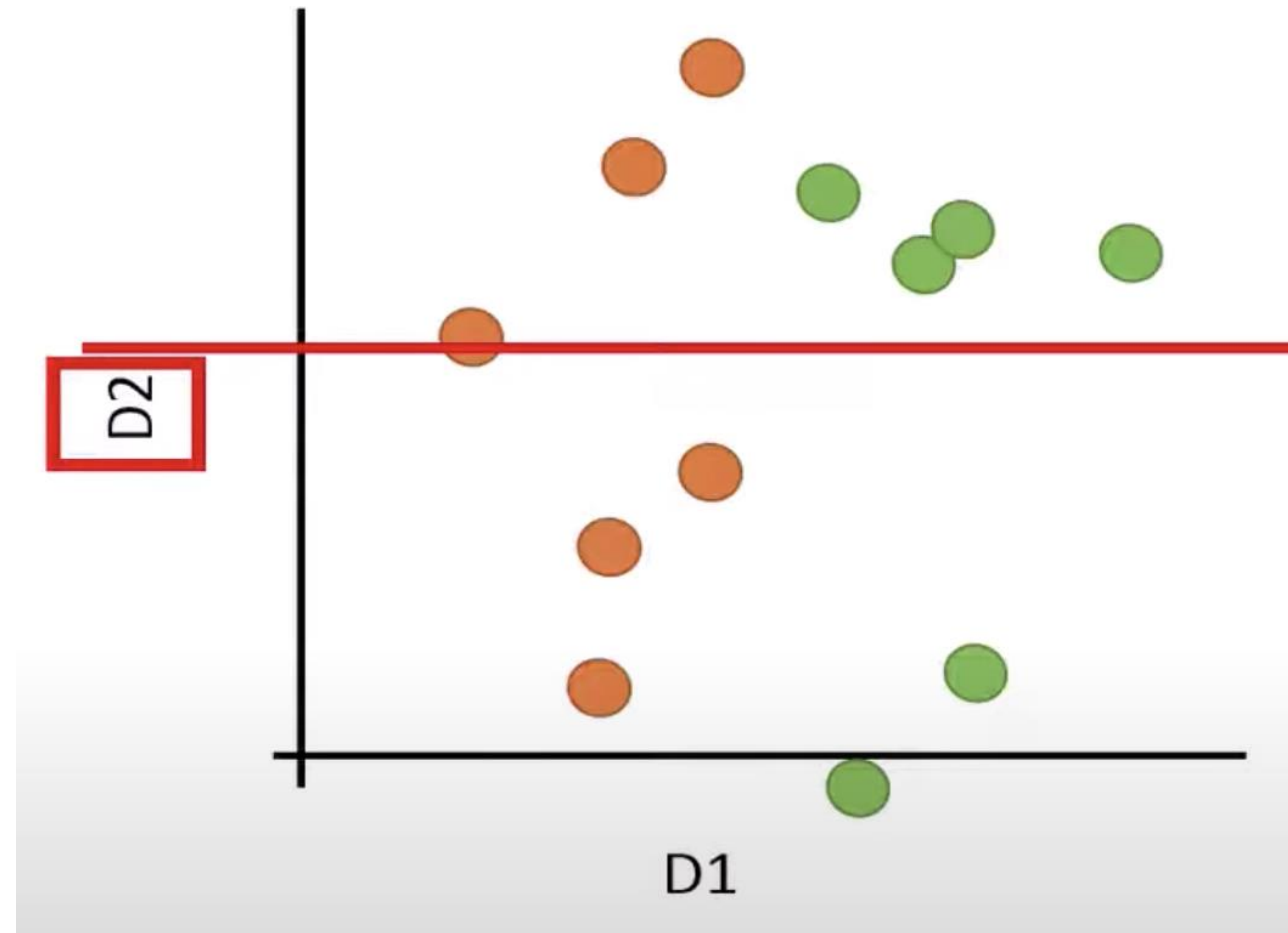
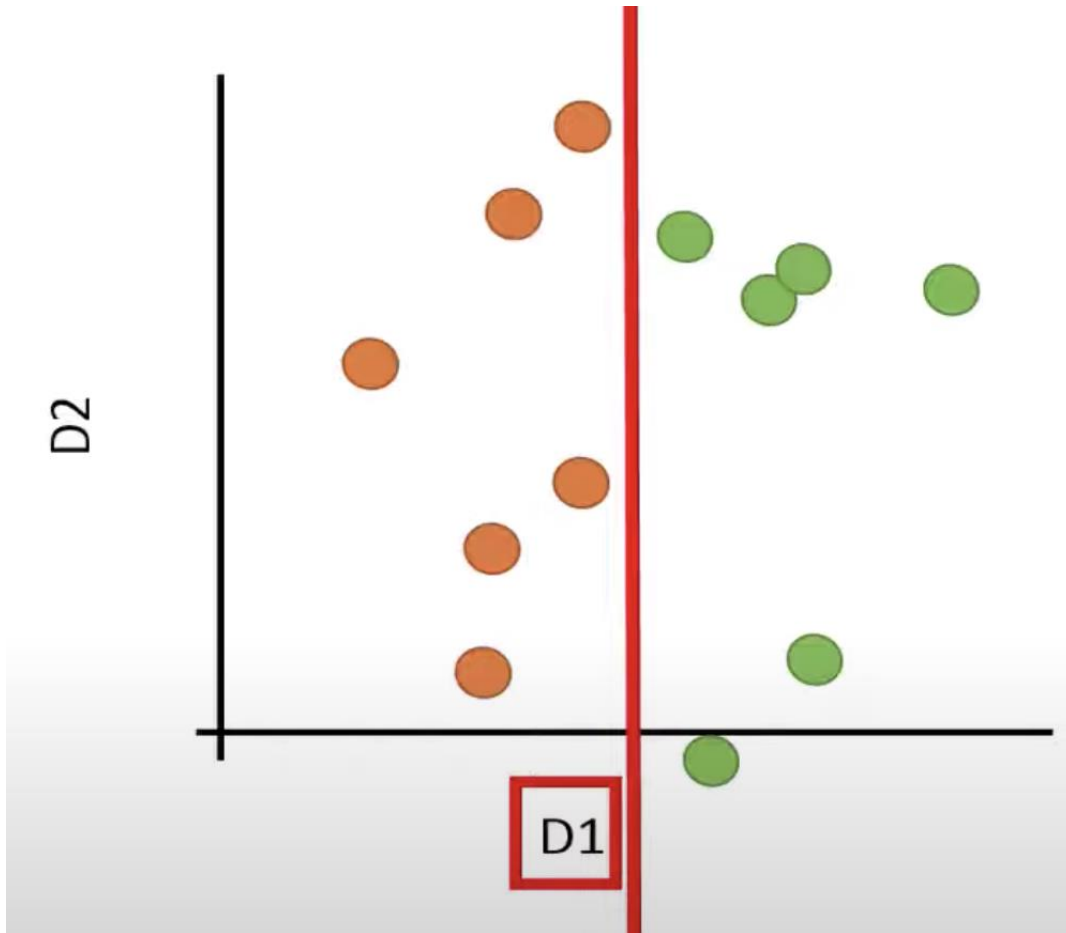
Data/Dimensionality reduction – PCA



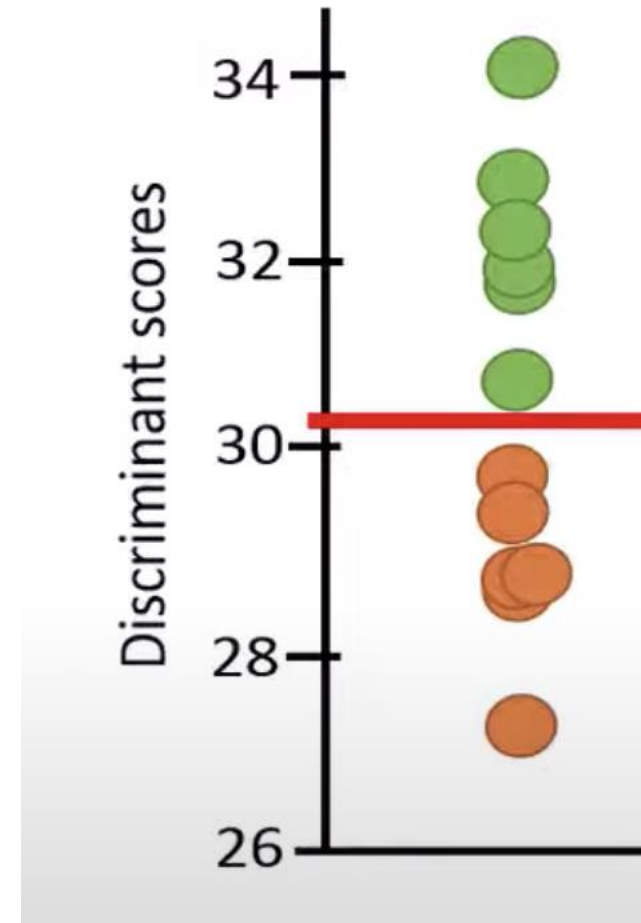
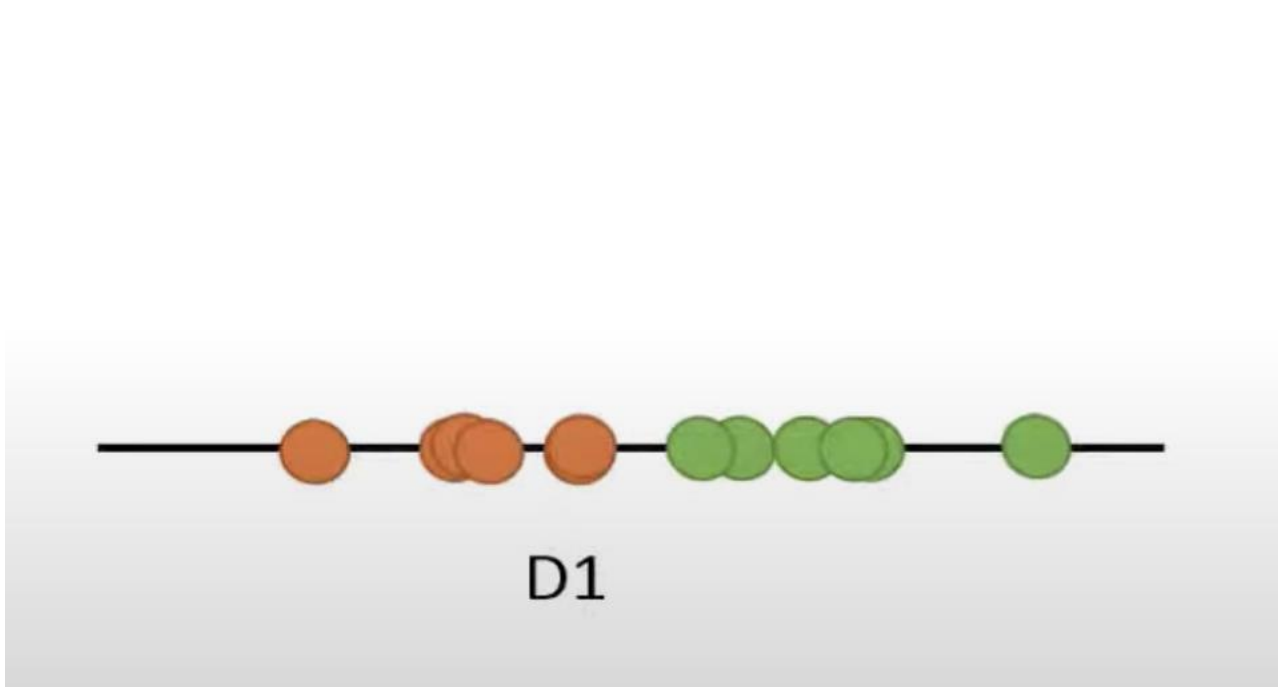
Data/Dimensionality reduction – LDA



Data/Dimensionality reduction – LDA



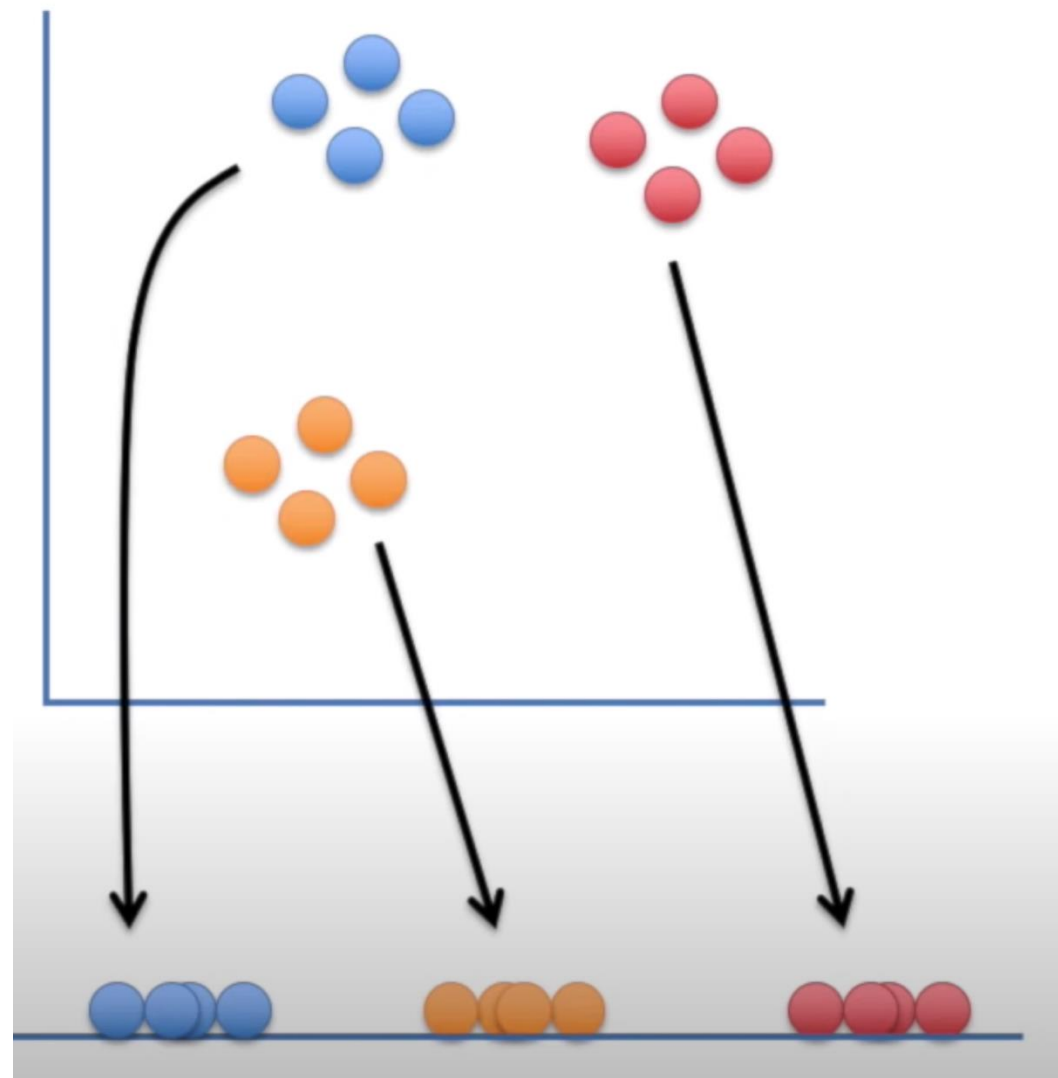
Data/Dimensionality reduction – LDA



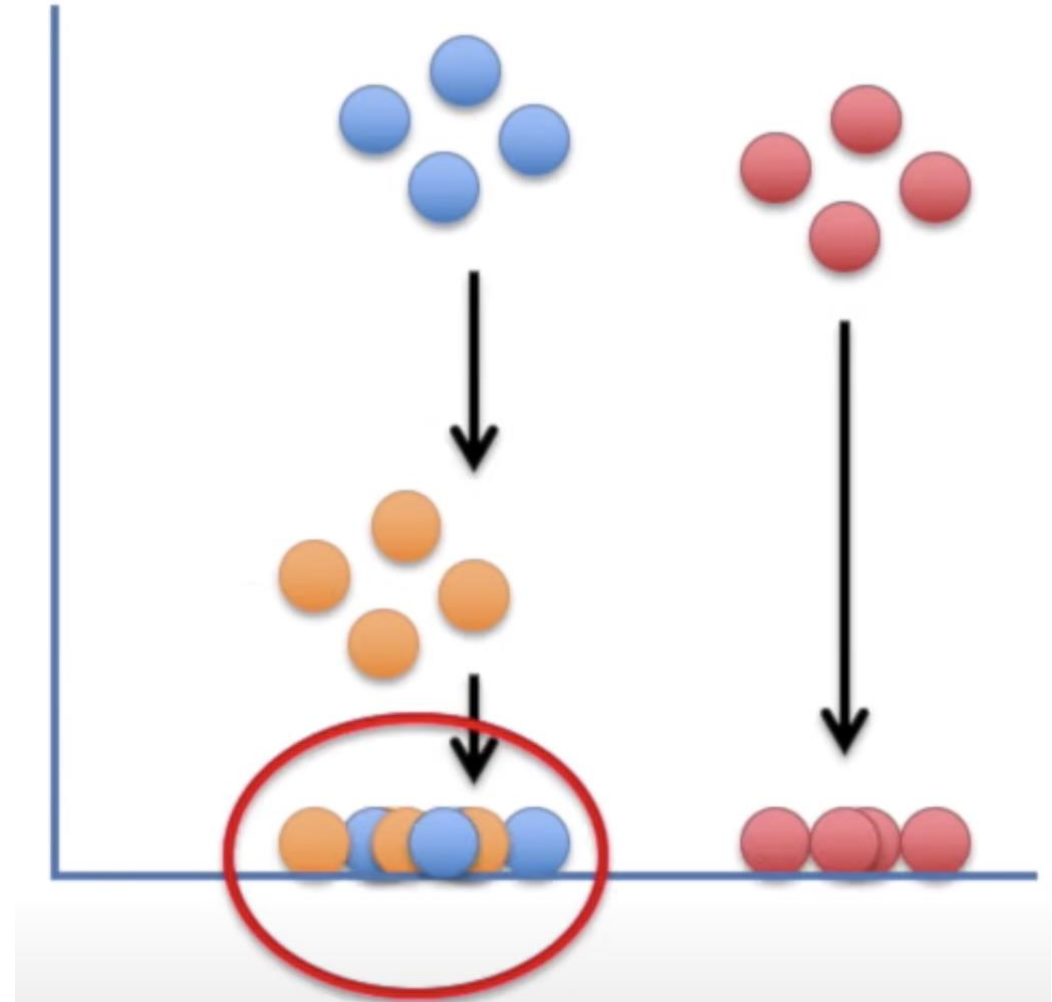
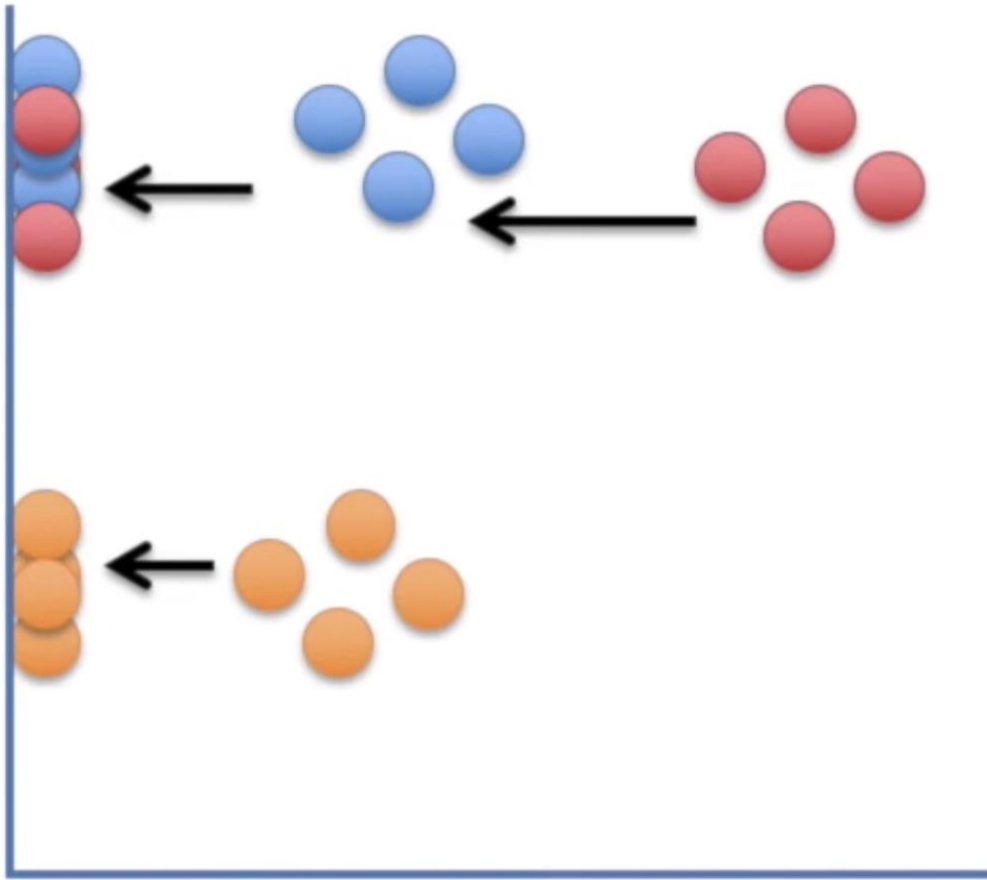
Data/Dimensionality reduction – LDA Vs PCA

- PCA ignores class labels and focuses on finding the principal components that maximizes the variance in a given data. Thus it is an **unsupervised** algorithm. On the other hand, LDA is a **supervised** algorithm that intends to find the linear discriminants that represents those axes which maximize separation between different classes.
- LDA performs better multi-class classification tasks than PCA. However, PCA performs better when the sample size is comparatively small. An example would be comparisons between classification accuracies that are used in image classification.
- Both LDA and PCA are used in case of dimensionality reduction. PCA is first followed by LDA.

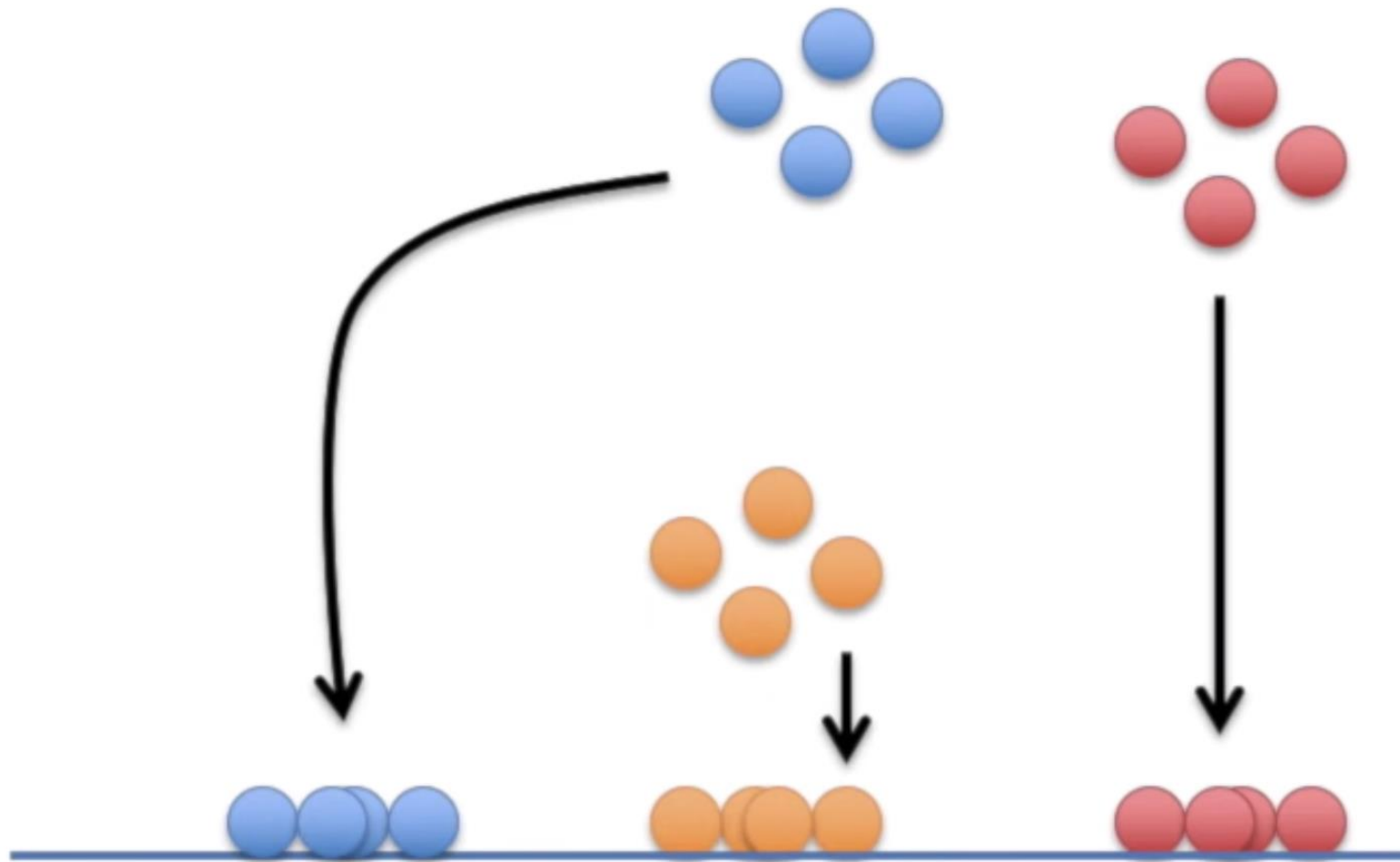
Data/Dimensionality reduction – t-SNE



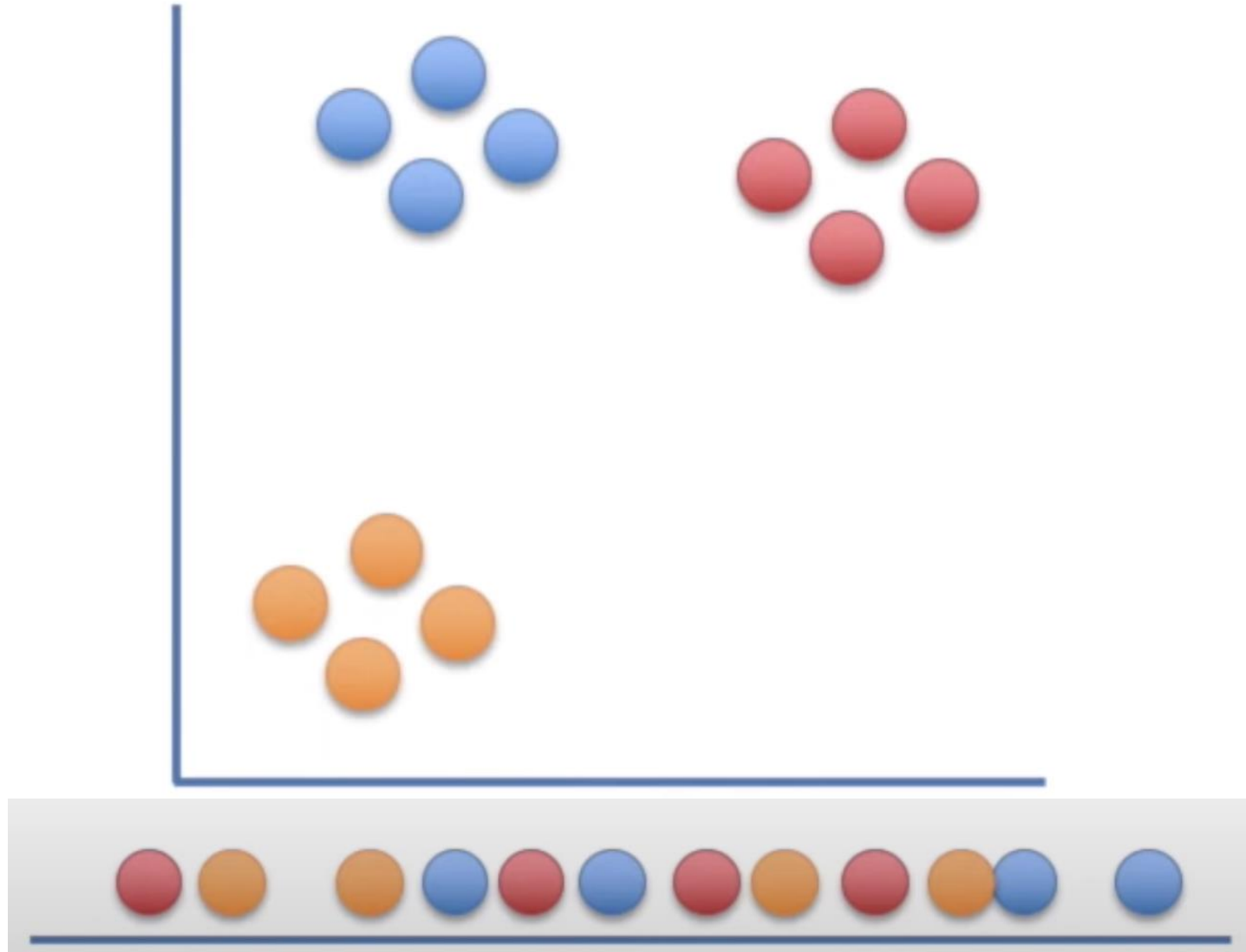
Data/Dimensionality reduction – t-SNE



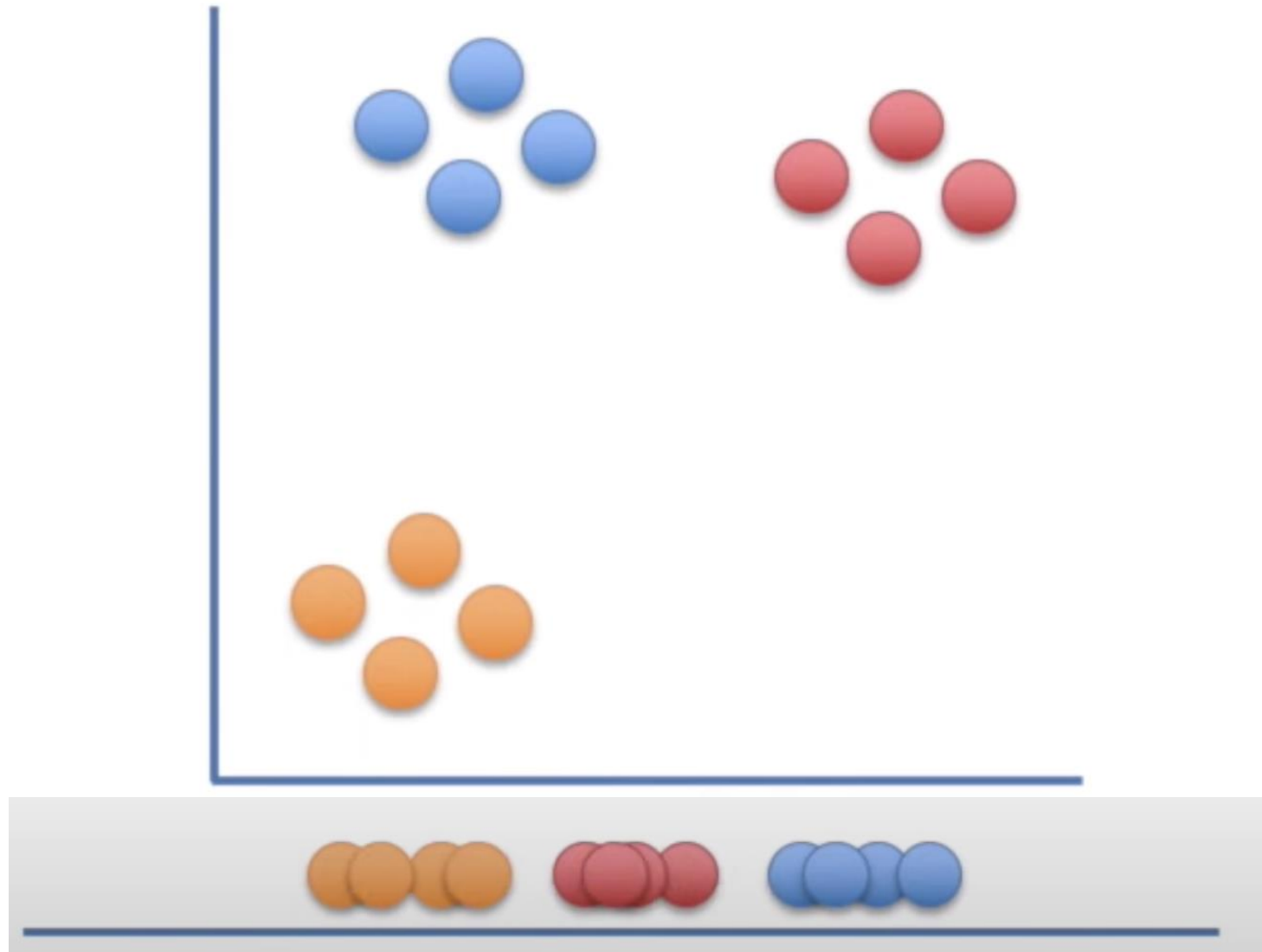
Data/Dimensionality reduction – t-SNE



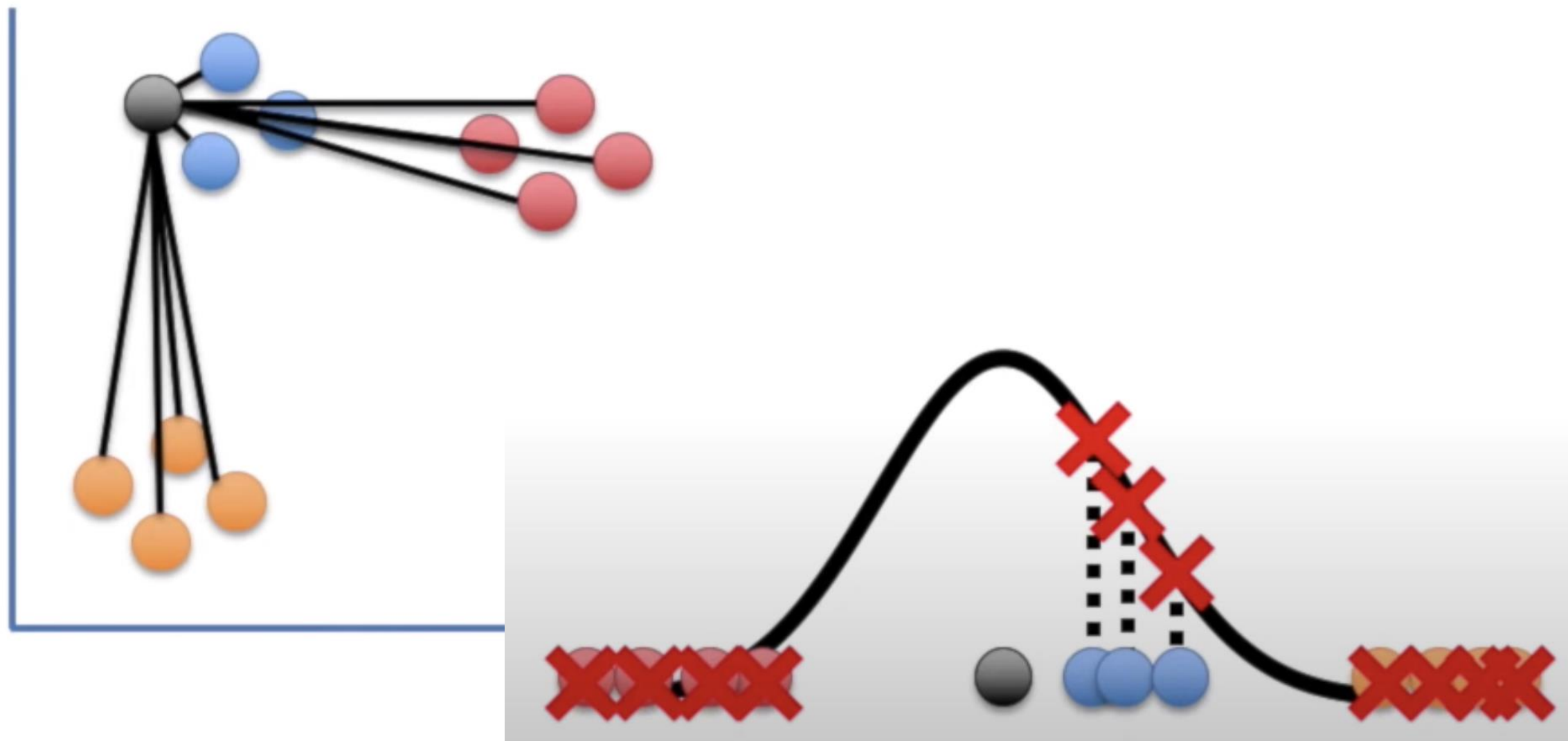
Data/Dimensionality reduction – t-SNE



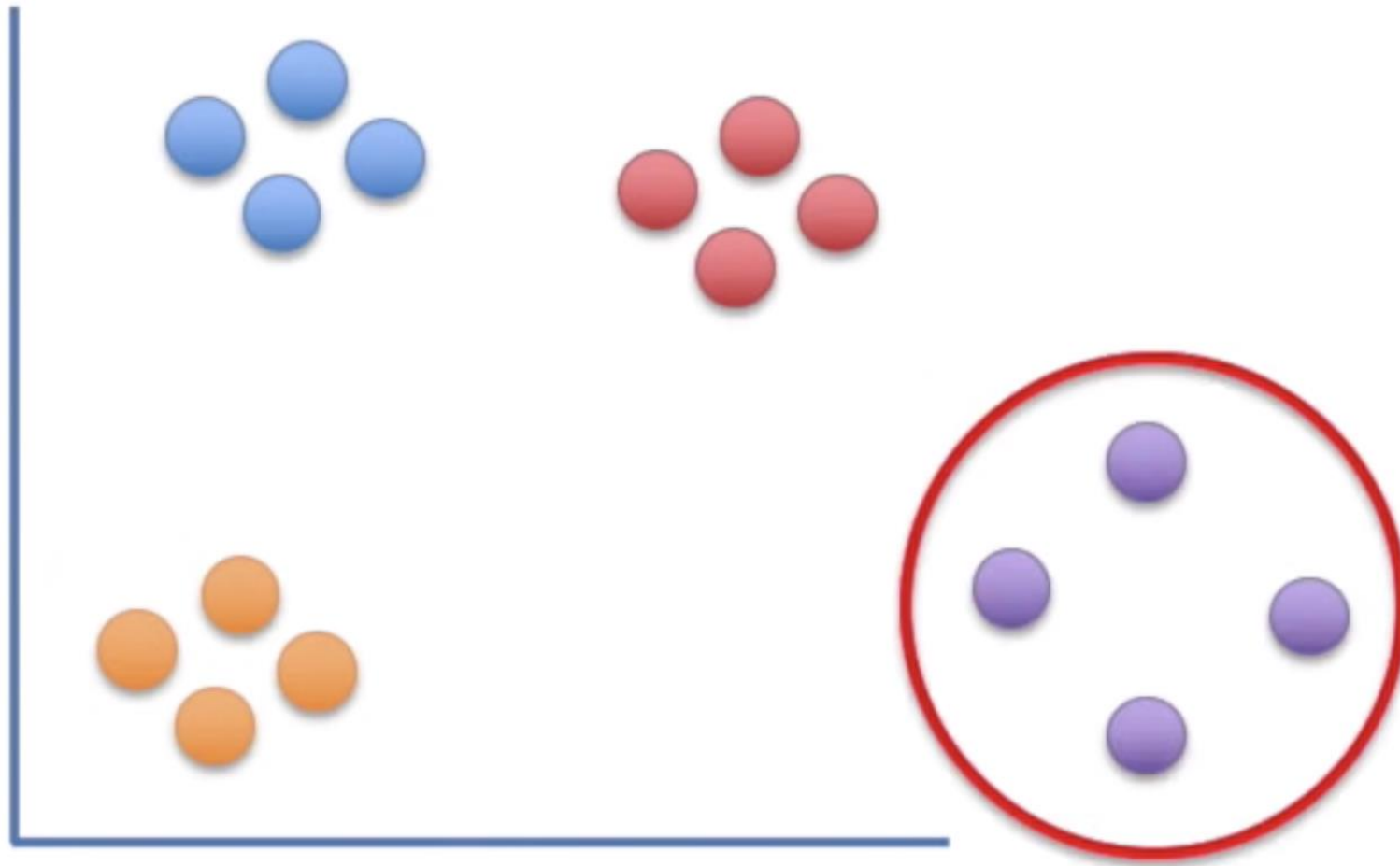
Data/Dimensionality reduction – t-SNE



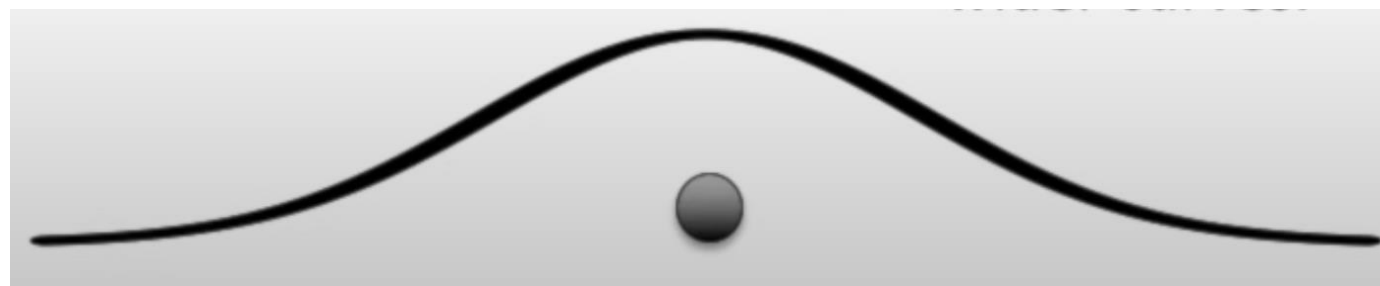
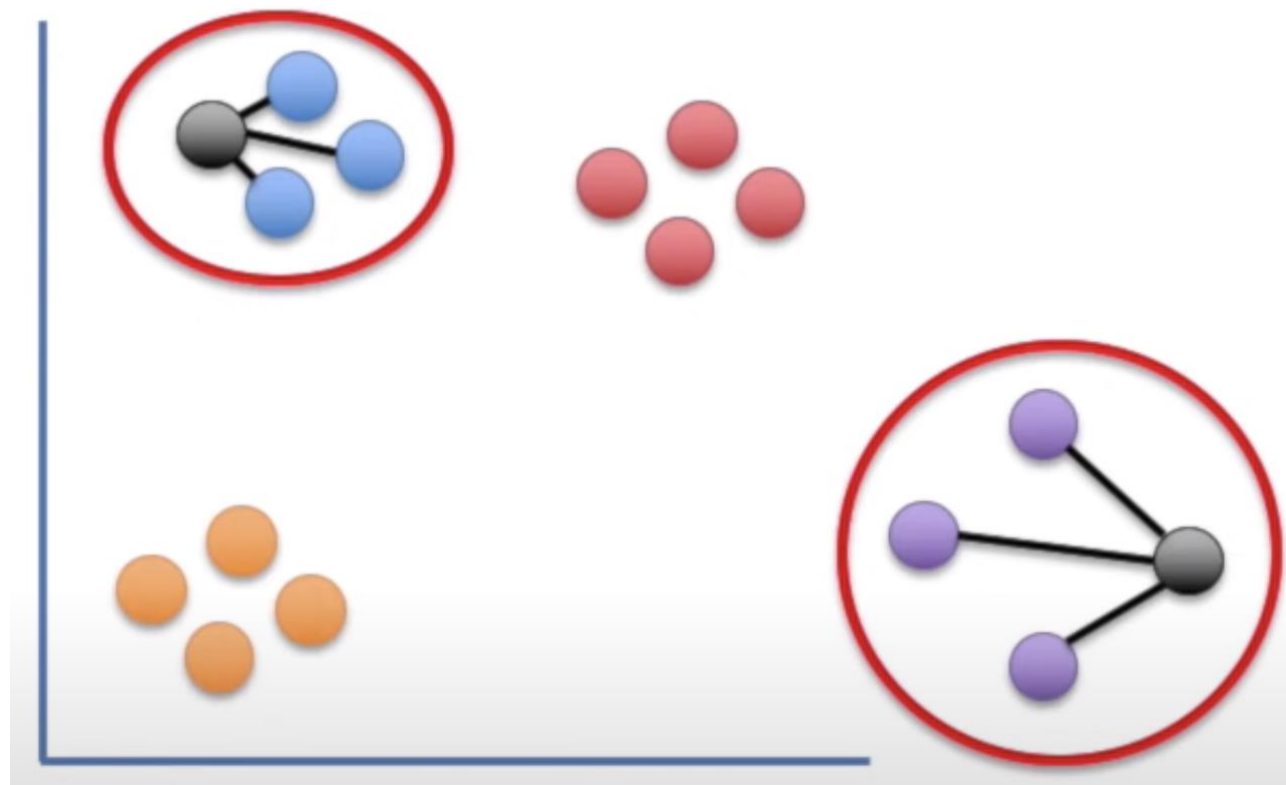
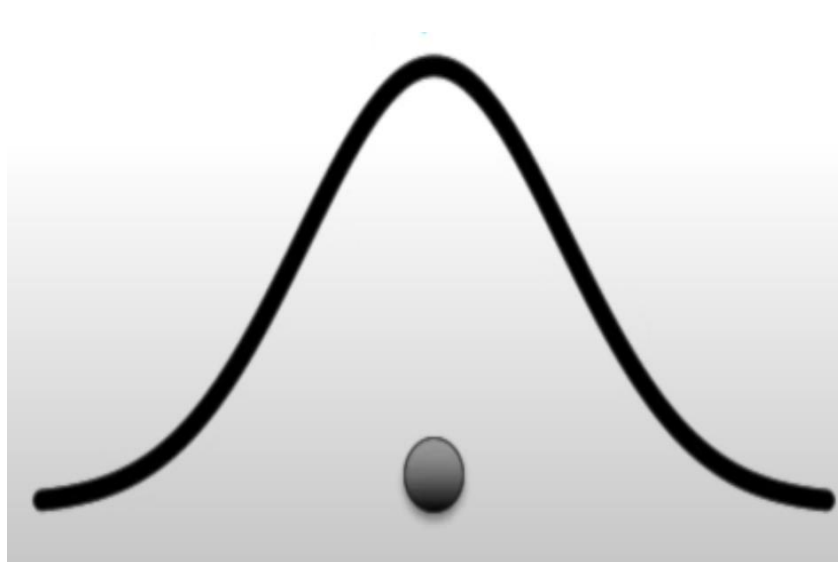
Data/Dimensionality reduction – t-SNE



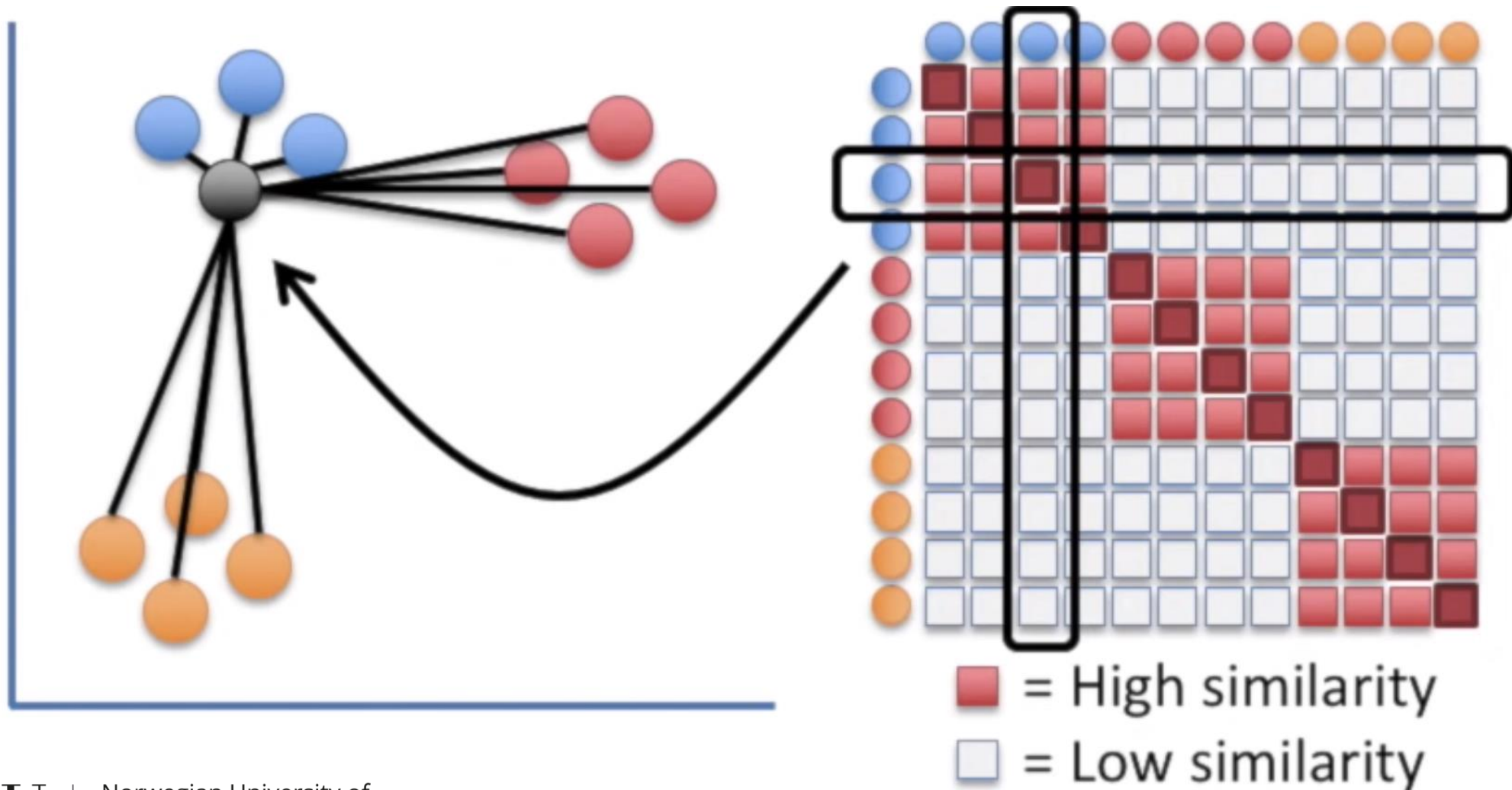
Data/Dimensionality reduction – t-SNE



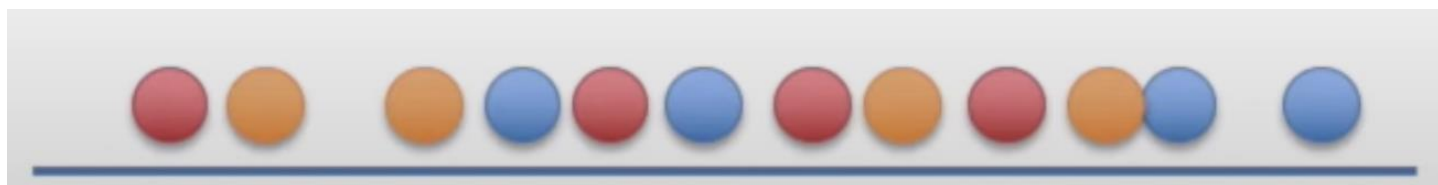
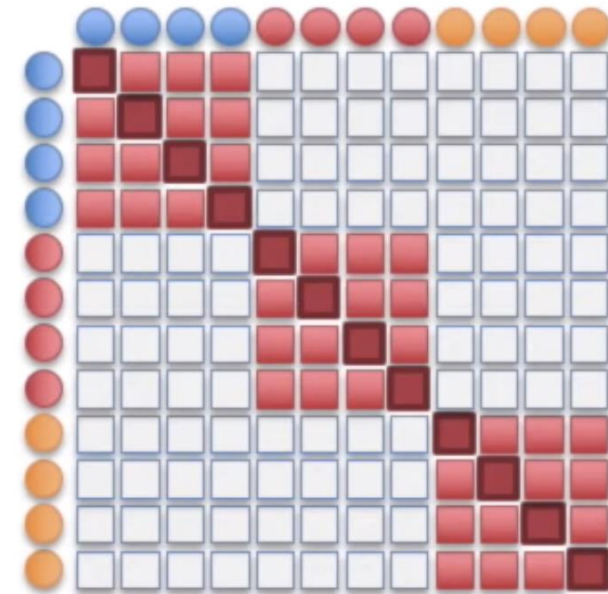
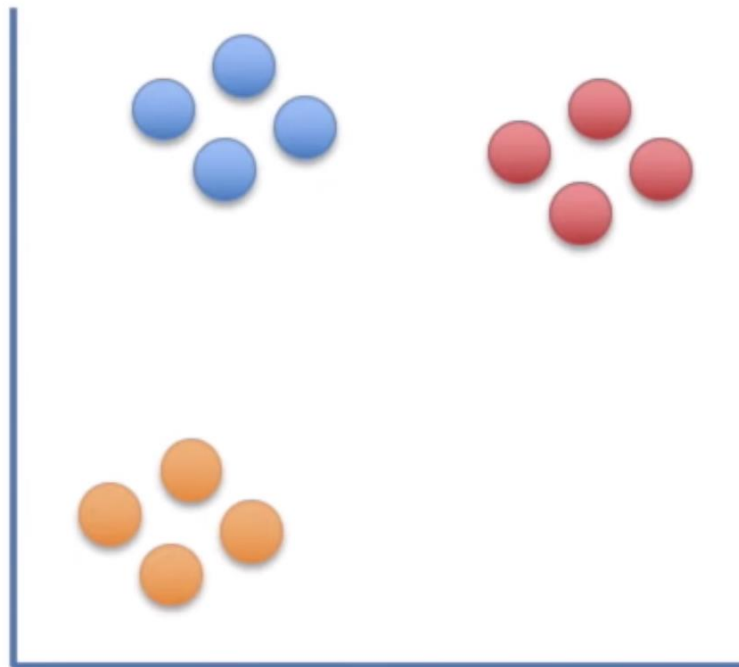
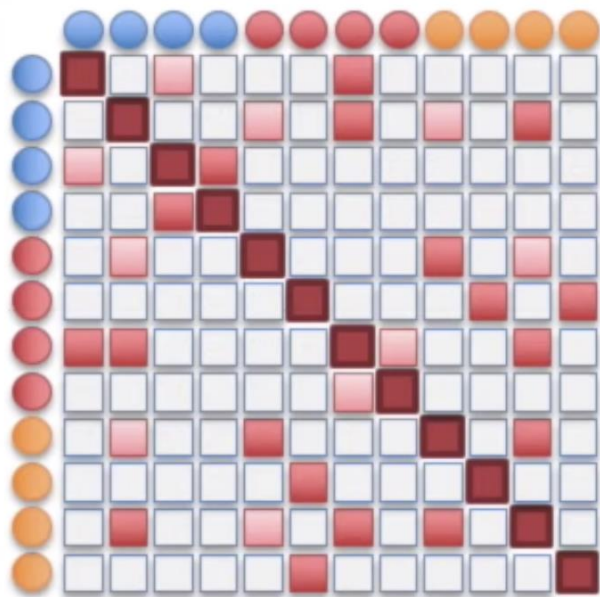
Data/Dimensionality reduction – t-SNE



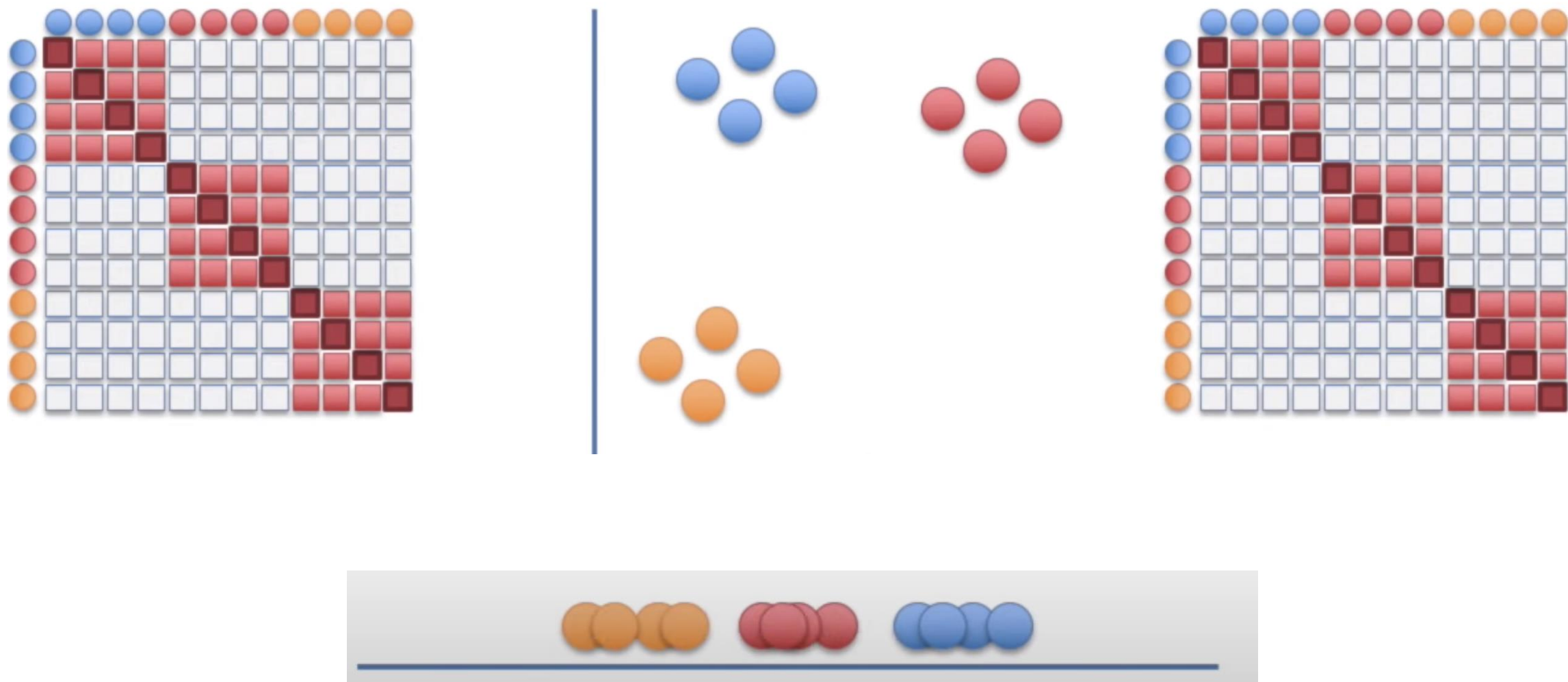
Data/Dimensionality reduction – t-SNE



Data/Dimensionality reduction – t-SNE



Data/Dimensionality reduction – t-SNE



T-SNE v. PCA & LDA

- Non-Linearity
- Preservation of Local Structures
- Robust to Outliers
- Effective for High-Dimensional Data
- No Assumption of Gaussian Distributions
- Multiclass Handling
- Global vs. Local Structure

Data/Dimensionality reduction

- **Autoencoders:**
- Autoencoders are neural network-based techniques used for unsupervised feature learning and dimensionality reduction.
- They consist of an encoder network that maps high-dimensional input data to a lower-dimensional latent space and a decoder network that reconstructs the input data from the latent space.
- Variational Autoencoders (VAEs) and denoising autoencoders are popular variations.

Data/Dimensionality reduction

- **Feature Selection:**

- Instead of transforming features, feature selection methods directly choose a subset of the most relevant features.
- Techniques include mutual information, feature importance from tree-based models, and recursive feature elimination (RFE).

Data/Dimensionality reduction

- **Random Projection:**
- Random projection techniques, such as random projection and Gaussian random projection, provide a simple and efficient way to reduce dimensionality while preserving pairwise distances to some extent.

Data/Dimensionality reduction

- **Factor Analysis:**
- Factor Analysis is a technique that models observed variables as linear combinations of underlying factors.
- It can be used for dimensionality reduction when you assume that the observed variables are influenced by a smaller number of latent factors.

General Data preprocessing and calibration

- Four key steps
 - Data cleaning
 - Data integration
 - Data transformation
 - Data/Dimensionality reduction

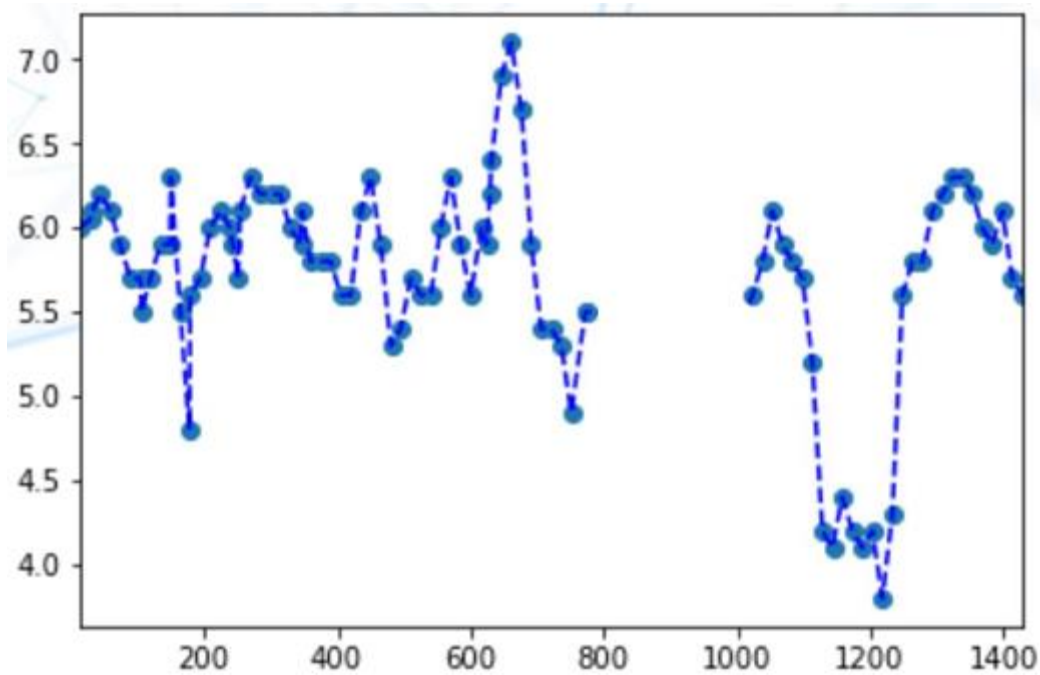
Common tips

- The first step in Data Preprocessing is to understand your data. Just looking at your dataset can give you an intuition of what things you need to focus on.
- Use statistical methods or pre-built libraries that help you visualize the dataset and give a clear image of how your data looks in terms of class distribution.
- Summarize your data in terms of the number of duplicates, missing values, and outliers present in the data.
- Drop the fields you think have no use for the modeling or are closely related to other attributes. Dimensionality reduction is one of the very important aspects of Data Preprocessing.
- Do some feature engineering and figure out which attributes contribute most towards model training.

Pre-processing and Noise Removal Examples

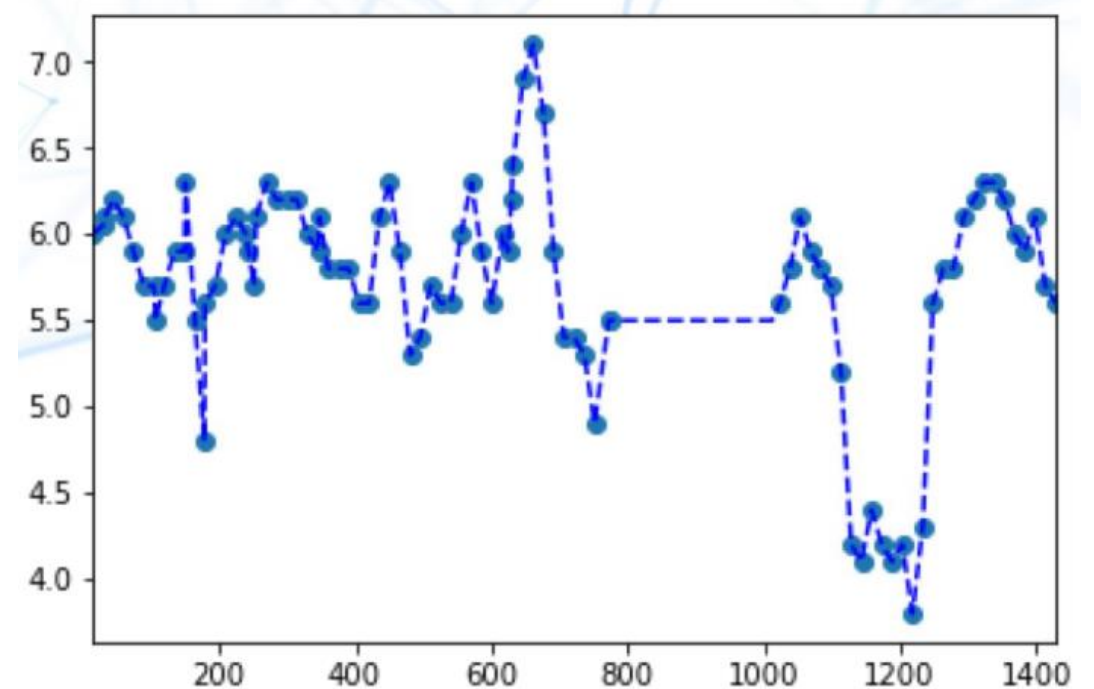
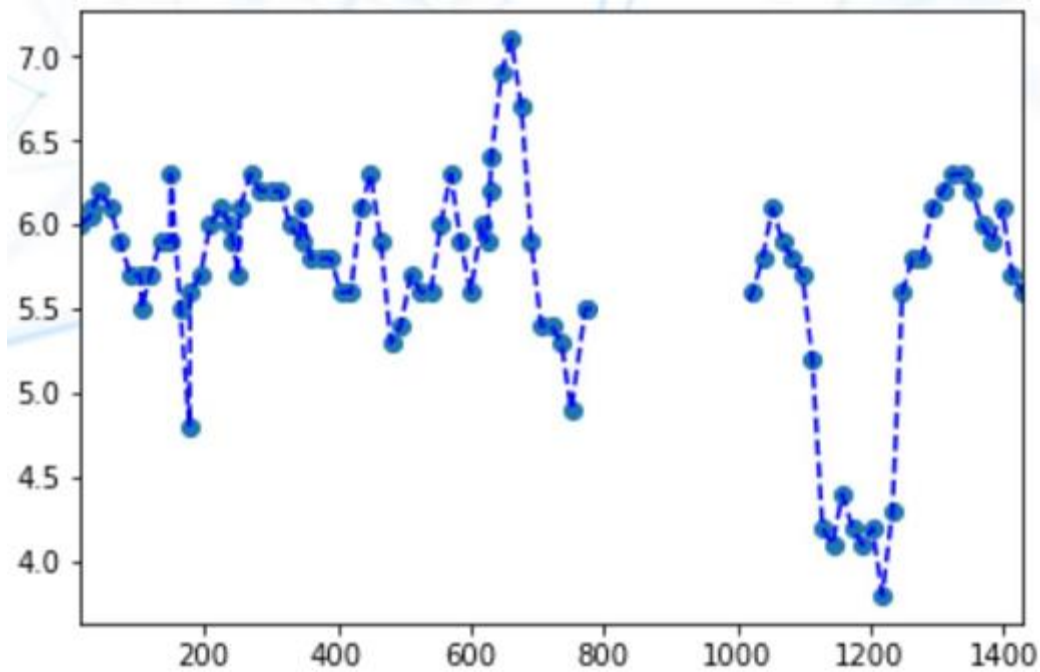
Time series data preprocessing

pandas → [fillna](#)(method = ???)



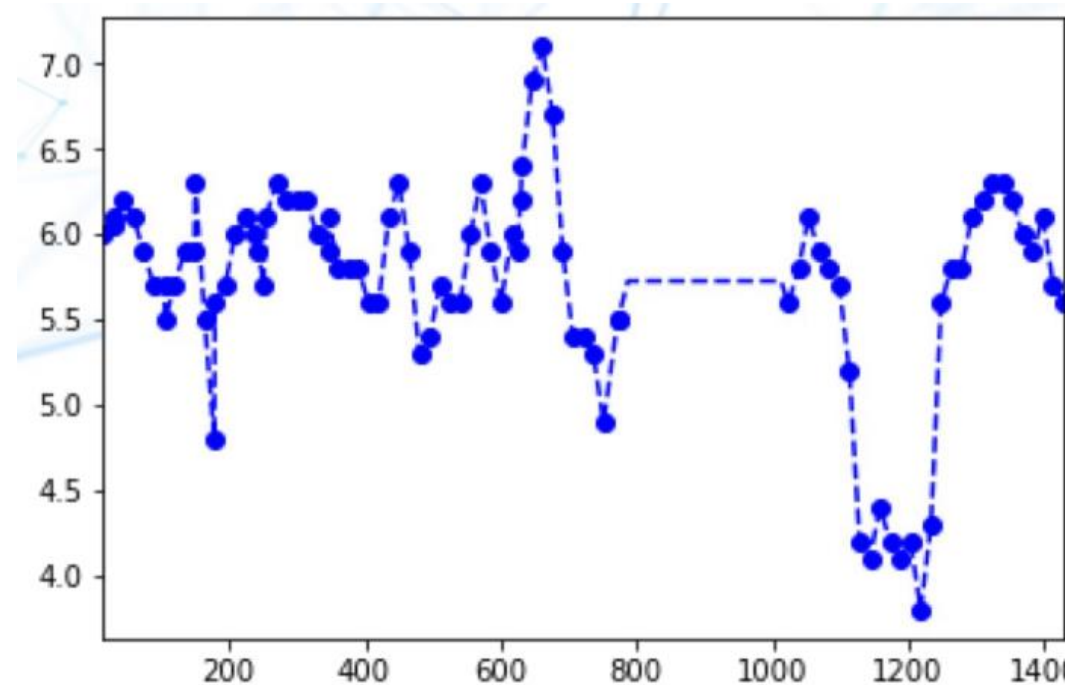
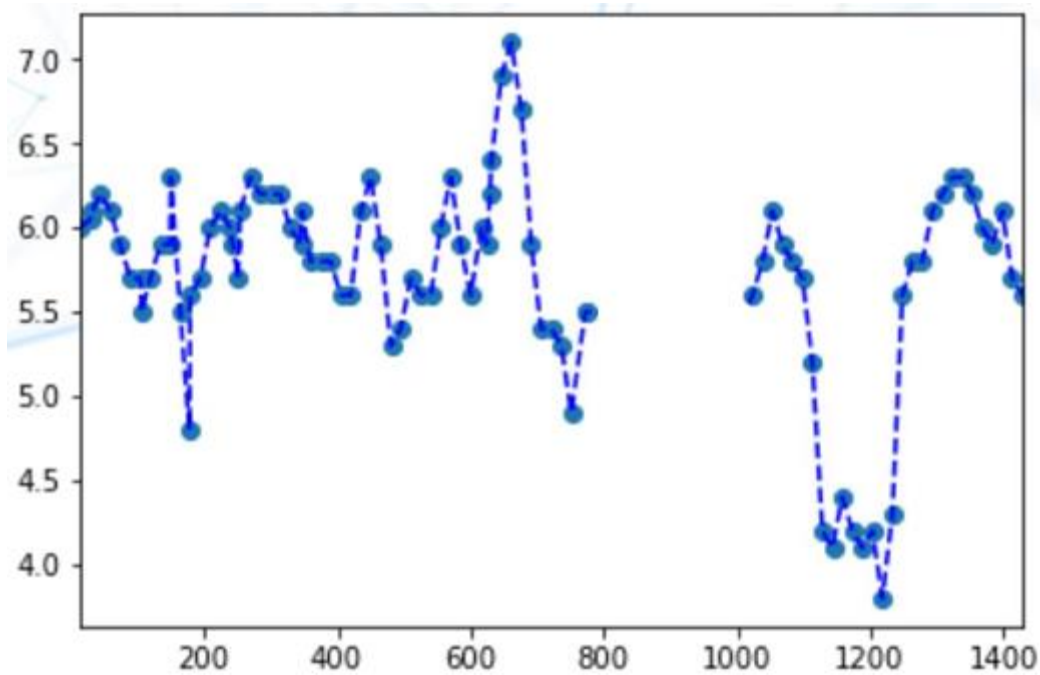
Time series data preprocessing

pandas → [fillna](#)(method = ffill) → Last observed value of a variable is used for all subsequent.



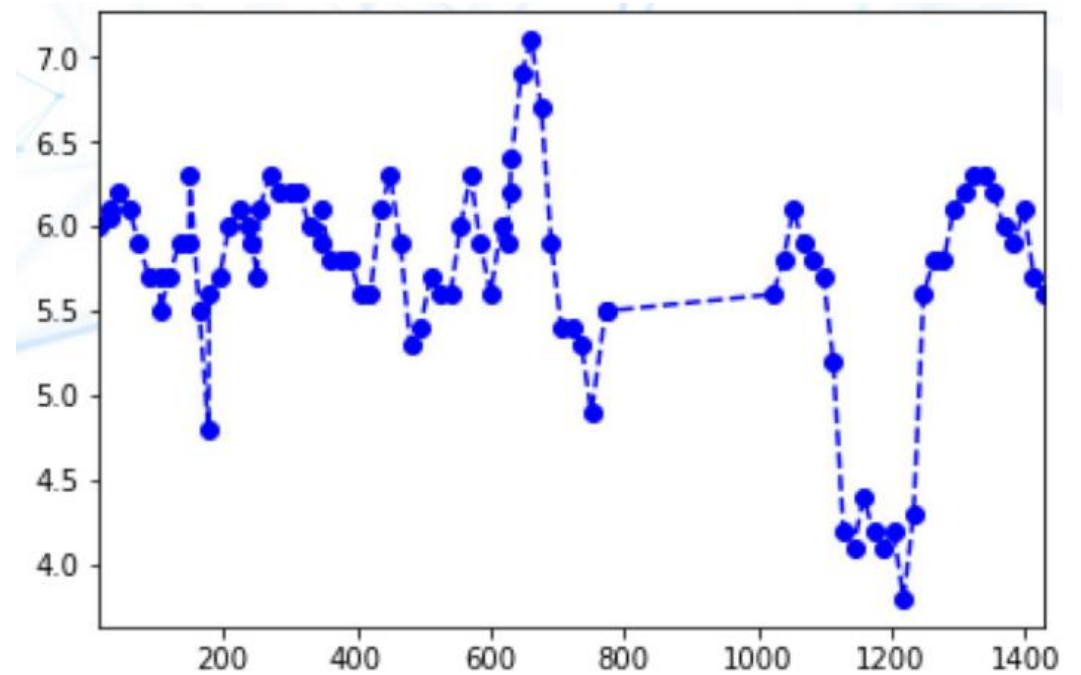
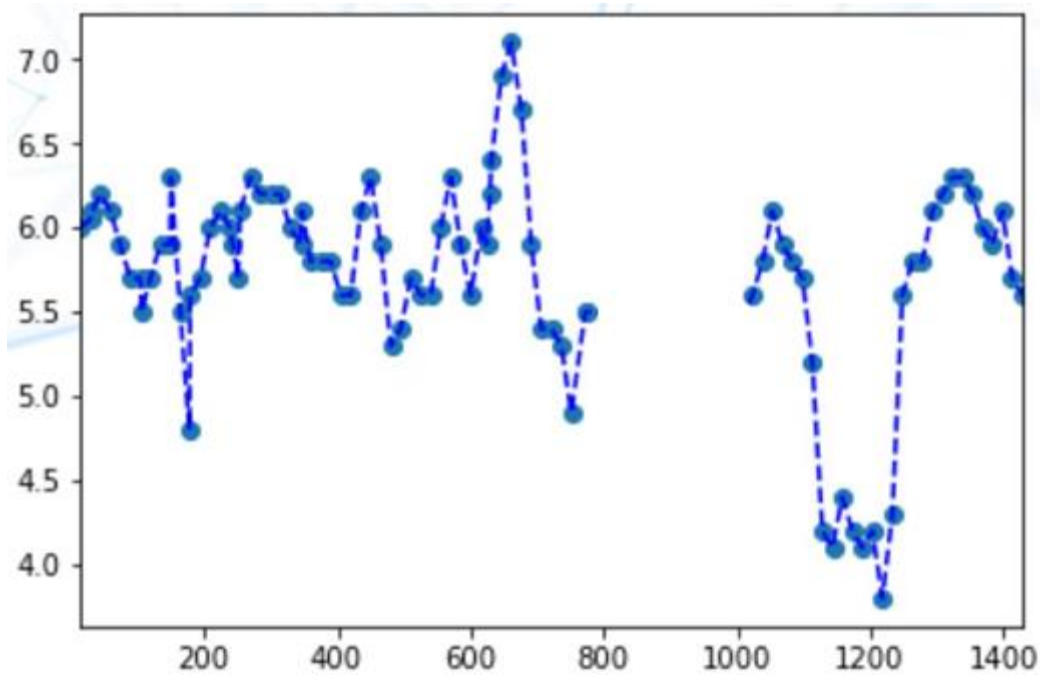
Time series data preprocessing

pandas → [fillna](#)(method = df[...].mean())



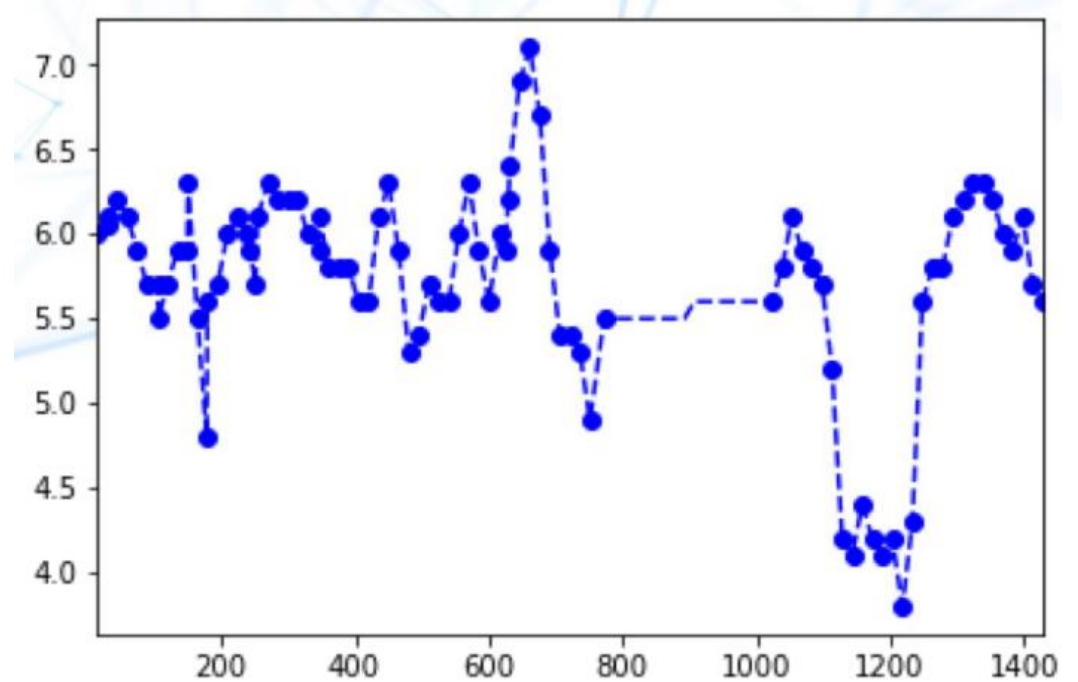
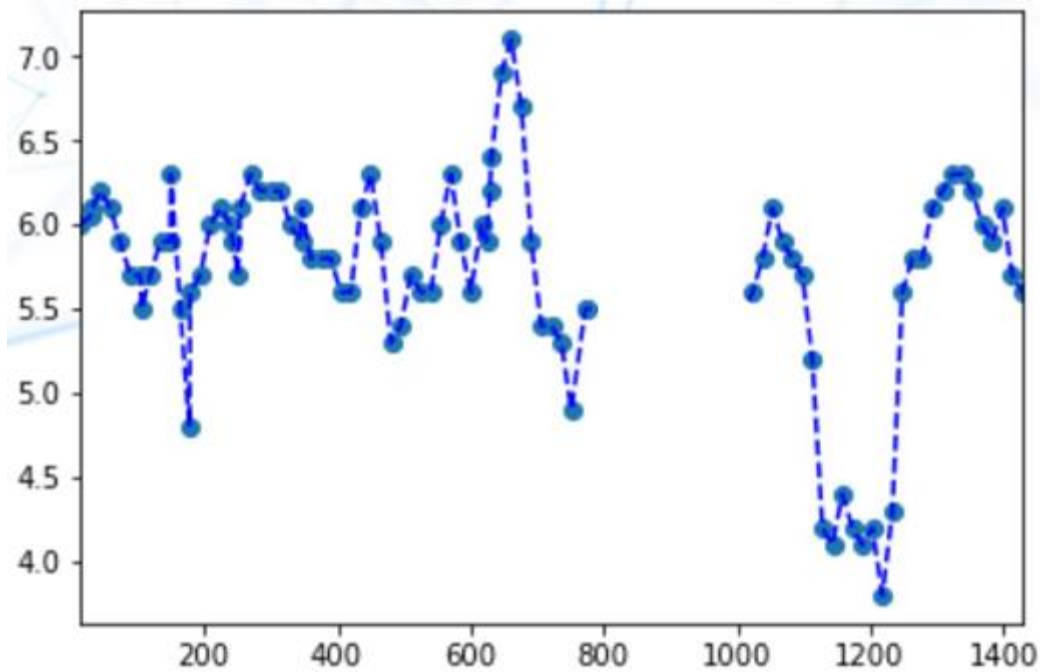
Time series data preprocessing

pandas → [fillna](#)(method = 'linear')



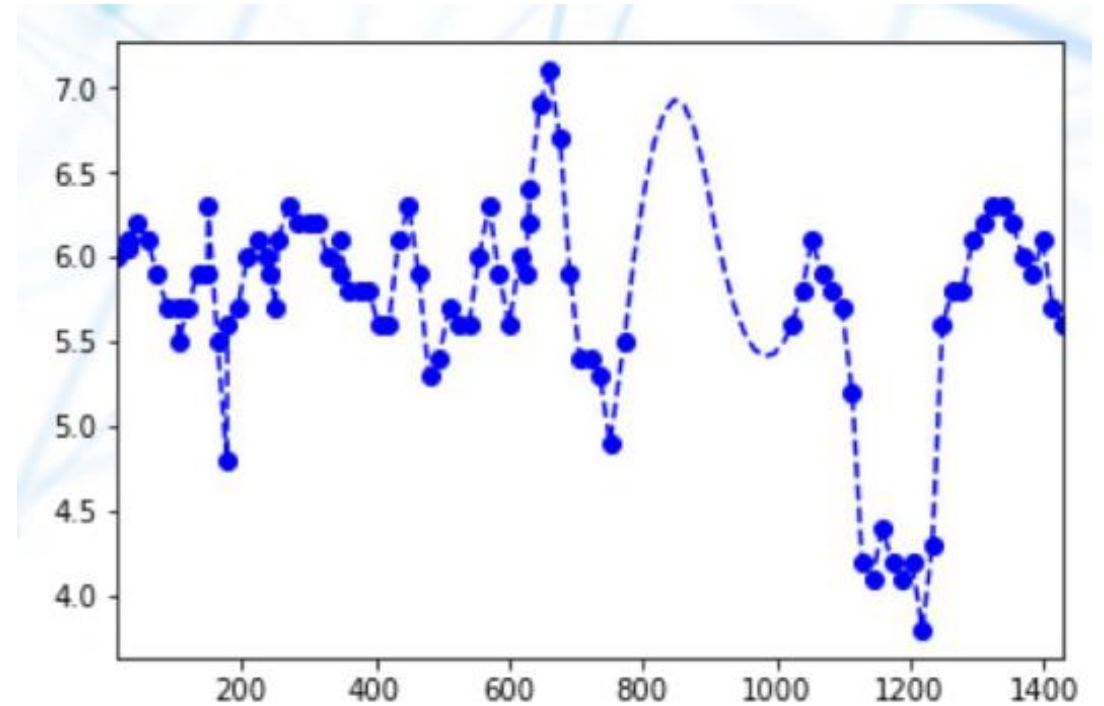
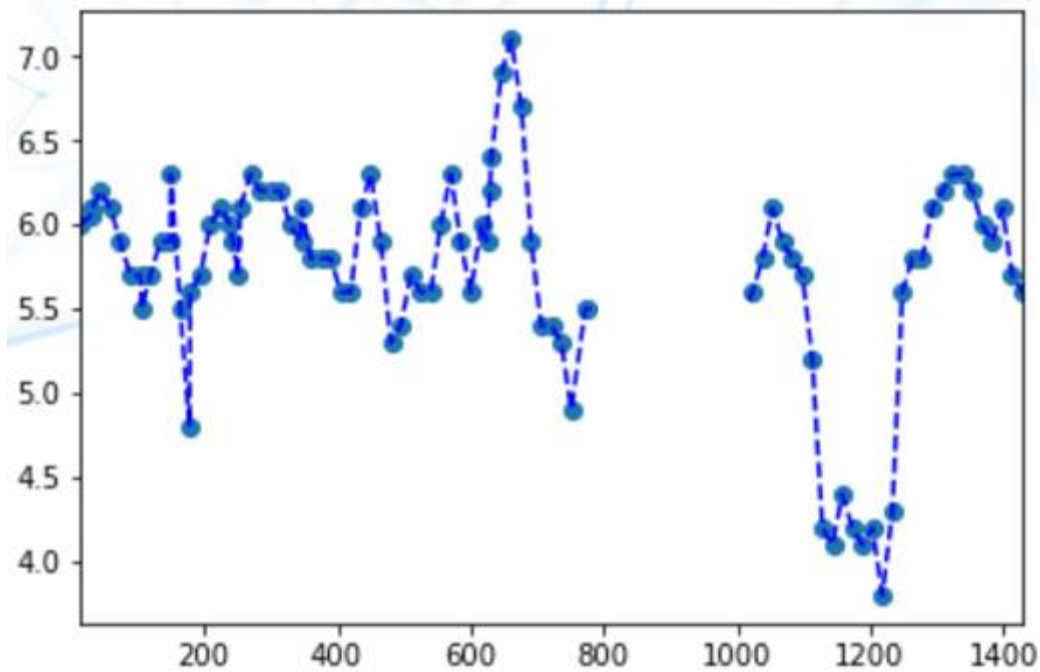
Time series data preprocessing

pandas → [fillna](#)(method = 'nearest')



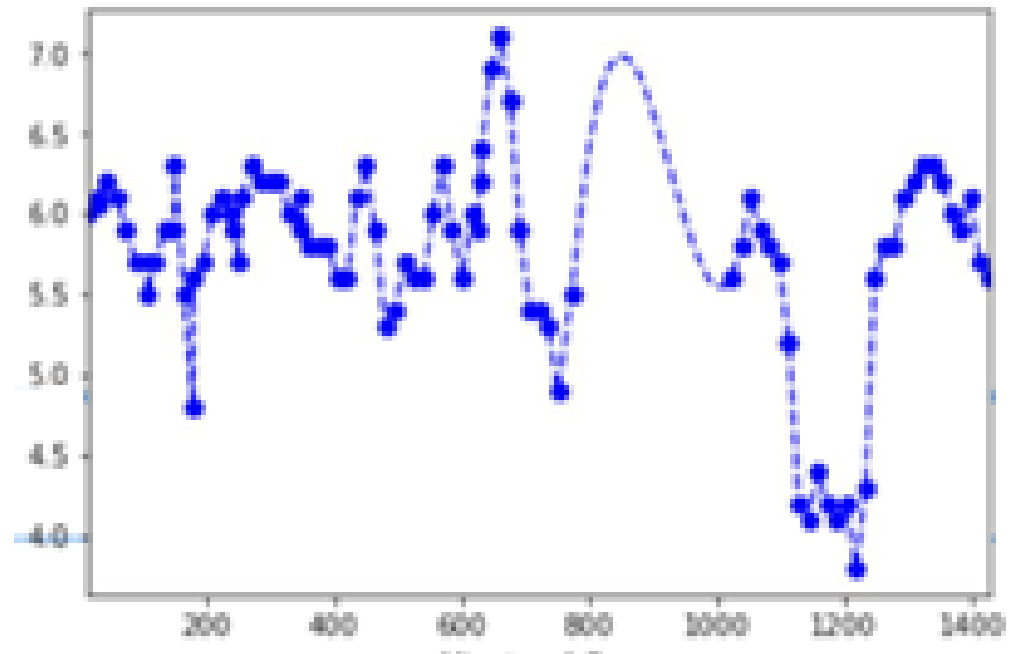
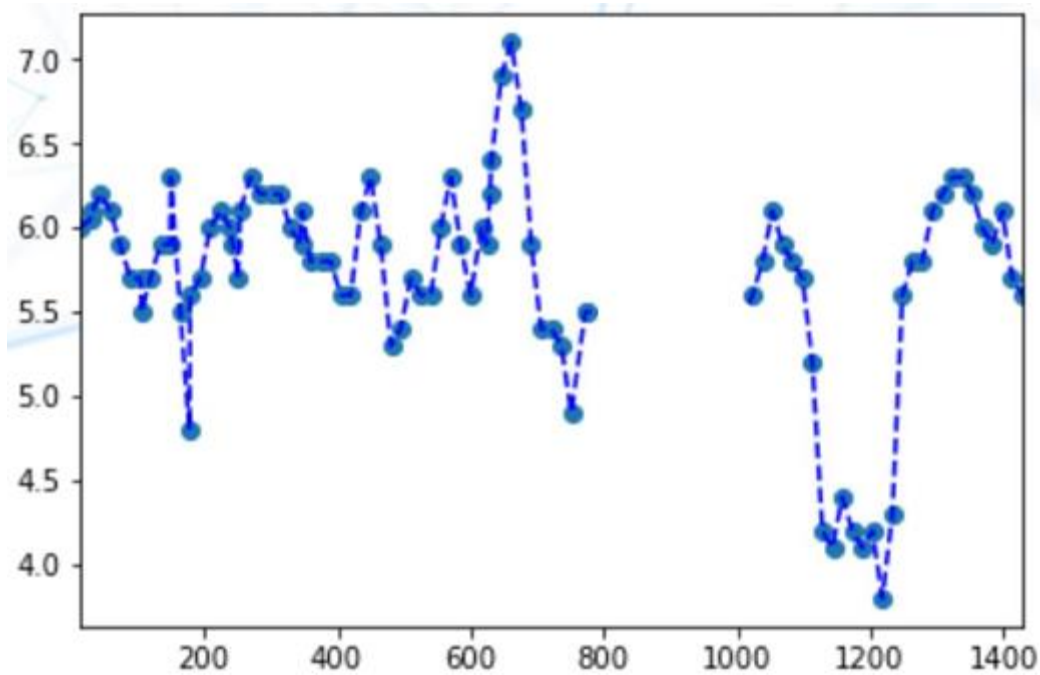
Time series data preprocessing

pandas → [fillna](#)(method = 'polynomial', order = 2)



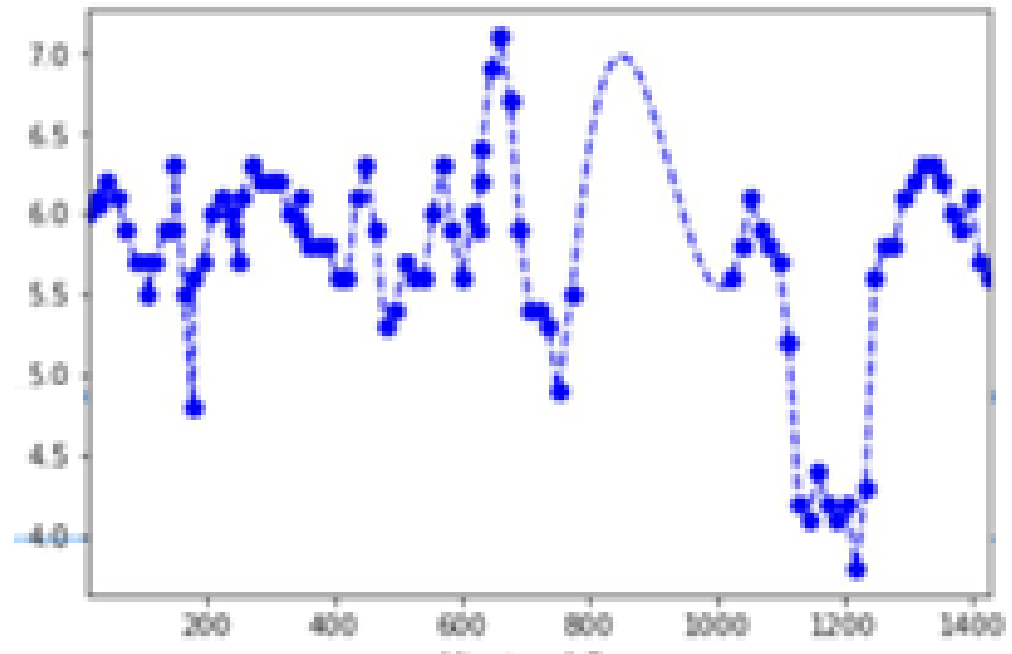
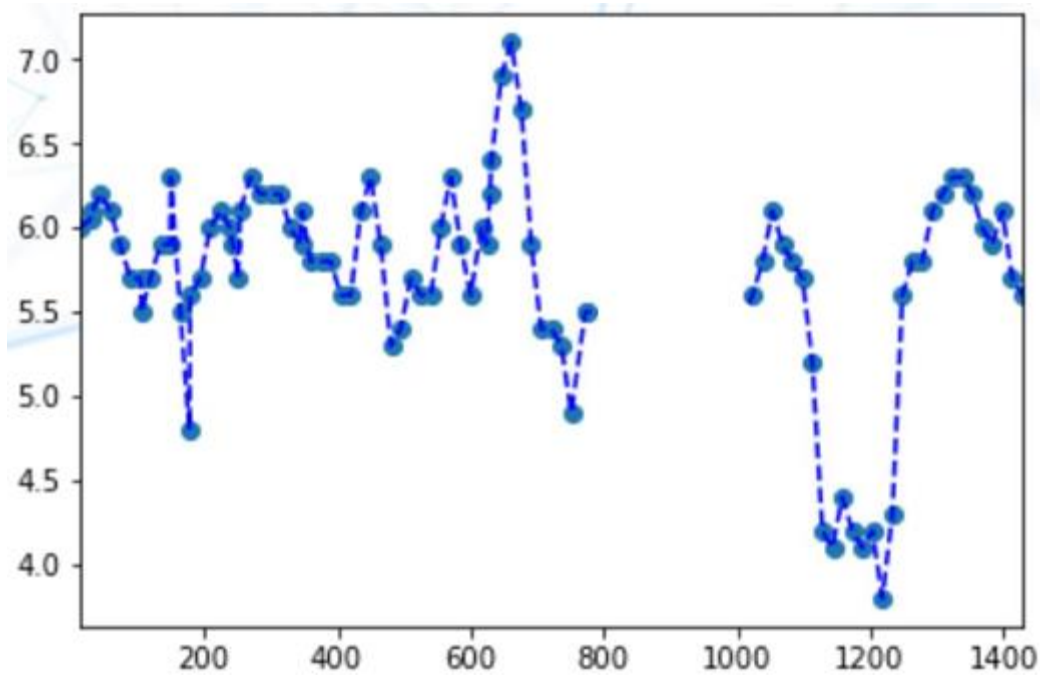
Time series data preprocessing

pandas → [fillna](#)(method = 'polynomial', order = 3)

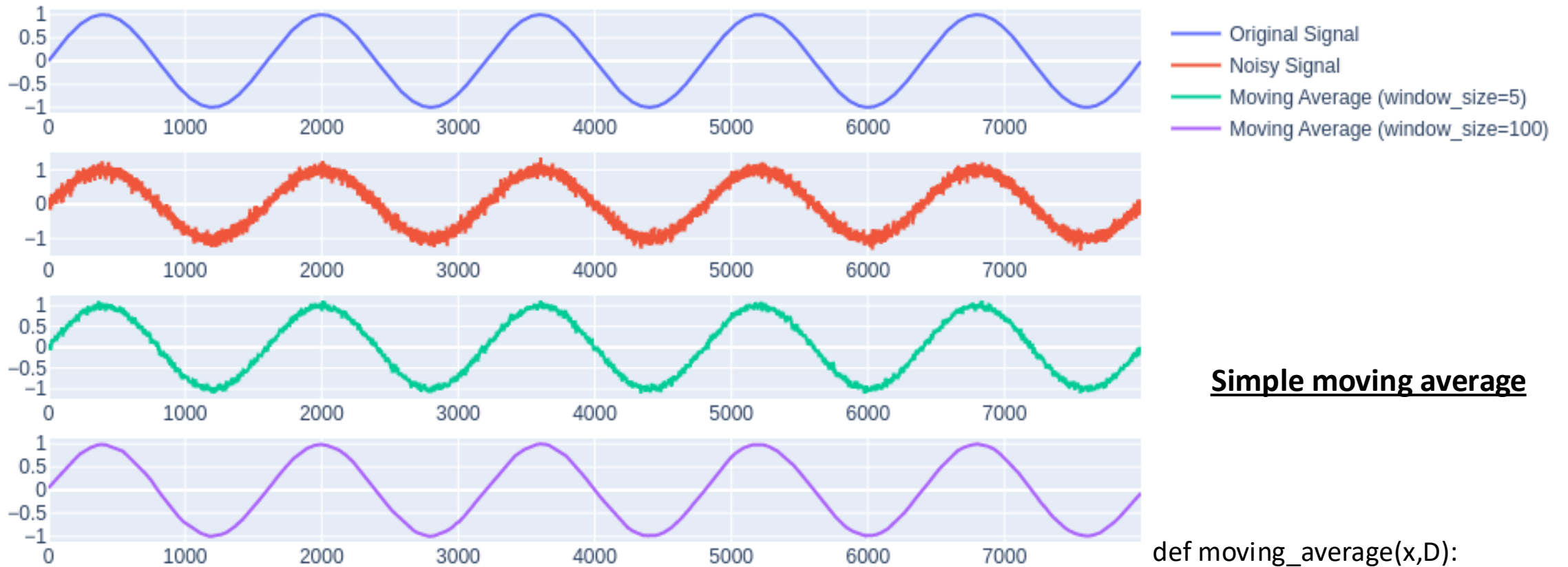


Time series data preprocessing

pandas → [fillna](#)(method = 'polynomial', order = 3)



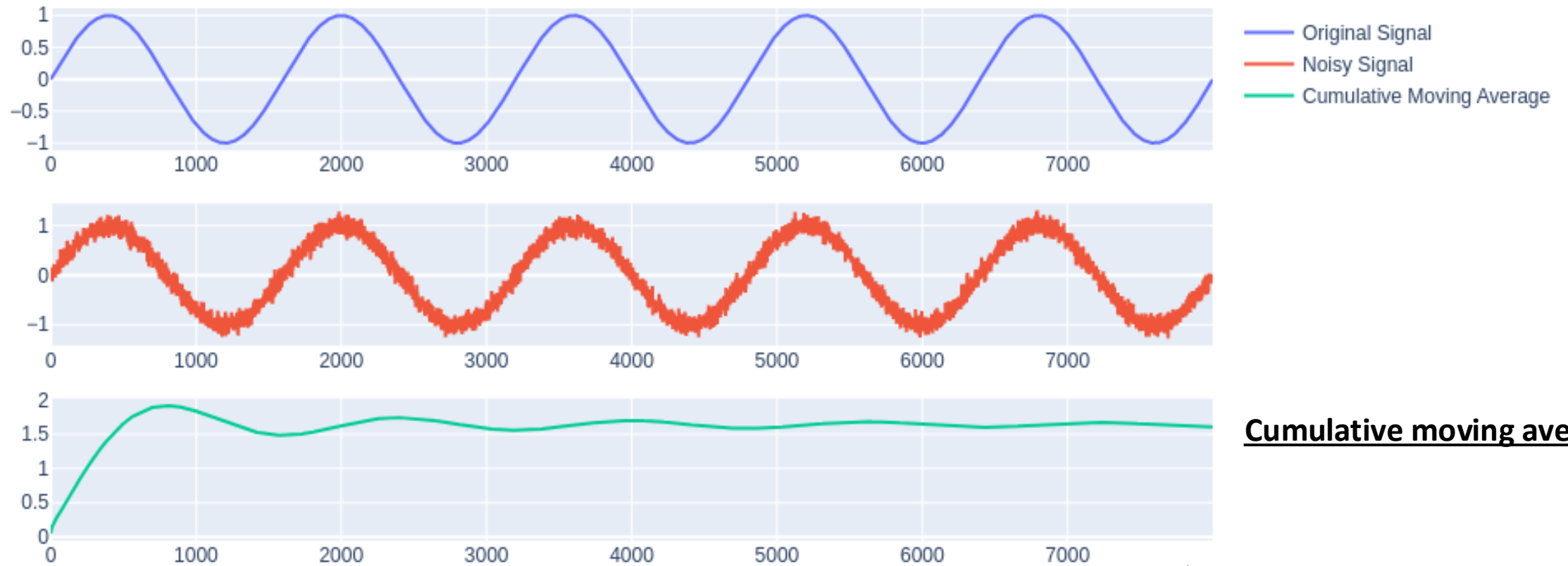
Time series data preprocessing



Simple moving average

```
def moving_average(x,D):  
    h = [v for v in x[:D]]  
    for p in range(len(x) - D):  
        h.append(sum(x[p:p+D])/float(D))  
    return h
```

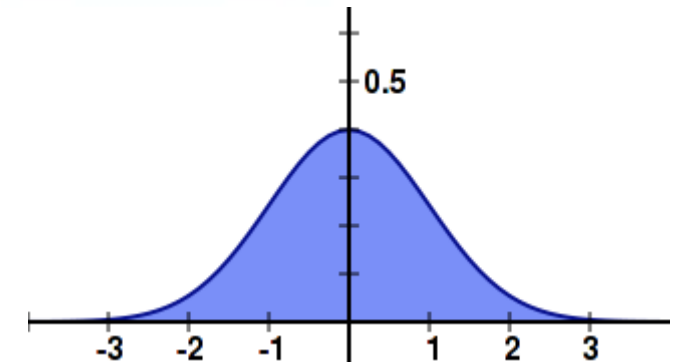
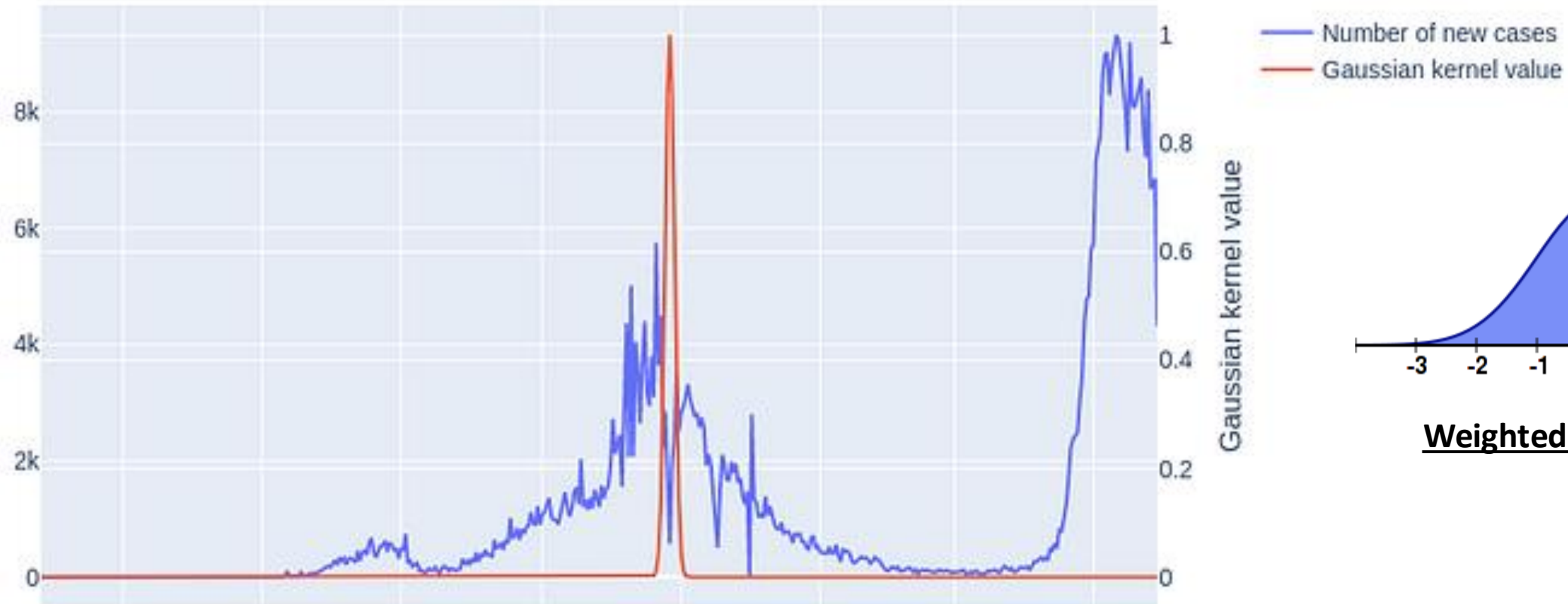
Time series data preprocessing



Cumulative moving average

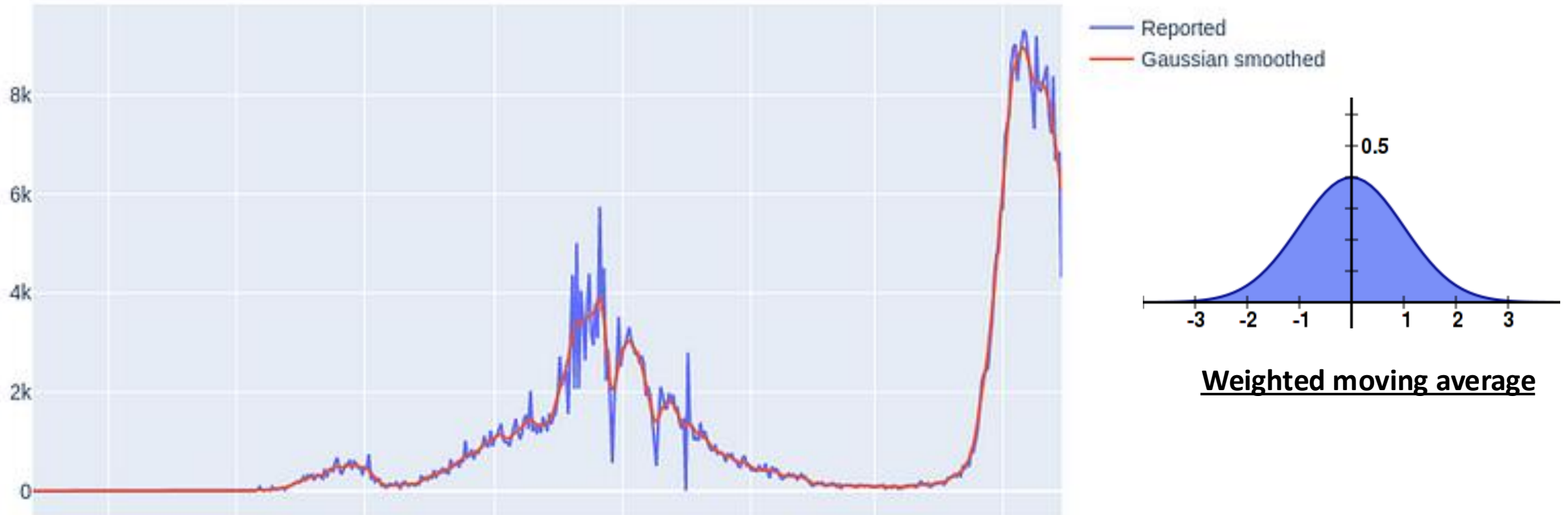
```
import numpy as np
def cumulative_moving_average(signal):
    cma = []
    cma.append(signal[0]) # data point at t=0
    for i, x in enumerate(signal[1:], start=1):
        cma.append(((x+(i+1)*cma[i-1])/(i+1)))
    return cma
```

Time series data preprocessing

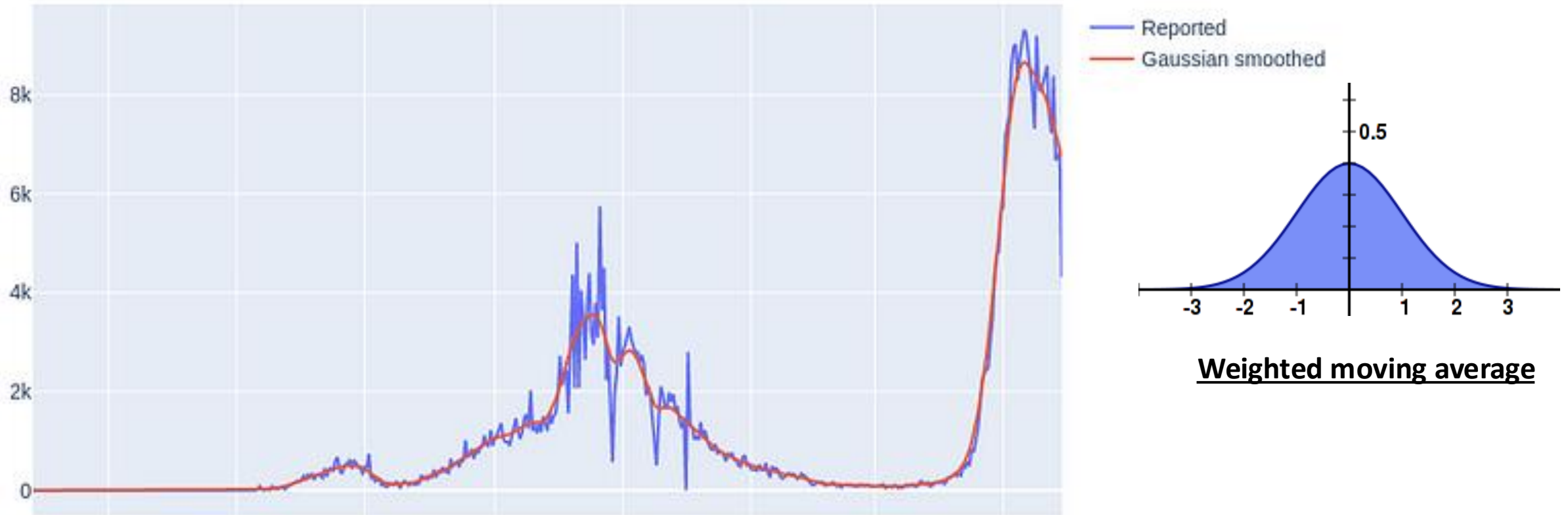


Weighted moving average

Time series data preprocessing



Time series data preprocessing



Text data pre-processing

v2	clean_msg
Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...	Go until jurong point crazy Available only in bugis n great world la e buffet Cine there got amore wat
Ok lar... Joking wif u oni...	Ok lar Joking wif u oni
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's	Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005 Text FA to 87121 to receive entry questionstd txt rateTCs apply 08452810075over18s
U dun say so early hor... U c already then say...	U dun say so early hor U c already then say
Nah I don't think he goes to usf, he lives around here though	Nah I dont think he goes to usf he lives around here though

```
import string
string.punctuation
#defining the function to remove punctuation
def remove_punctuation(text):
    punctuationfree="".join([i for i in text if i not in string.punctuation])
    return punctuationfree
#storing the punctuation free text
data['clean_msg']= data['v2'].apply(lambda x:remove_punctuation(x))
data.head()
```

Text data pre-processing

clean_msg	msg_lower
Go until jurong point crazy Available only in bugis n great world la e buffet Cine there got amore wat	go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat
Ok lar Joking wif u oni	ok lar joking wif u oni
Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005 Text FA to 87121 to receive entry questionstd txt rateTCs apply 08452810075over18s	free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s
U dun say so early hor U c already then say	u dun say so early hor u c already then say
Nah I dont think he goes to usf he lives around here though	nah i dont think he goes to usf he lives around here though

```
data['msg_lower']= data['clean_msg'].apply(lambda x: x.lower())
```

Text data pre-processing

msg_lower	msg_tokenied
go until jurong point crazy available only in bugis n great world la e buffet cine there got amore wat	[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]
ok lar joking wif u oni	[ok, lar, joking, wif, u, oni]
free entry in 2 a wkly comp to win fa cup final tkts 21st may 2005 text fa to 87121 to receive entry questionstd txt ratetcs apply 08452810075over18s	[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
u dun say so early hor u c already then say	[u, dun, say, so, early, hor, u, c, already, then, say]
nah i dont think he goes to usf he lives around here though	[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]

```
import re
def tokenization(text):
    tokens = re.split('W+',text)
    return tokens
#applying function to the column
data['msg_tokenied']= data['msg_lower'].apply(lambda x: tokenization(x))
```

Text data pre-processing

msg_tokenied	no_stopwords
[go, until, jurong, point, crazy, available, only, in, bugis, n, great, world, la, e, buffet, cine, there, got, amore, wat]	[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joking, wif, u, oni]
[free, entry, in, 2, a, wkly, comp, to, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, to, 87121, to, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
[u, dun, say, so, early, hor, u, c, already, then, say]	[u, dun, say, early, hor, u, c, already, say]
[nah, i, dont, think, he, goes, to, usf, he, lives, around, here, though]	[nah, dont, think, goes, usf, lives, around, though]

```
import nltk
```

```
#Stop words present in the library
```

```
stopwords = nltk.corpus.stopwords.words('english')
```

```
#defining the function to remove stopwords from tokenized text
```

```
def remove_stopwords(text):
```

```
    output= [i for i in text if i not in stopwords]
```

```
    return output
```

```
#applying the function
```

```
data['no_stopwords']= data['msg_tokenied'].apply(lambda x:remove_stopwords(x))
```

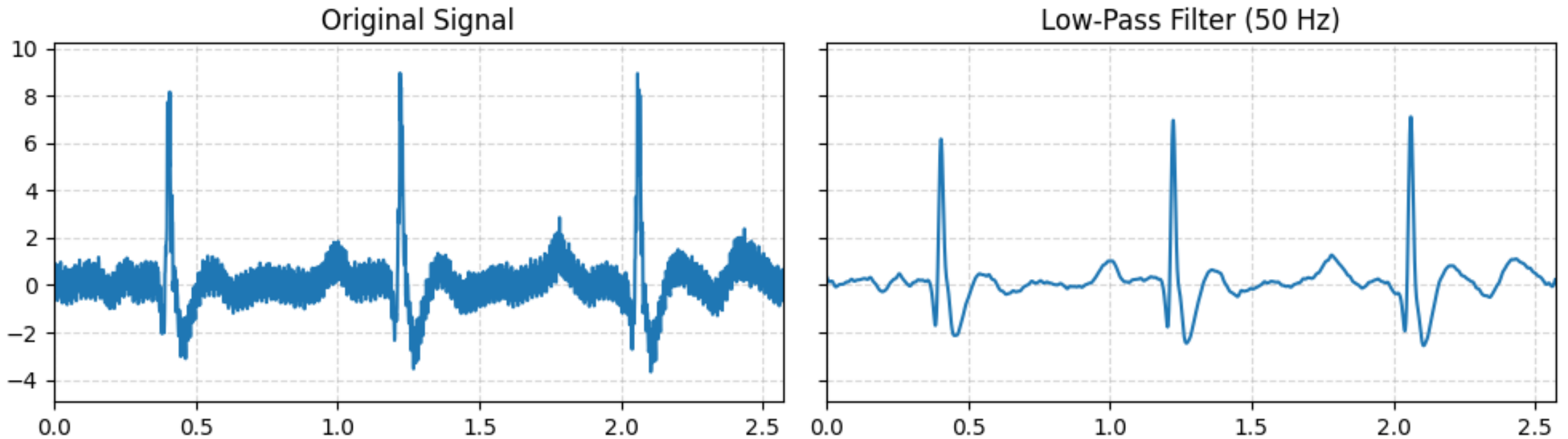
Text data pre-processing

no_stopwords	msg_stemmed	msg_lemmatized
[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]	[go, jurong, point, crazi, avail, bugi, n, great, world, la, e, buffet, cine, got, amor, wat]	[go, jurong, point, crazy, available, bugis, n, great, world, la, e, buffet, cine, got, amore, wat]
[ok, lar, joking, wif, u, oni]	[ok, lar, joke, wif, u, oni]	[ok, lar, joking, wif, u, oni]
[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]	[free, entri, 2, wkli, comp, win, fa, cup, final, tkt, 21st, may, 2005, text, fa, 87121, receiv, entri, questionstd, txt, ratetc, appli, 08452810075over18]	[free, entry, 2, wkly, comp, win, fa, cup, final, tkts, 21st, may, 2005, text, fa, 87121, receive, entry, questionstd, txt, ratetcs, apply, 08452810075over18s]
[u, dun, say, early, hor, u, c, already, say]	[u, dun, say, earli, hor, u, c, already, say]	[u, dun, say, early, hor, u, c, already, say]
[nah, dont, think, goes, usf, lives, around, though]	[nah, dont, think, goe, usf, live, around, though]	[nah, dont, think, go, usf, life, around, though]

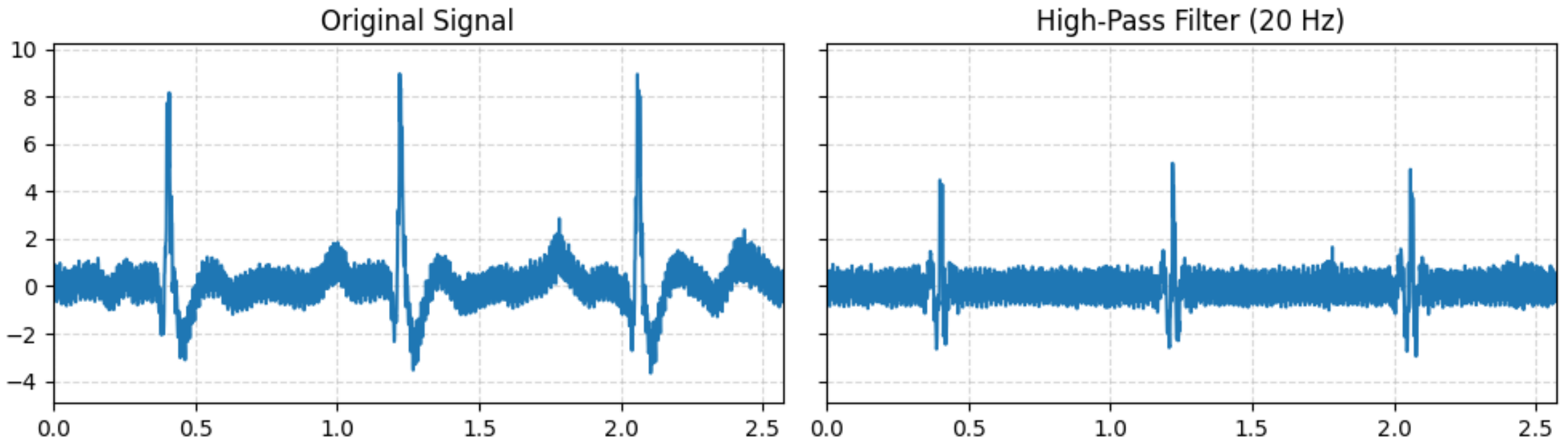
```
from nltk.stem.porter import PorterStemmer
porter_stemmer = PorterStemmer()
#defining a function for stemming
def stemming(text):
    stem_text = [porter_stemmer.stem(word) for word in text]
    return stem_text
data['msg_stemmed']=data['no_sw_msg'].apply(lambda x:
stemming(x))
```

```
from nltk.stem import WordNetLemmatizer
wordnet_lemmatizer = WordNetLemmatizer()
#defining the function for lemmatization
def lemmatizer(text):
    lemm_text = [wordnet_lemmatizer.lemmatize(word) for word in text]
    return lemm_text
data['msg_lemmatized']=data['no_stopwords'].apply(lambda
x:lemmatizer(x))
```

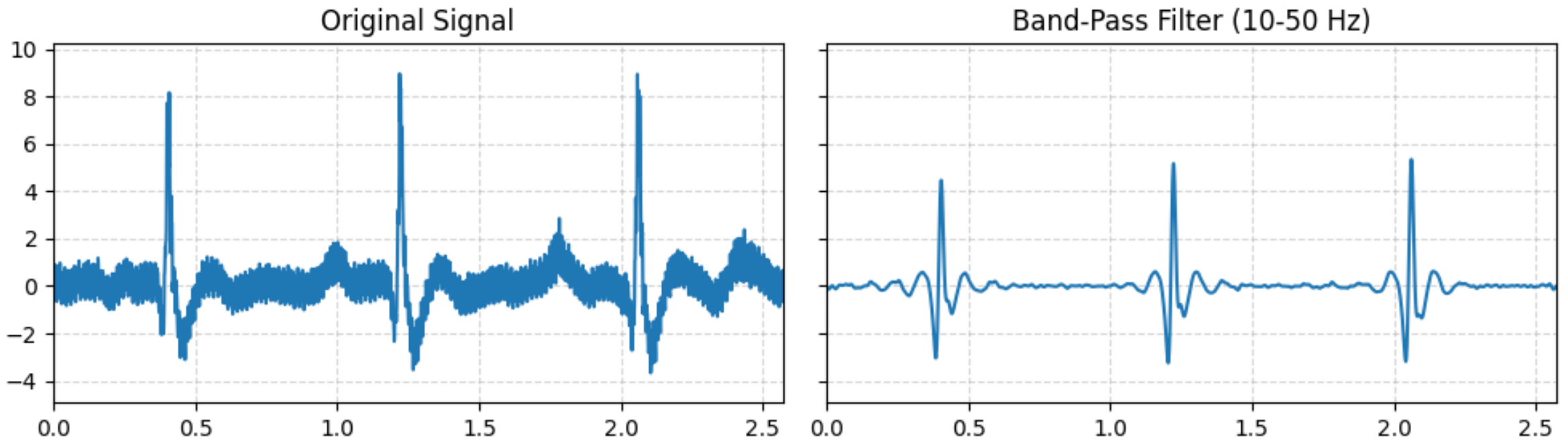
Frequency based signal pre-processing (aka Filtering)



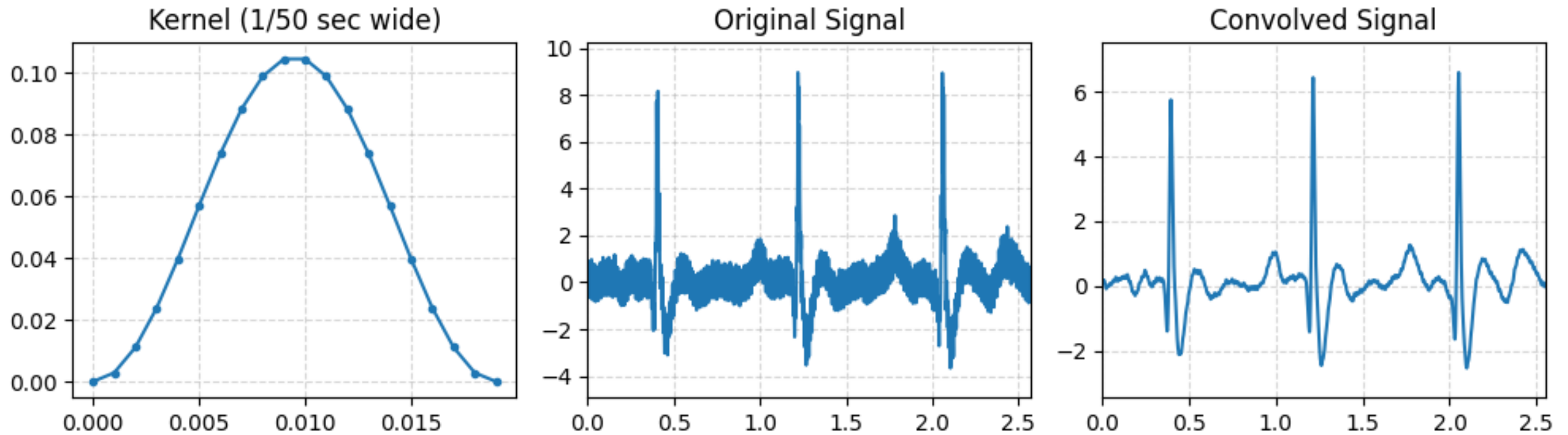
Frequency based signal pre-processing (aka Filtering)



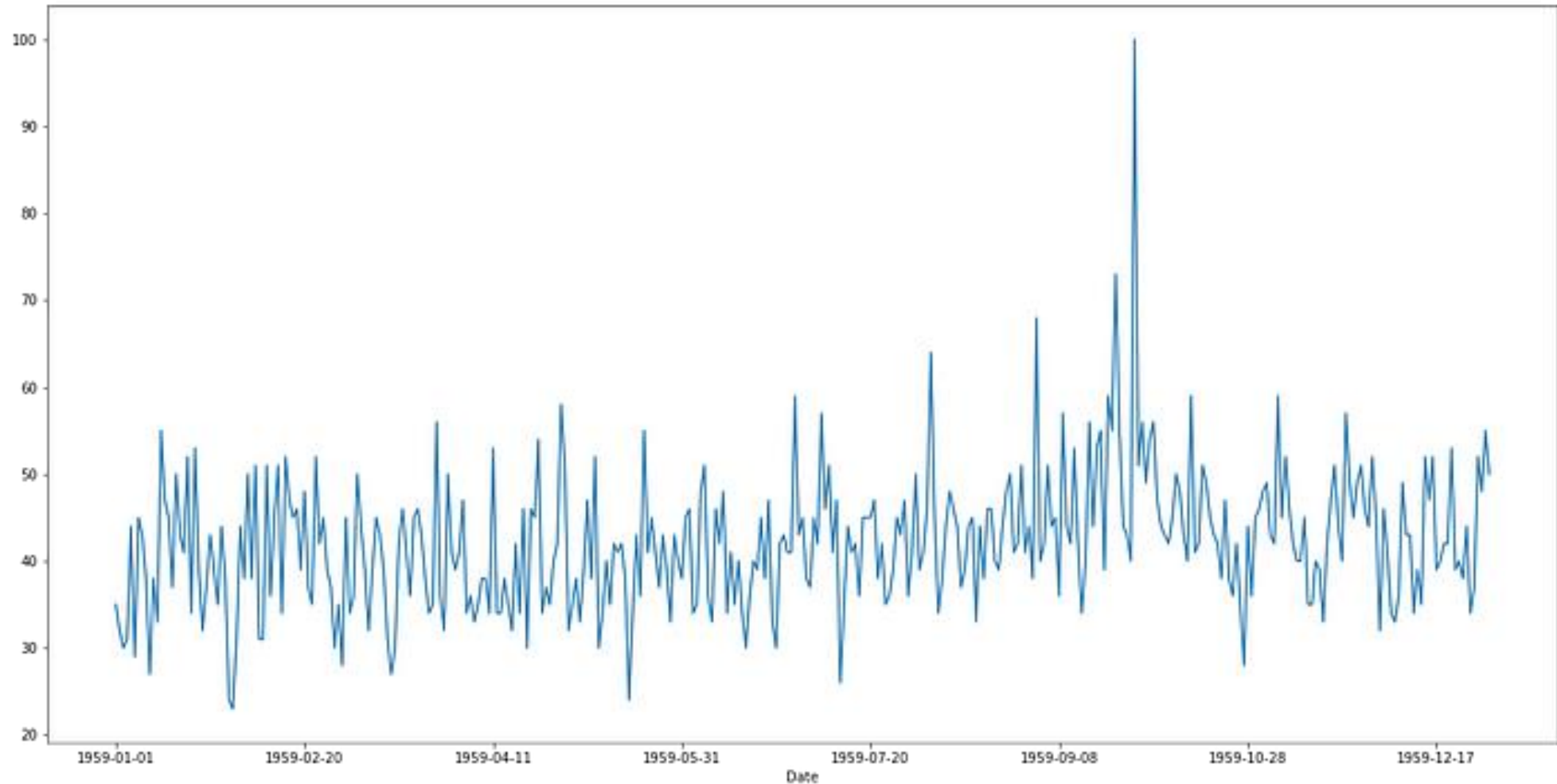
Frequency based signal pre-processing (aka Filtering)



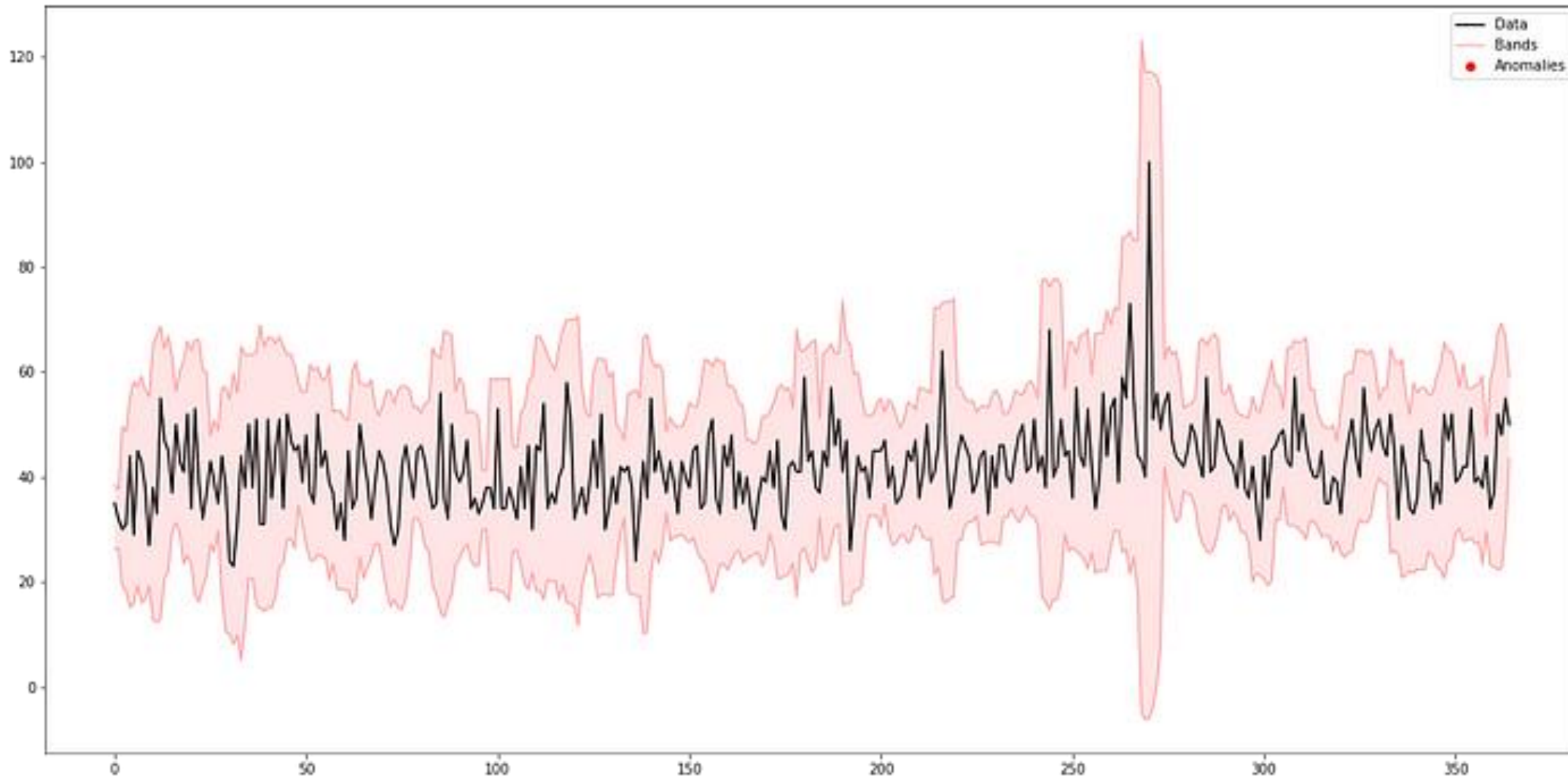
Frequency based signal pre-processing (aka Filtering)



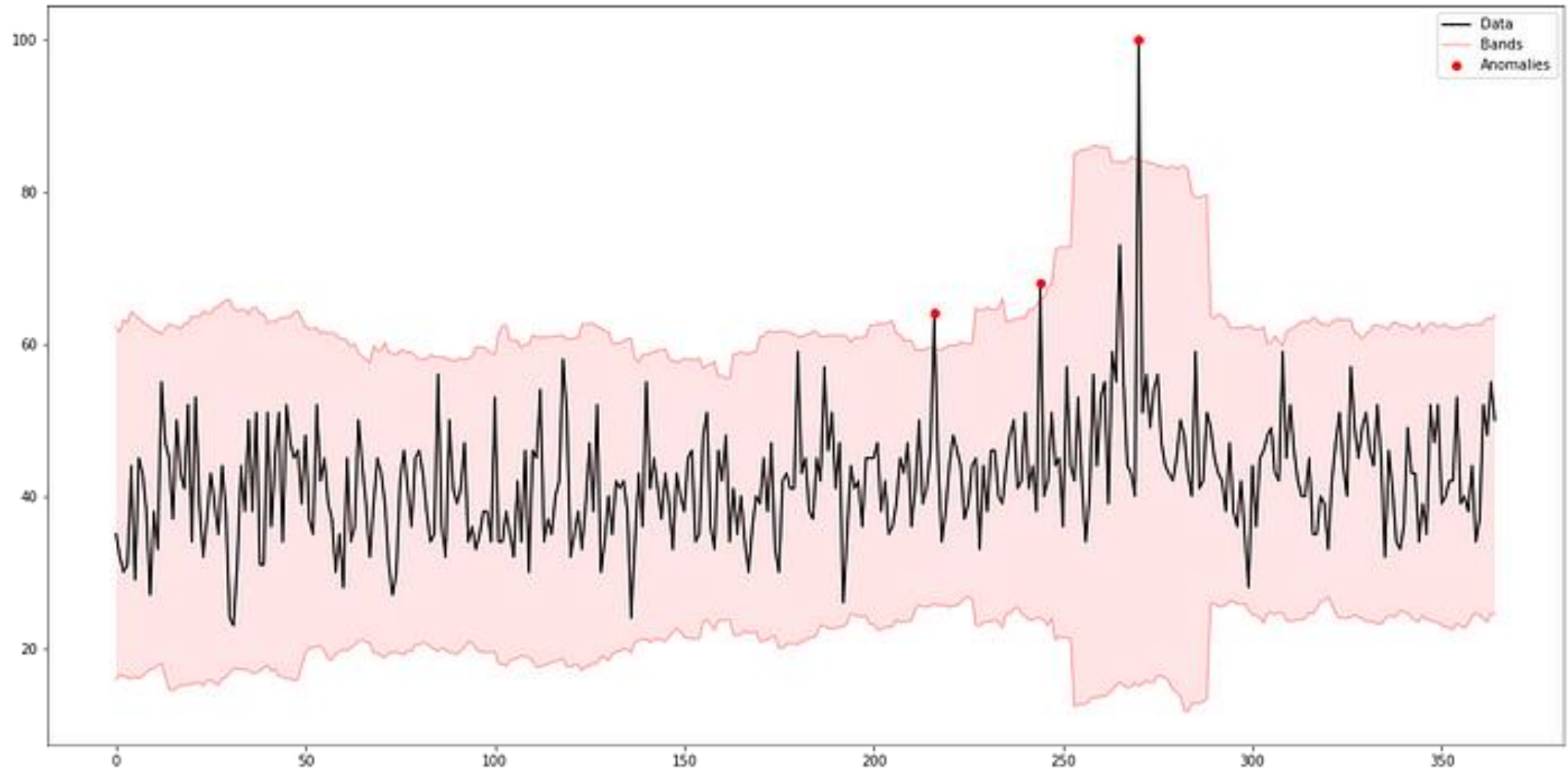
Time series outlier detection



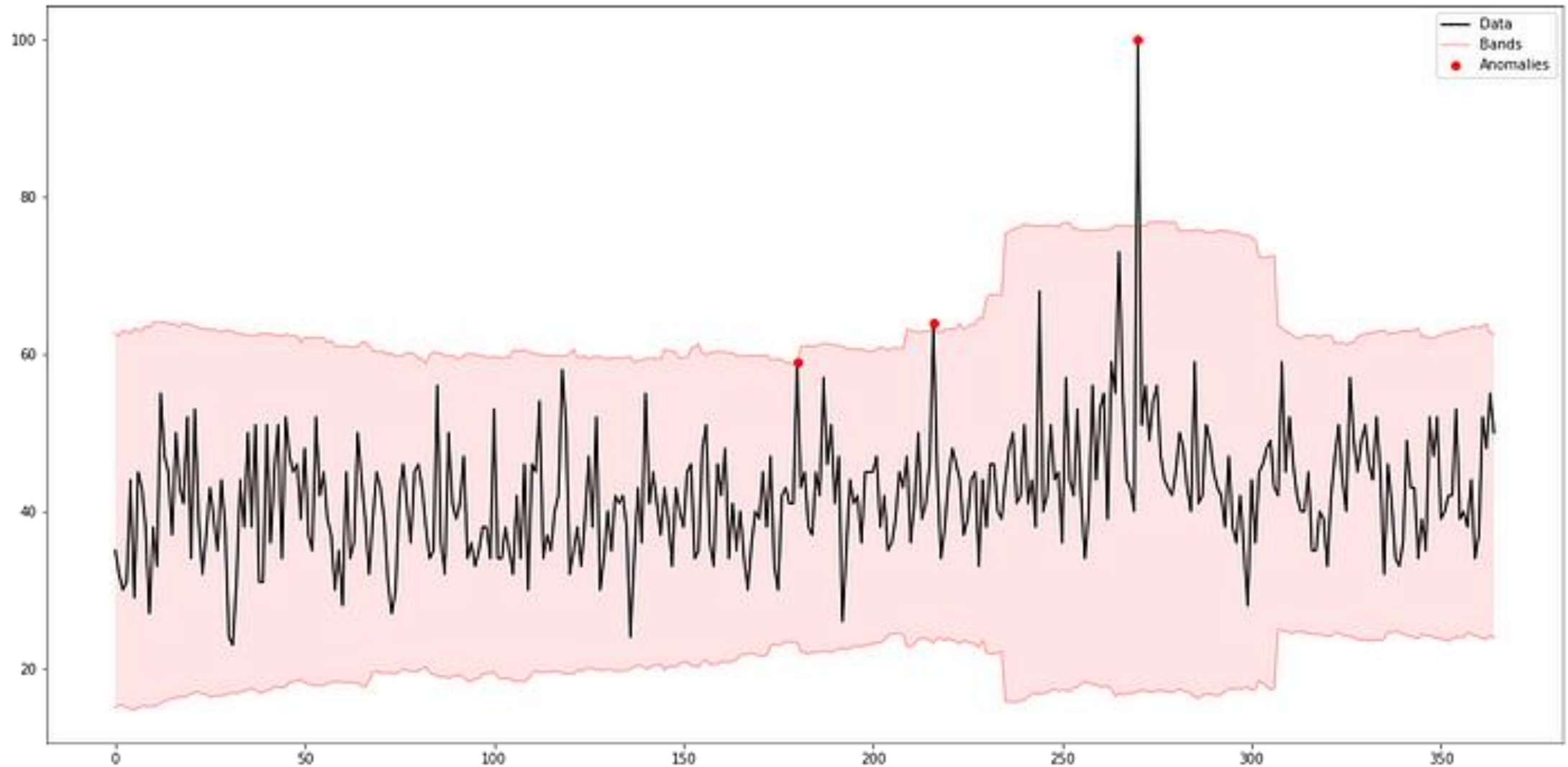
Time series outlier detection



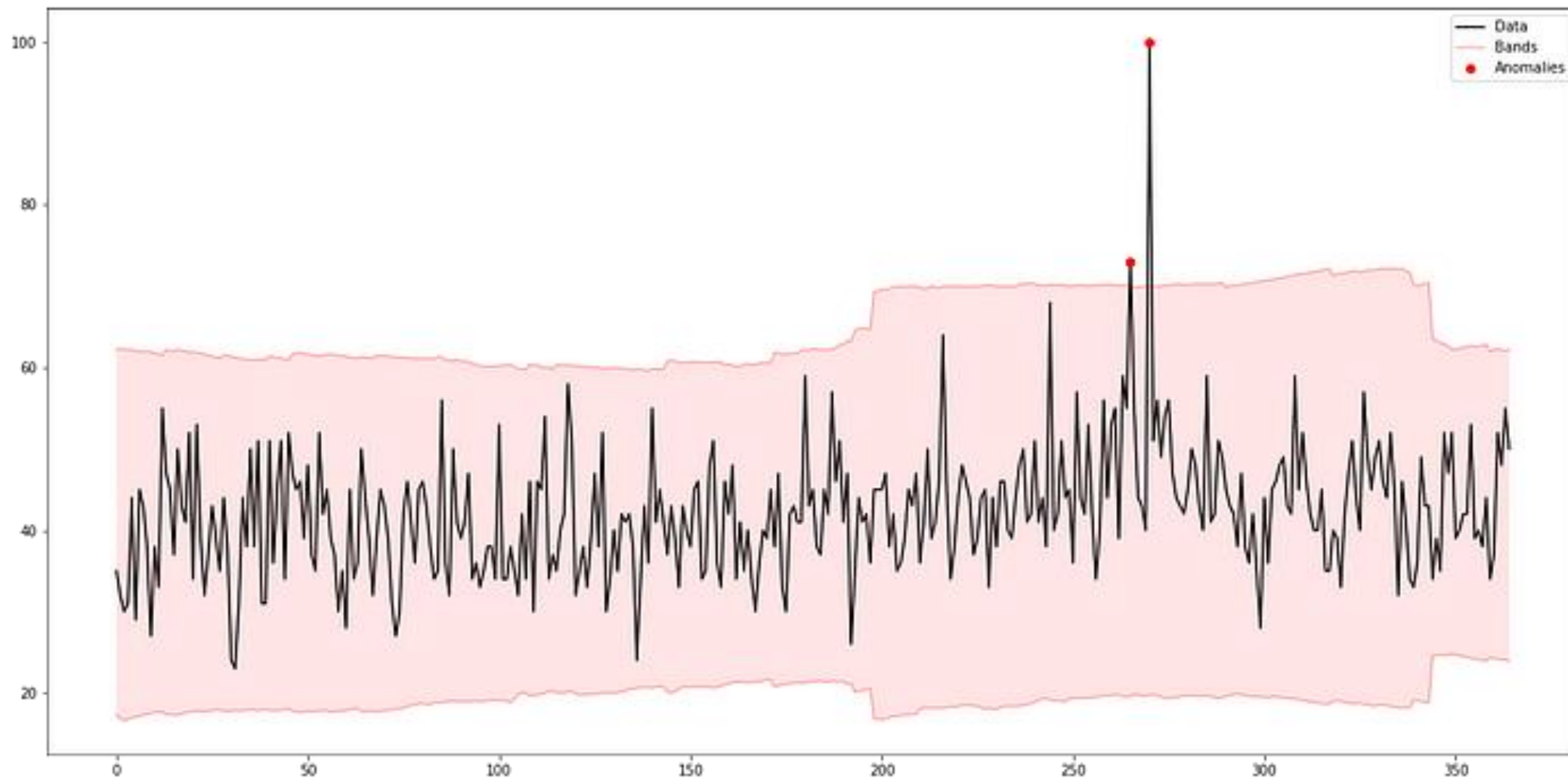
Time series outlier detection



Time series outlier detection



Time series outlier detection



Time series outlier detection

```
data = pd.read_csv('daily-total-female-births.csv')
data['Births'][270] = 100
column = data['Births']
column.plot(figsize=(20,10))
```

```
window_percentage = 3
k = int(len(column) * (window_percentage/2/100))
N = len(column)
```

```
get_bands = lambda data : (np.mean(data) + 3*np.std(data), np.mean(data) - 3*np.std(data))
```

```
get_bands = lambda data : (np.mean(data) + np.nanquantile(data,0.98), np.mean(data) - np.nanquantile(data,0.98))
```

```
bands = [get_bands(column[range(0 if i - k < 0 else i-k , i + k if i + k < N else N)])
          for i in range(0,N)] upper, lower = zip(*bands)
```

```
anomalies = (column > upper) | (column < lower)
```

Image data preprocessing

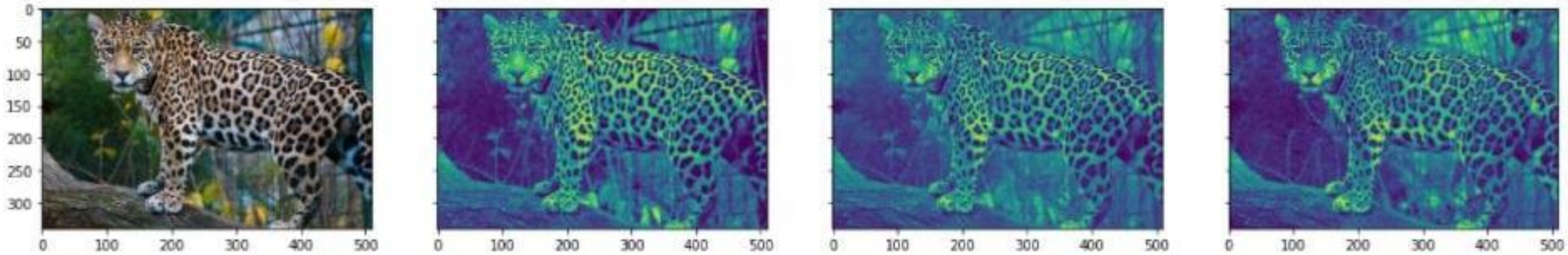
Grayscale conversion

Normalisation

Data Augmentation

Image standardization

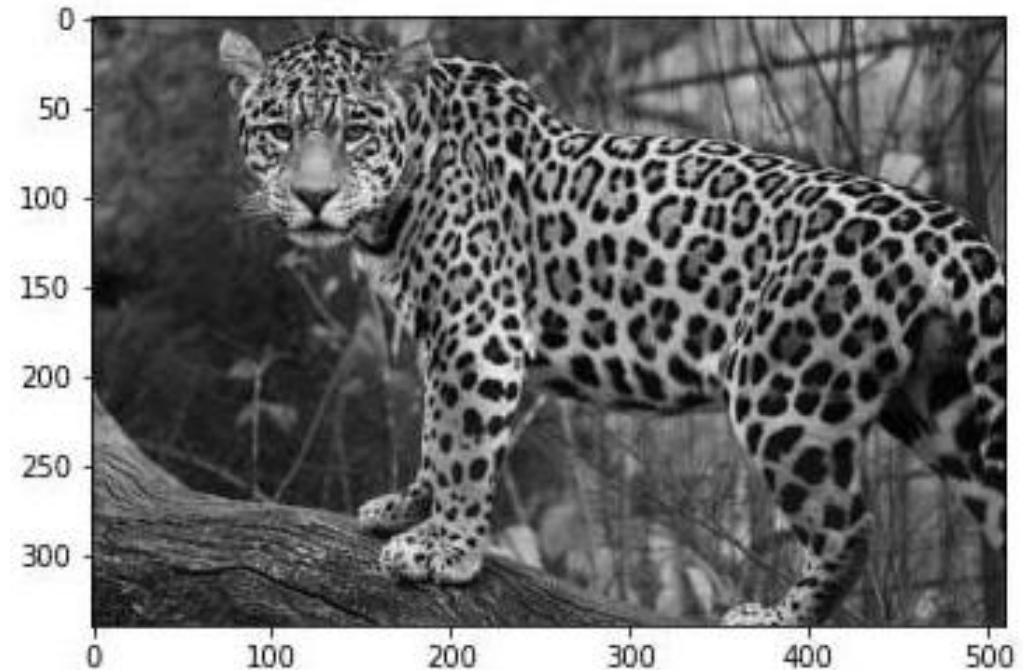
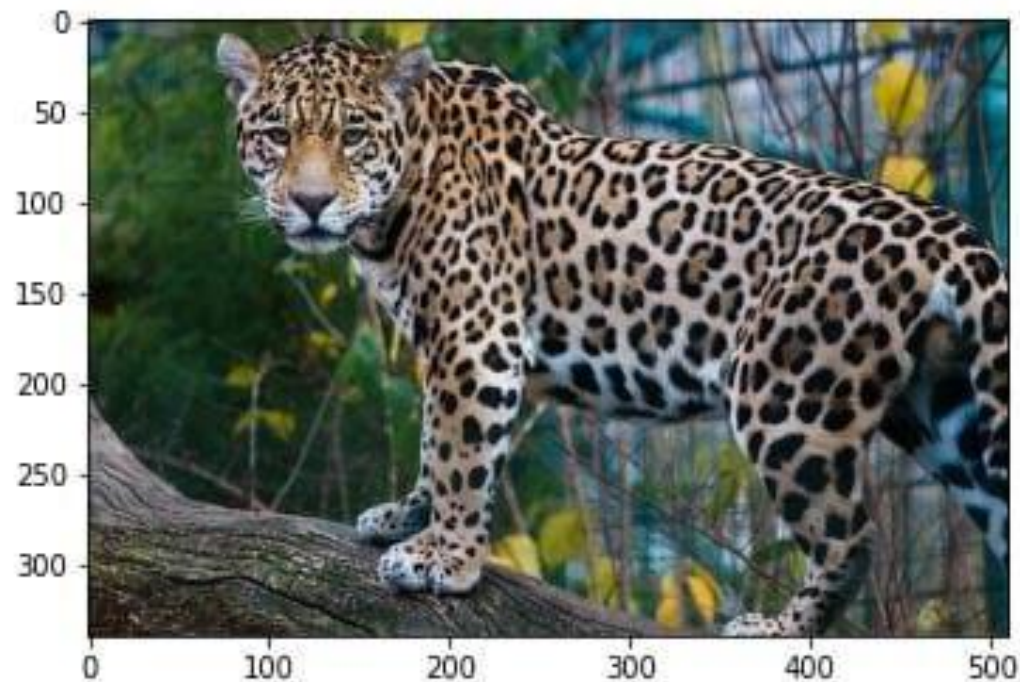
Image data preprocessing: Grayscale



```
i, (im1, im2, im3, im4) = plt.subplots(1, 4, sharey=True)  
i.set_figwidth(20)
```

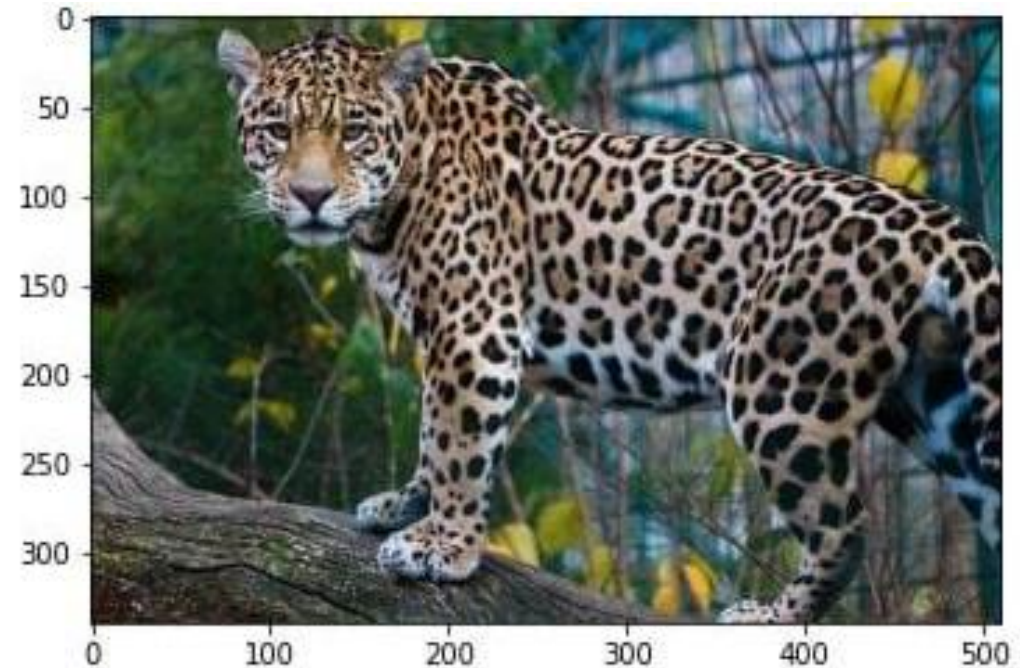
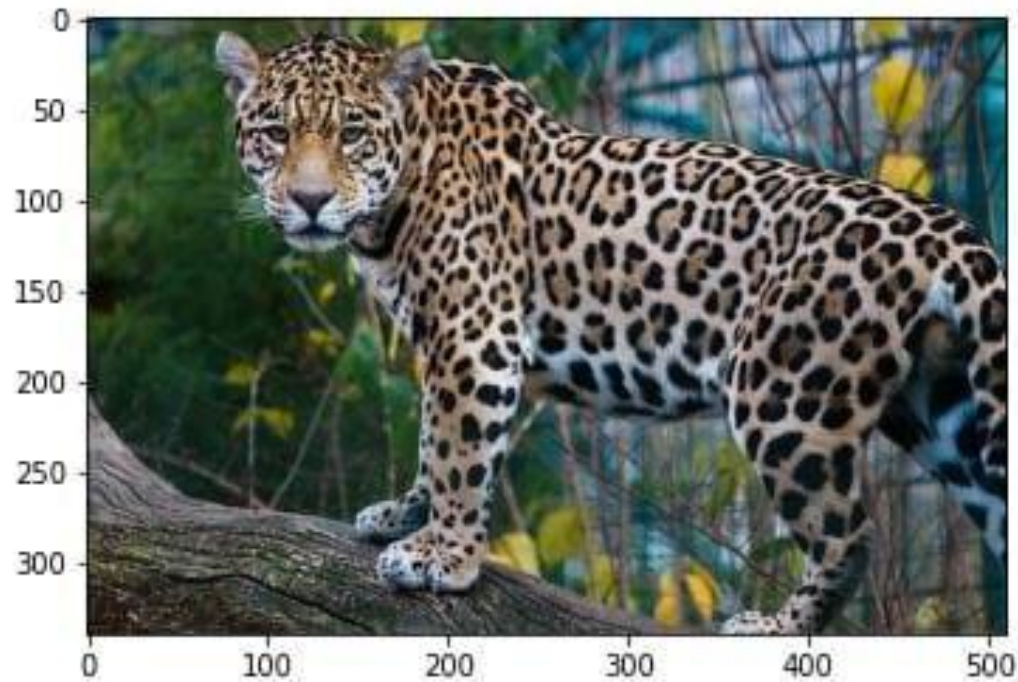
```
im1.imshow(image) #Original image  
im2.imshow(image[:, :, 0]) #Red  
im3.imshow(image[:, :, 1]) #Green  
im4.imshow(image[:, :, 2]) #Blue
```

Image data preprocessing : Grayscale



```
gray_image = skimage.color.rgb2gray(image)  
plt.imshow(gray_image, cmap = 'gray')
```

Image data preprocessing: Normalization



```
norm_image = (gray_image - np.min(gray_image)) / (np.max(gray_image) - np.min(gray_image))  
plt.imshow(norm_image)
```

Image data preprocessing: Augmentation

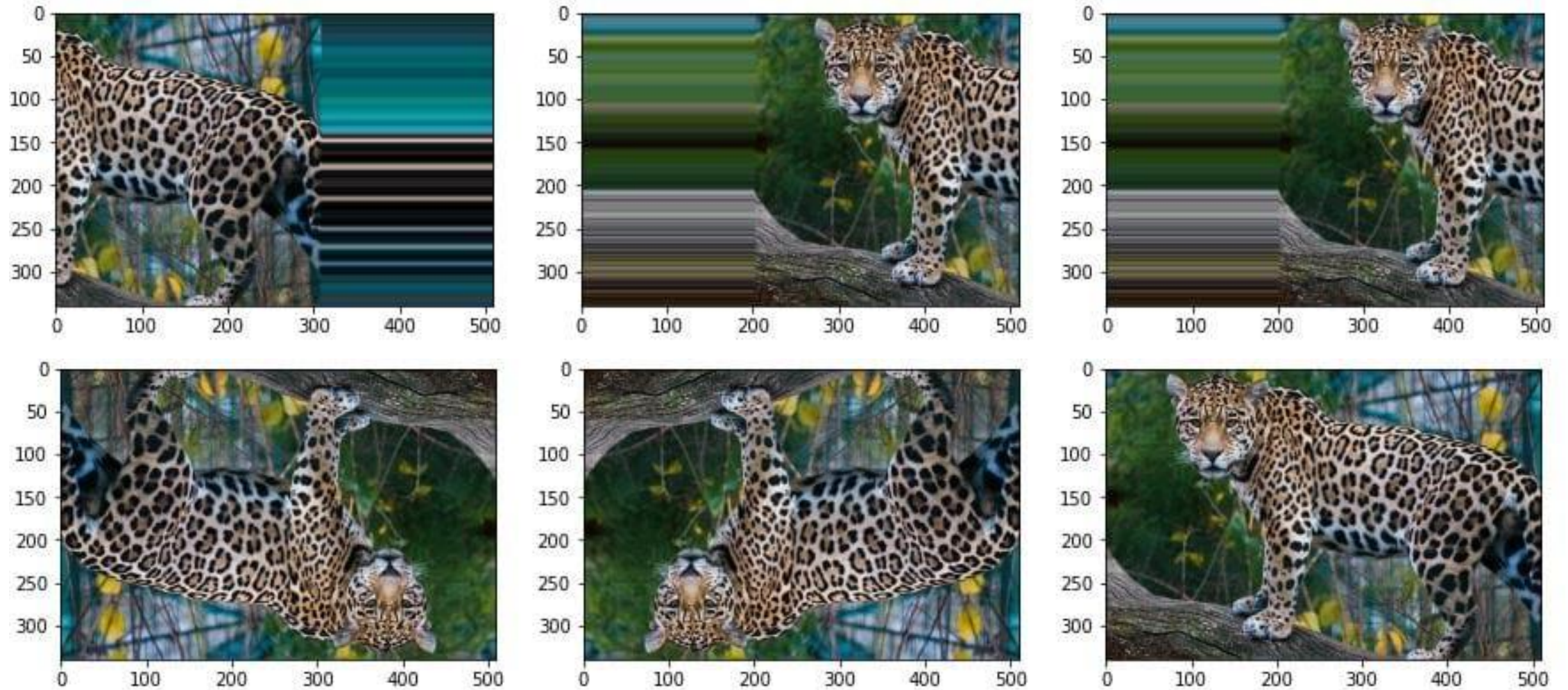


Image data preprocessing : Augmentation

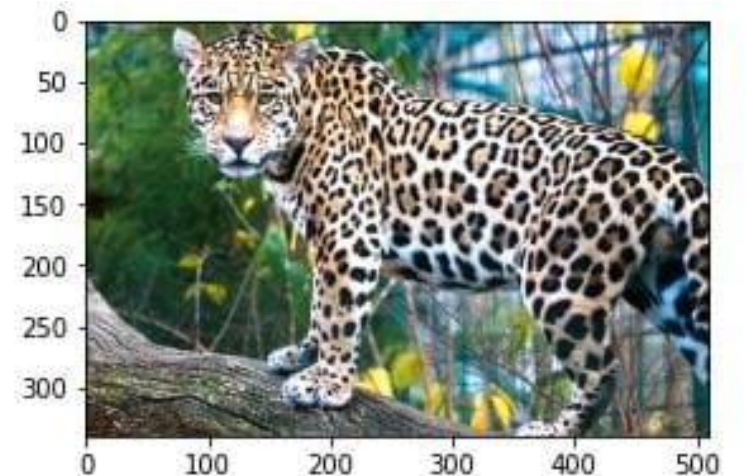
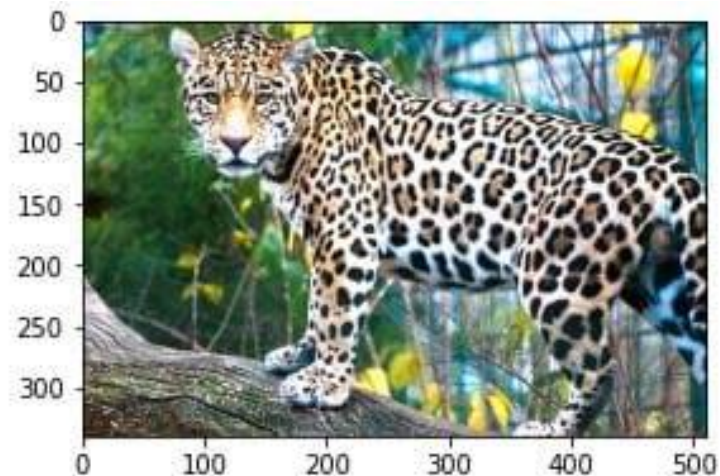
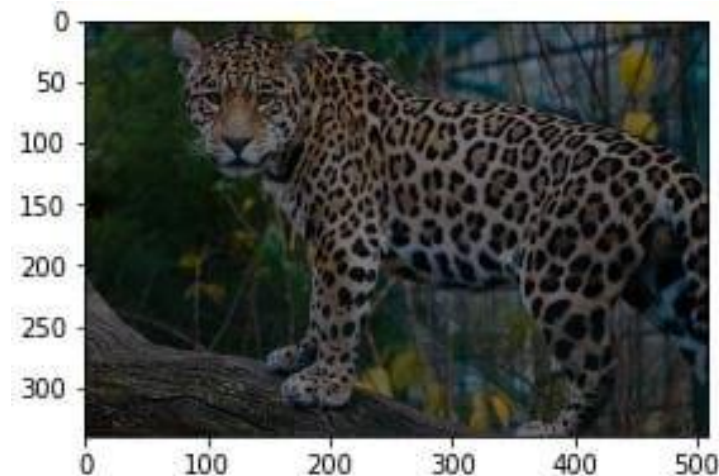
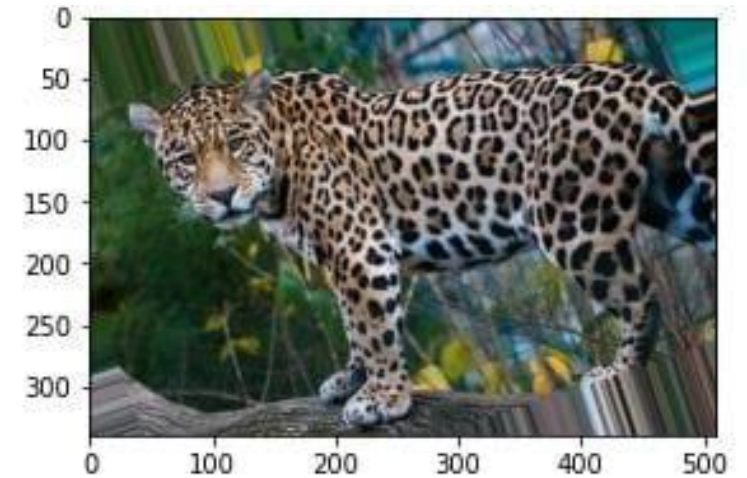
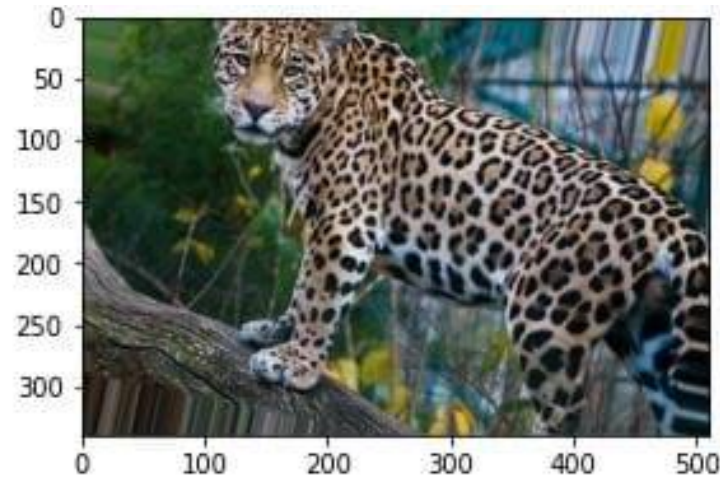
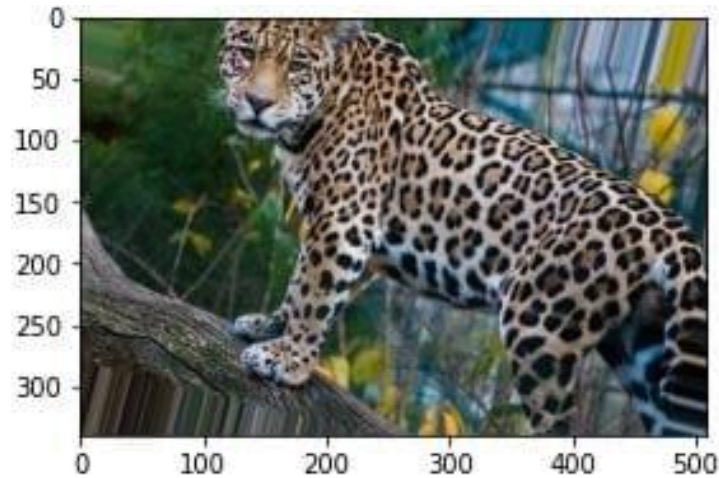


Image data preprocessing : Augmentation

```
from numpy import expand_dims
from keras.preprocessing.image import load_img
from keras.preprocessing.image import img_to_array
from keras.preprocessing.image import ImageDataGenerator

data = img_to_array(image) # convert to numpy array
samples = expand_dims(image, 0) # expand dimension to one sample
datagen = ImageDataGenerator(width_shift_range=[-200,200]) # create image data augmentation generator

# create an iterator
it = datagen.flow(samples, batch_size=1)
fig, im = plt.subplots(nrows=1, ncols=3, figsize=(15,15))

# generate batch of images
for i in range(3):
    image = next(it)[0].astype('uint8') # convert to unsigned integers
    im[i].imshow(image) # plot image

datagen = ImageDataGenerator(horizontal_flip=True, vertical_flip=True) # ImageDataGenerator for flipping
datagen = ImageDataGenerator(rotation_range=20, fill_mode='nearest') # ImageDataGenerator for rotation
datagen = ImageDataGenerator(brightness_range=[0.5,2.0]) # ImageDataGenerator for changing brightness
```

Image data preprocessing : Augmentation

- More augmentation
 - Cropping
 - Scaling

Image data preprocessing : Standardization

- Standardization is a method that scales and preprocesses images to have similar heights and widths.
- It re-scales data to have a standard deviation of 1 (unit variance) and a mean of 0.
- This is done to improve the quality and consistency of the data

creating the image data generator to standardize images

```
datagen = ImageDataGenerator(featurewise_center=True, featurewise_std_normalization=True)
```

Further reading

- [Top 8 Most Useful Anomaly Detection Algorithms For Time Series And Common Libraries For Implementation](#)

Thank you !!
Questions ??